



中國人民大學
RENMIN UNIVERSITY OF CHINA

融合FlagOS的智能计算课程建设

中国人民大学 王晶

jwang@ruc.edu.cn

2026年8月8日



我做过什么

- 关于国产生态
- 华为
 - 教育部-华为智能基座优秀教师（每年全国20人）
 - 教育部-华为智能基座虚拟教研室学科协作组秘书处核心成员，联合工作组教研与教学方法组副组长
- 曙光
 - 全国大学生系统能力大赛先导杯技术委员会委员
- 寒武纪
 - 《智能计算系统》课程合作单位
- 智源
 - 2025年科研的交流，碰撞了教学的火花
 - 2026年3月FlagOS引入本科生必修课
 - 2026年5月学生参加算子赏金赛天津站第一名



我遇到的问题

- 2023年第一次开课
 - 2020年计划开课，加入培养方案
 - 大三下选修课
 - 2023年选课学生个位数
- 2024年
 - 上课人数大于选课人数（负担重）
 - 增加苏州校区研究生课
- 2025年
 - 人工智能学院必修课
 - 赛教融合教改，学生竞赛全国一等奖
 - 选课人数80
- 2026年
 - 加入计算机必修课
 - 选课人数达到三位数
 - 北京市优秀本科教学资源，实验案例一等奖

GPT横空出世

SORA横空出世

DeepSeek横空出世

Agent席卷全球



我要讲什么

01 课程建设背景

02 国内外课程情况

03 AI技术栈和课程建设思路

04 硬件体系结构与 AI 芯片

05 AI 编译和算子优化

06 并行计算和分布式训练

07 课程思政



应该培养什么样的人才？

两个参考问题

- 汽车专业应该培养什么样的人才？
 - 同济大学汽车专业培养目标：具有从事汽车、发动机和汽车电子的研究、设计、制造及汽车营销和物流、车身与空气动力学、汽车试验学等方面工作的基本能力
- 计算机专业应该培养什么样的人才？
 - 计算机专业当培养计算机整机或子系统的设计者和研究者

人工智能方向应该培养人工智能
(子) 系统的设计者和研究者



一些思考

9月26日播出的Podcast节目《Bg2 Po
d》中，英伟达CEO黄仁勋发表观点称，“中
国落后美国几纳秒，所以我们必须去竞争。”
这一言论迅速引发业界广泛讨论。作为全球AI
芯片领域的领军者，黄仁勋公开肯定了中国芯
片的发展潜力，打破了传统以工艺节点衡量差
距的框架，转而用时间单位暗示动态竞争中的
微小间距。

纳秒必争

原创 人大观澜 长安街知事 2025年10月19日 23:39

北京 830人 星标



在全球科技产业战场，芯片已成为衡量国家科技
硬实力、产业话语权的核心标尺。英伟达CEO黄
仁勋“中国芯片制造仅落后美国几纳秒”的判断，
既点出了全球芯片竞争的极致激烈，更让人们关
注到这场较量的本质——它是涵盖芯片设计制
造、软件生态构建、人才储备培育的全方位系统
性竞争。

中国芯片产业的“纳秒级追赶”，正在三大核心战
场中以突破求生存、以创新谋发展。

根基之战

芯片设计与制造的技术突围

芯片产业的竞争，首先是“硬技术”的较量。从电
路图纸到物理芯片的转化核心是设计与制造两大
技术密集型环节，中国芯片产业的追赶之路，就
开始于这两个环节的突破。

在设计环节，中国
“差异化创新”的
“源头”，决定了芯
从系统架构到物理
尖的技术积累与工
历了从“依赖国外”
历程。

命脉之战

芯片软件生态的闭环构建

硬件是芯片的“骨架”，软件生态则是其“灵魂”。
全球芯片产业的竞争史早已证明：谁掌握了软件
生态的主导权，谁就能在芯片竞争中占据主动。

本源之战

芯片人才储备的规模与质量

芯片产业是“智力密集型”产业。中国芯片产业的
“纳秒级追赶”，本质上是人才储备的“规模追赶”
与“质量追赶”。

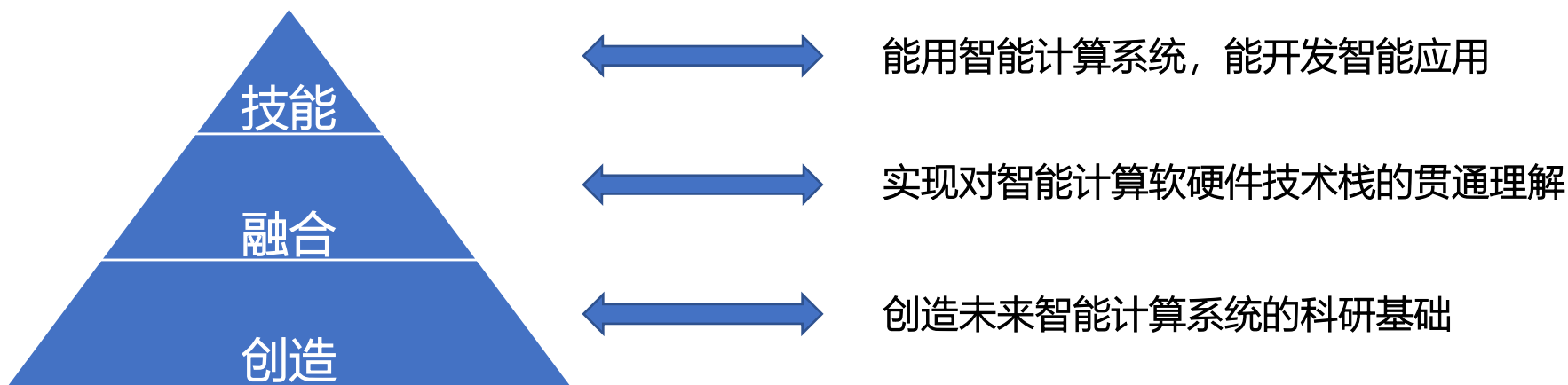
中国芯片产业人才必须实现“规模突围”。21世纪
初，国内高校芯片相关专业的培养能力有限，国
外芯片企业拥有高薪优势，导致本土人才不断外
流，中国芯片产业面临严重的“人才荒”。后来，
中国从“扩大招生规模”“完善培养体系”等方向入
手，增设“集成电路科学与工程”一级学科，人才
供给的“规模短板”得到初步缓解。

“从无到有”的深刻建
硬件研发，忽视了软
的操作系统、开发工
问题，中国企业与科
与“开发平台构建”着
的发展建设。



课程目标和目的

- 中国需要一大批智能基础设施的开发者和设计者
- 专业普及课程
 - 应用驱动，全栈贯通
- 智能计算系统
 - 建立智能计算系统设计及应用的知识体系
 - 掌握智能应用开发的基本技能
 - 培养开展智能计算系统基础研究的兴趣和能力





计算机专业培养计划的启示



研究生入学考试统考科目408
四门考研课程



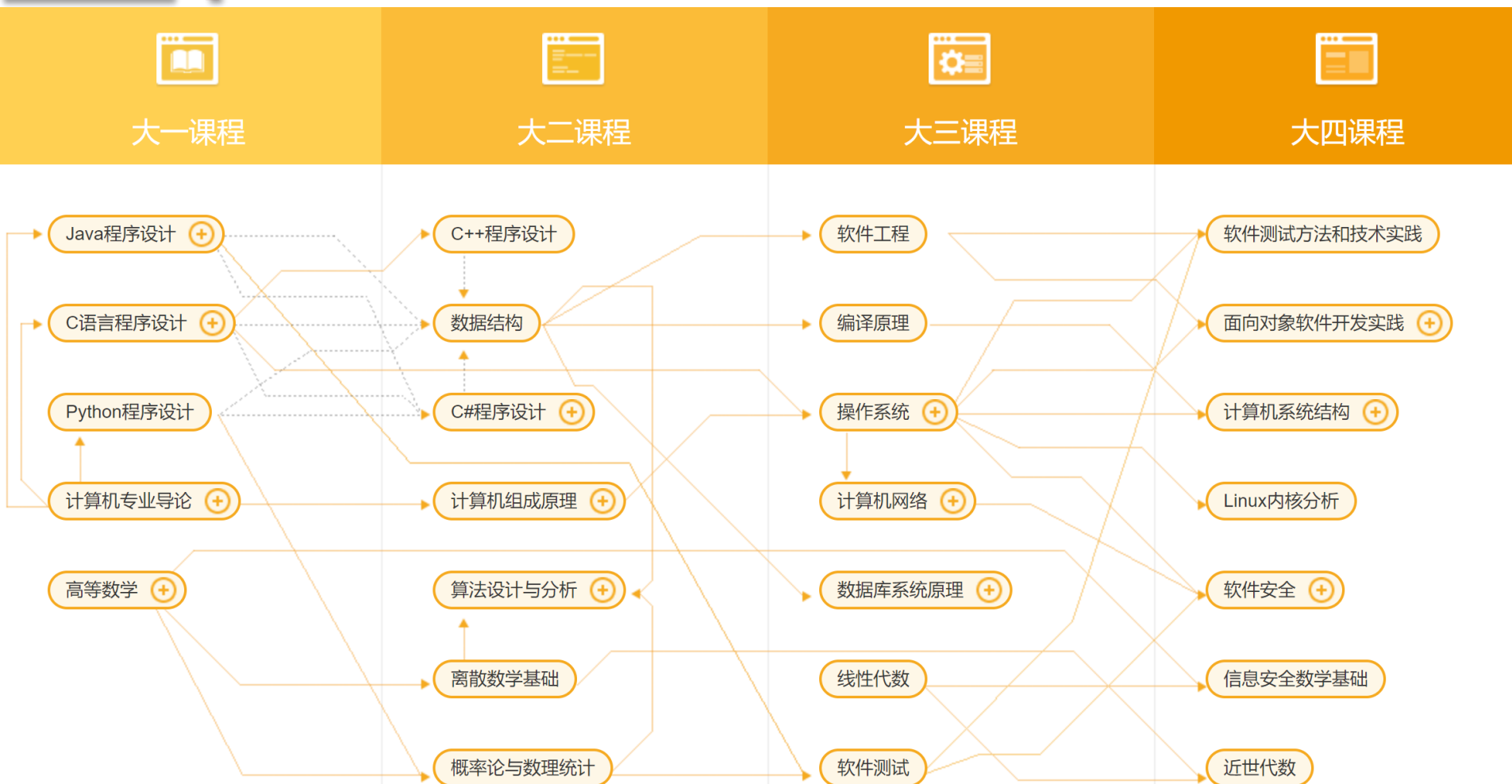
计算机专业培养计划的启示



计算机系统课程核心（全国系统能力大赛赛道）



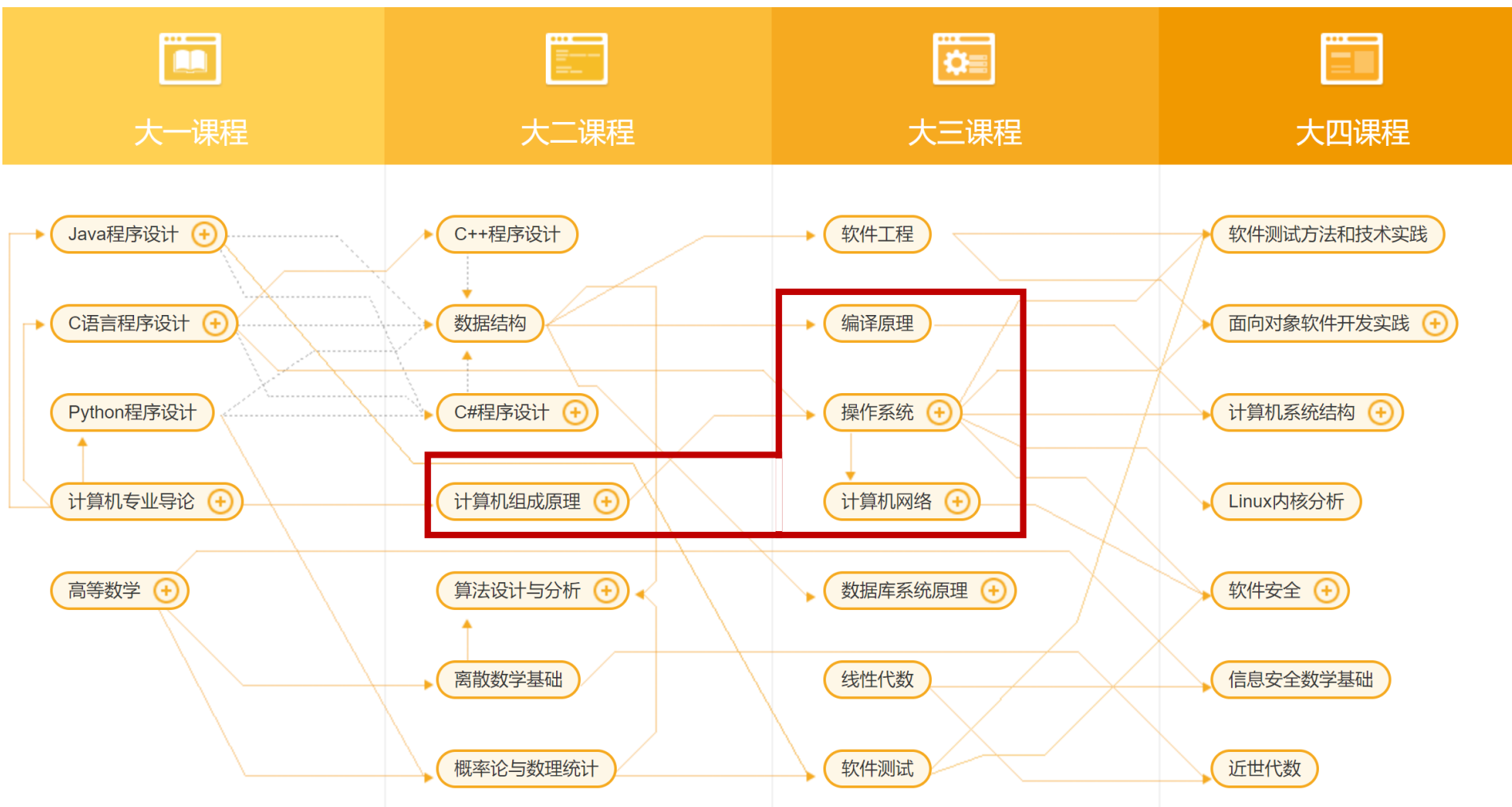
计算机专业培养计划的启示



课程条块分割，学生不能融会贯通做出一个完整系统，导致我国信息产业全栈式人才缺乏，核心硬科技竞争力缺失



计算机专业培养计划的启示



计算机系统基础 ICS



智能计算能力图谱

- 从注重“程序设计”转变为注重“系统设计”
- 从培养“程序性能开发能力”转变为“系统性设计能力”

扩展能力



专业能力



基础能力

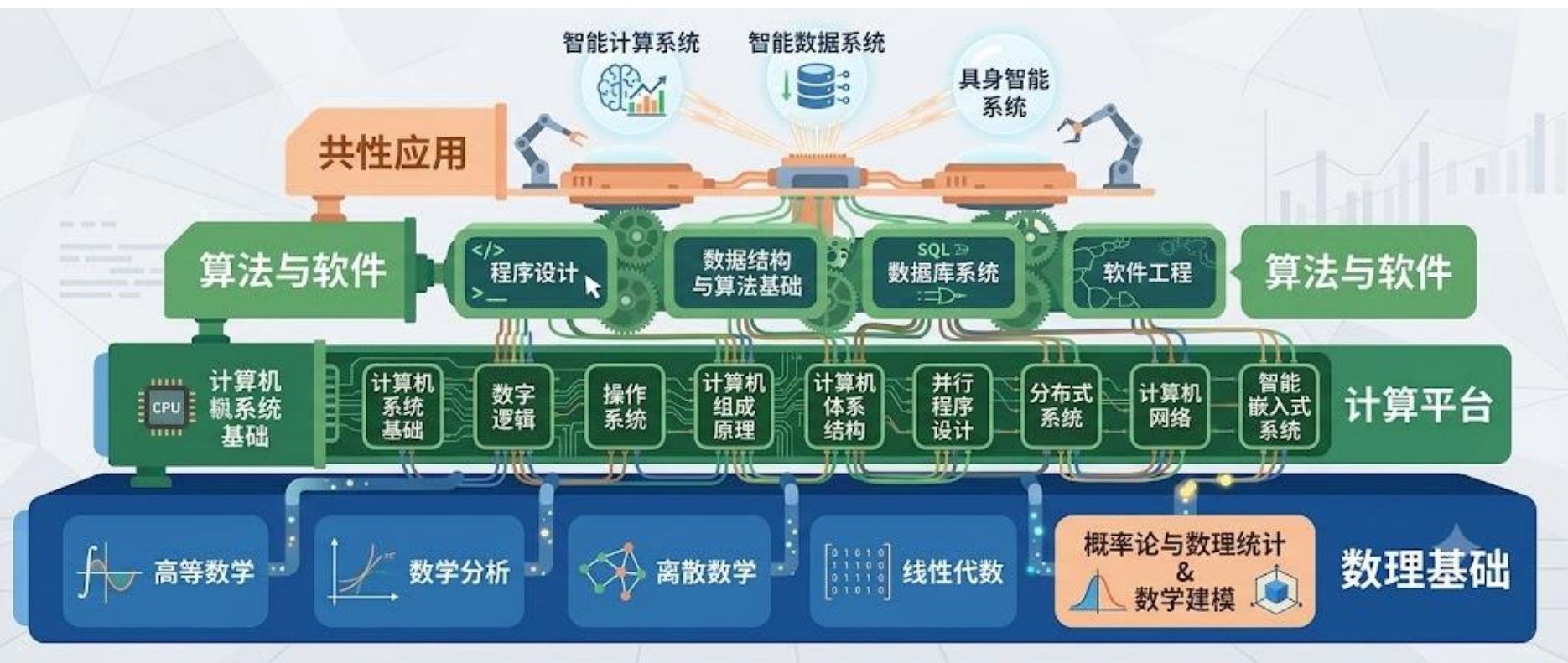


智能时代计算机专业系统能力培养纲要—计算机教指委，2021



智能计算核心课知识图谱

➤数理为基，系统为主，算法为核，开放为策



智能时代计算机专业系统能力培养纲要—计算机教指委，2021

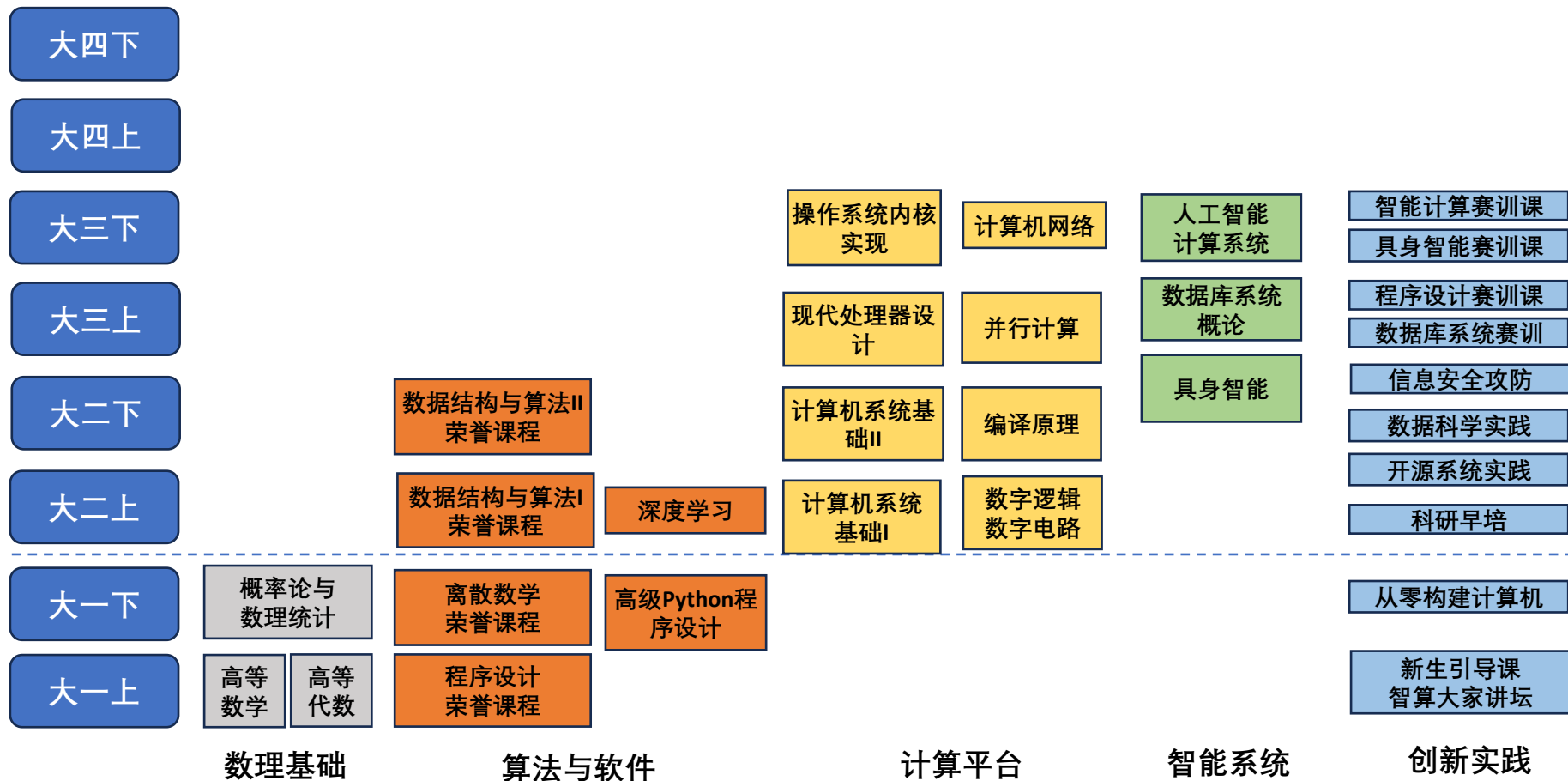


智能计算方向课程体系

101计划课程体系

长周期系统方向课程

研究实践教学





跨课程跨层次的课程改革方案

横向 跨课程

- 汇编语言程序设计和计算机组成指令集层面**贯通教学**
- 建立指令流驱动的**软硬件协同观念**



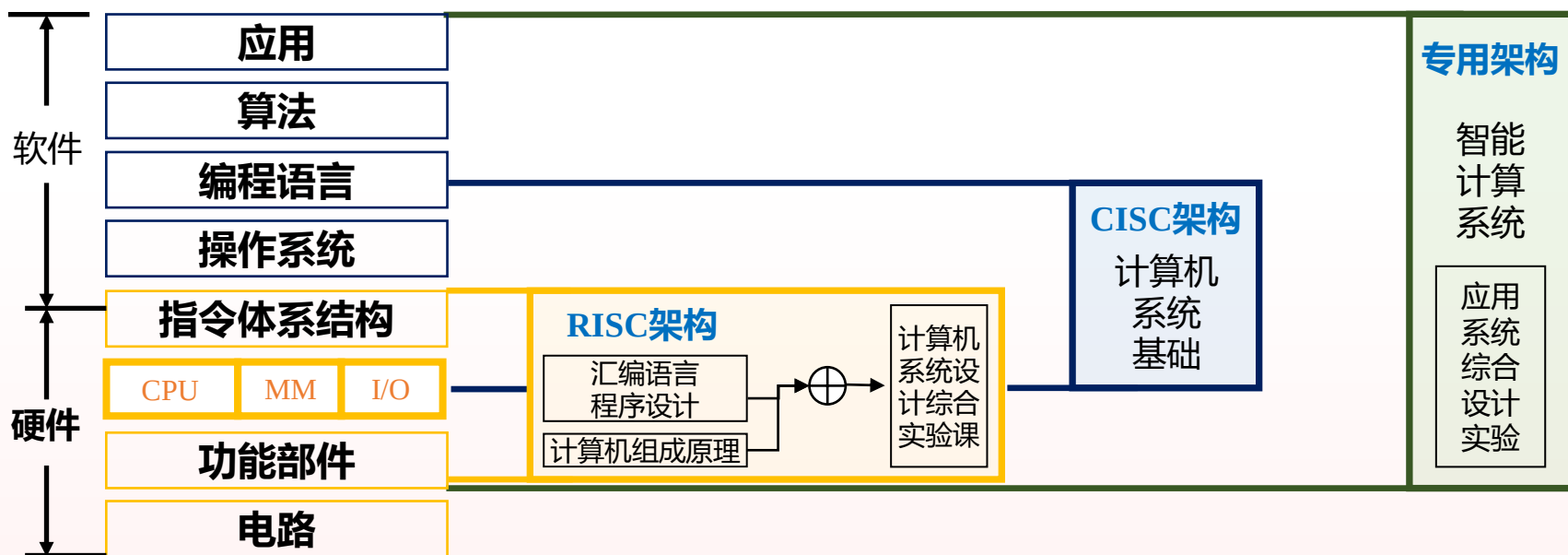
**重实践
强关联**

纵向 跨层次

- 从高级语言**贯通**到机器代码
- 建立从智能应用到功能部件的**系统观念**



**广综合
强系统**



能力递进
内容递进

基础系统能力
概念系统学习

核心技术能力
真实系统实现

综合应用能力
系统综合应用



贯通式课程建设

面向计算机系统能力培养的课程改革实践

二上

通用计算机

从高级语言贯通到机器代码

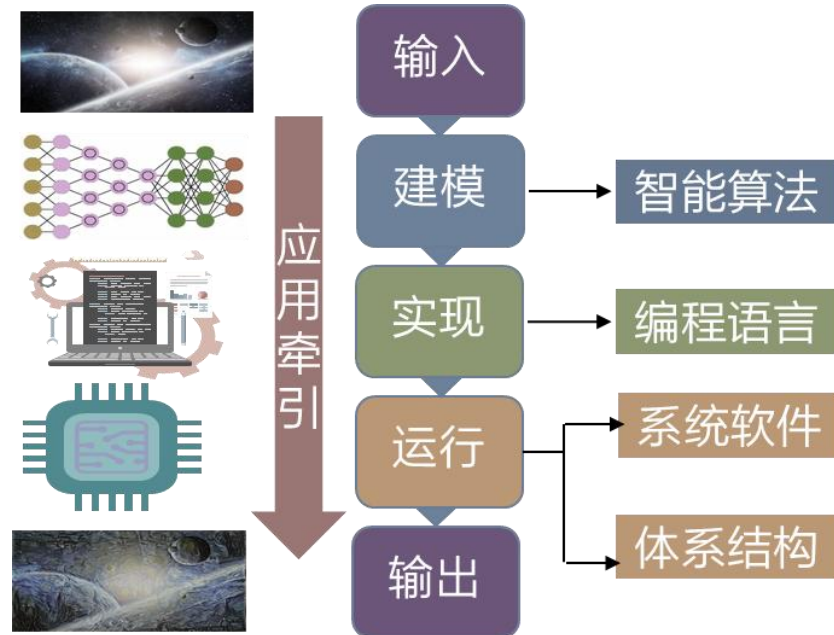
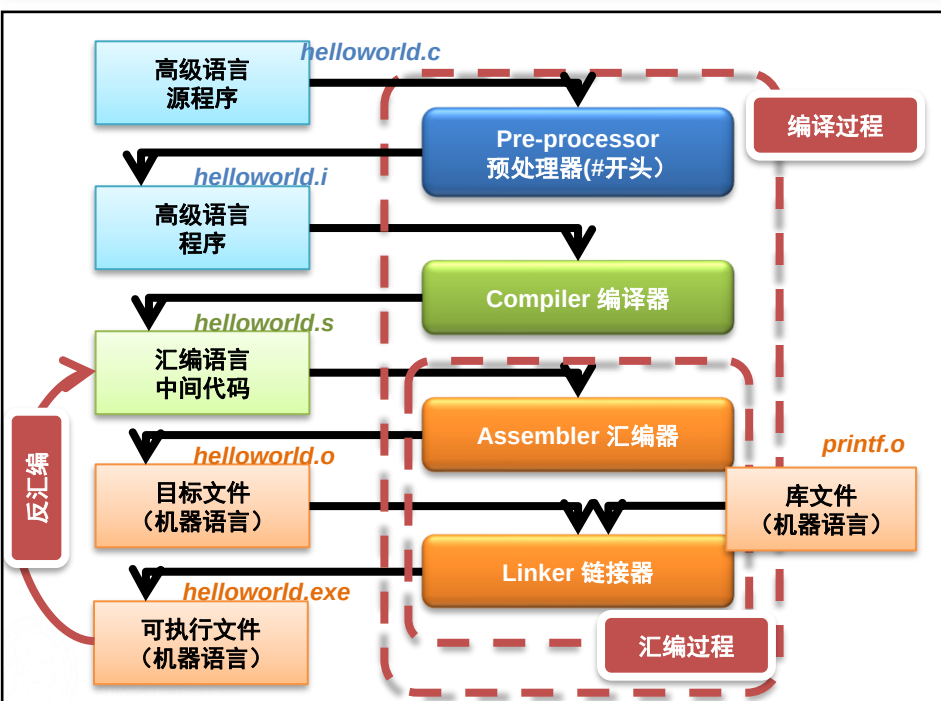
- 本科生必修课《计算机系统基础》
- 加深存储部分/编译链接部分

专用智能系统

三下

从智能应用贯通到体系结构

- 赛教融合：系统层优化打榜
- 基于人大定制, 全栈式, 应用驱动





自底向上全栈贯通的教学方案

❖ 2026年入选北京高校优质本科基本教学资源

AI 技术栈理论知识点

体系结构	底层硬件	国产/异构芯片：CPU / GPU / 昇腾 NPU / 海光 DCU / 寒武纪 MLU / FPGA			
		理论联系实践			
 异构与编译 (Compiler) 	编程编译	 前端优化  后端优化	数据格式 布局转换  代码优化 代码生成 	内存分配 算子循环优化	公共表达式 消除  令与内存 优化 
	底层通信	编层模型：CUDA / Ascend C / CANN		AI 编译器：TVM / XLA / TC 	
		 LLVM / GCC / HCCL / NCCL / 通信原语 / 集合通信算法 			

理论联系实践

实践方案创新

图形化 AI 加速单元硬件设计



4Bit 量化矩阵乘法
算子设计与实现



$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \div \begin{bmatrix} e & f \\ g & h \end{bmatrix}_x$$

面向矩阵乘与激活函数的
融合算子设计与实现



理论联系实践

 框架与算法 (Frameworks) AI Model	AI 框架 推理引擎 ✓ Optimization ✓ Optimization	中间表示 (IR) 前缀 API	编译优化 (Compilation)	图优化与 图块行调度	推理优化
		基本数据结构 张量 (Tensor)	词法 / 语法 语义分析	计算图动静统一	常量折叠 冗余节点消除
		基本计算单元 有向无环图 (DAG)	自动微分	计算图控制流	模型压缩 蒸馏 / 剪枝 / 量化



LLM

KV Cache 与 FlashAttention
联合优化

大模型推理效率提升实验



- | 2026年全国大学生计算机系统能力大赛
智能计算创新设计赛(先导杯) | | | | | | | | | | |
|---|--|--|--|--|--|--|--|--|--|--|
| 主办单位：工业和信息化部教育发展中心、国家网信办教育研究专家室、国家网信办教育研究项目发展院 | | | | | | | | | | |
| 承办单位：经济大学、烽火技术产业集团信息事业部 | | | | | | | | | | |
| 协办单位：机械工业出版社 | | | | | | | | | | |
| 支持单位：CGF 赛翼平台 2025 光电信息 SCMA SCMT 共同发起者社区 | | | | | | | | | | |

2026年全国大学生计算机系统能力大赛-智能计算创新设计赛（先导杯）											
© 基于国产加速卡的QueenA大模型推理服务性能、初赛										参赛队伍 245	
初赛 2025.12.05										2025.12.05	
初赛										排名	
基于国产加速卡的QueenA大模型推理服务性能-初赛											
比赛数据来源于烽火技术产业集团信息事业部											
#	参赛队伍	队名	学校	提交记录 (ASC)	最终提交时间 (UTC+8)	16K- 32K 实验得分	44K-64K 实验得分	64K-102K 实验得分	SLA得分	最终得分	排名
1	T2026100509911438	了不起的队	南京大学	111	2026-07-15 07:04:25	16.78	21.75	19.84	0	89.8704	10.1296
2	T2026100509911438	了不起的队	南京大学	111	2026-07-15 09:58:44	16.78	21.75	19.84	0	89.8704	10.1296
3	T2026100509912163	CUYUE	中国人民大学	63	2026-07-15 14:21:33	16.11	22.02	19.91	0	89.7409	10.2691
4	T2026100509912115	北航A08	北京航空航天大学	14	2026-07-15 02:17:56	16.87	21.78	19.32	0	89.6989	10.3011
5	T2026102879412460	蓝谷F4	南京信息工程大学	101	2026-07-15 03:26:12	16.03	21.78	19.64	0	89.6828	10.3372
6	T20261008159912016	罗蒙战队	南京邮电大学	132	2026-07-15 07:27:26	16.87	20.44	19.78	0	89.3228	10.8722
7	T20261008159912016	罗蒙一队	南京邮电大学	140	2026-07-15 09:03:28	16.72	20.76	19.68	0	89.2801	10.7619
8	T2026100029912285	Tiktok创意	中国人民大学	113	2026-07-15 14:49:48	16.98	21.34	19.21	0	89.1751	10.8249
9	T20261045099118178	天气晴晴	郑州大学	103	2026-07-15 10:33:56	13.91	21.52	19.96	0	89.1668	10.8432
10	T20261046912473	洪城战队	江西理工大学	67	2026-07-15 11:12:16	16.01	20.89	19.2	0	89.1219	10.8791
11	T2026104794791213	coffeeater	同济大学	28	2026-07-15 14:18:11	16.3	20.68	19.23	0	89.1083	10.8917
12	T20261047947911005	这个个人真是麻烦呀！ 都是七边形的	华中科技大学	9	2026-07-15 10:17:48	19.39	20.89	19.42	0	89.0784	10.9276
13	T2026104229911958	赵良鑫的编程课！	山东大学（青岛）	95	2026-07-15 09:55:31	16.09	20.89	19.81	0	88.9826	11.0740
14	T2026101329911177	赵良鑫的编程课！ 爱爱爱爱爱	山东大学（青岛）	42	2026-07-14 21:28:30	16.64	21.02	19.85	0	88.9309	11.0920
15	T2026100509911288	西木	南京大学	182	2026-07-15 10:23:30	16.16	20.88	19.83	0	88.902	11.0980
16	T20261038799119122	牟宇	中国矿业大学北京	24	2026-07-15 07:03:12	16.94	20.41	19.17	0	88.860	11.1251
17	T20261045099114226	4204P	郑州大学	31	2026-07-15 09:43:58	14.86	20.94	19.29	0	88.7979	11.1387
18	T2026100029912488	李智超的编程名字才学	中国人民大学	96	2026-07-15 10:16:39	16.78	20.79	19.73	0	88.7033	11.1298
19	T2026104509911611	HYSC 王浩源战队	天津大学	121	2026-07-15						





提纲

01 课程建设背景

02 国内外课程情况

03 AI技术栈和课程建设思路

04 硬件体系结构与 AI 芯片

05 AI 编译和算子优化

06 并行计算和分布式训练

07 课程思政



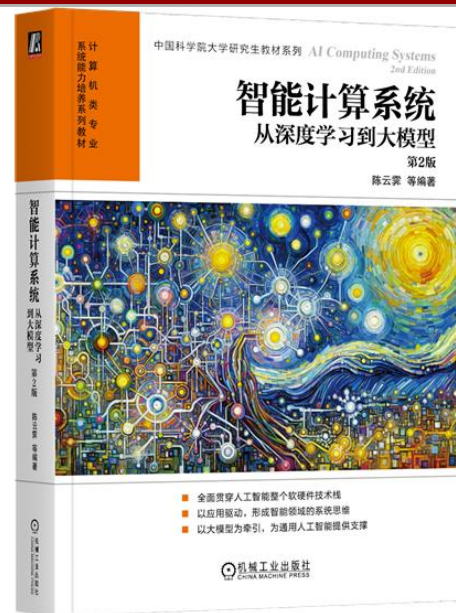
国内外相关课程

- 美国计算机方向Top4高校Stanford、CMU、UC Berkley和MIT以及多个国际单位联合发布了白皮书——“SysML: The New Frontier of Machine Learning Systems”
 - 包括Yann LeCun、Michael I. Jordan、Bill Dally和Jeff Dean等
- 中科院《智能计算系统》（<https://novel.ict.ac.cn/aics/>）
- 微软亚洲研究院《人工智能系统》
（<https://github.com/microsoft/AI-System>）
- MIT6.5940《TinyML and Efficient Deep Learning Computing》
（<https://hanlab.mit.edu/courses/2023-fall-65940>）
- CMU《Deep Learning Systems》（<https://dlsyscourse.org/>）
- UC Berkeley《Machine Learning Systems》
（<https://ucbsky.github.io/aisys-fa2024/schedule/>）



智能计算系统课程提纲

- 第一章 概述
- 第二章 深度学习基础
- 第三章 深度学习应用
- 第四章 编程框架使用
- 第五章 编程框架机理
- 第六章 面向深度学习的处理器原理
- 第七章 深度学习处理器架构
- 第八章 智能编程语言
- 第九章 大模型



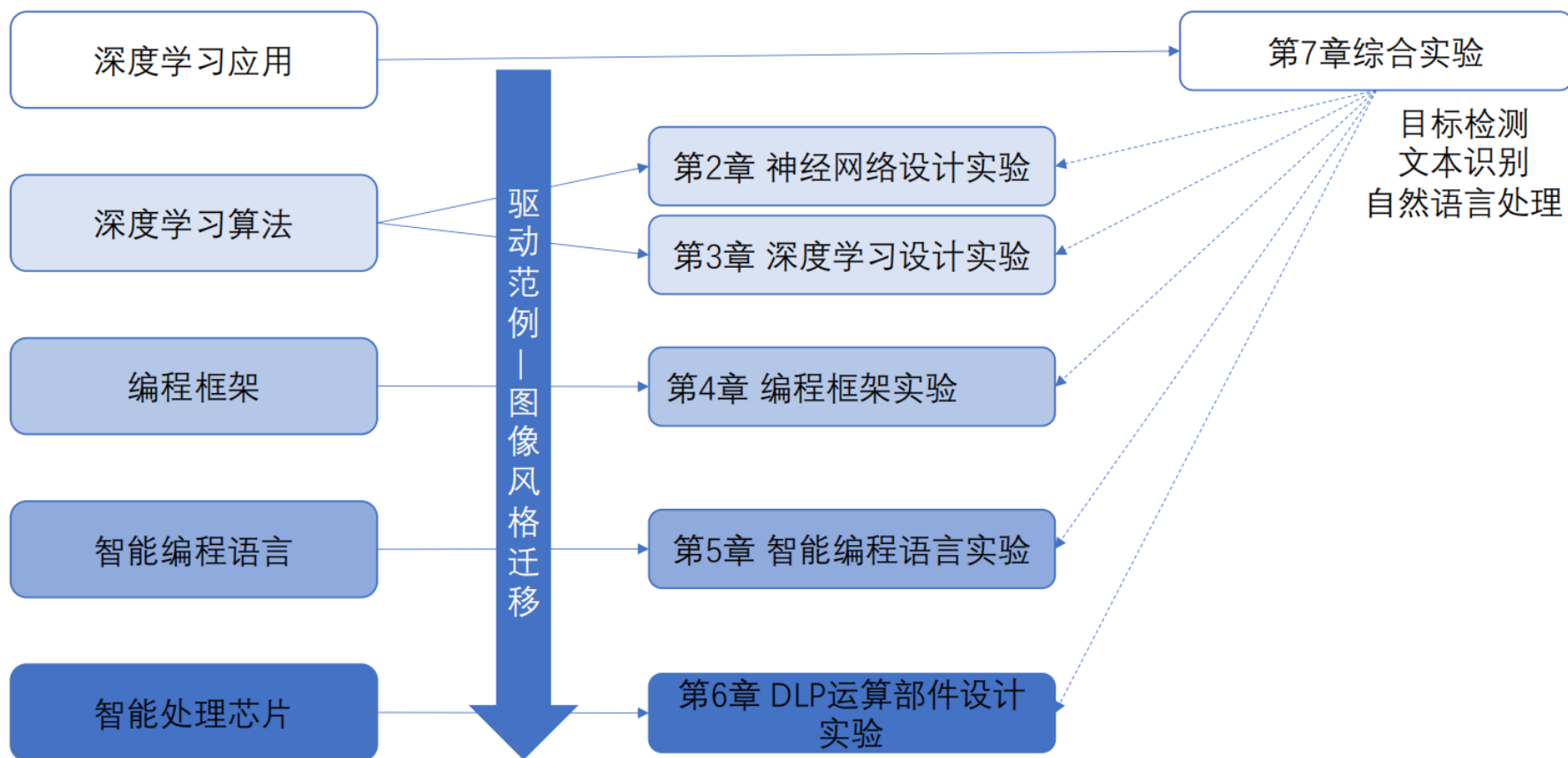


智能计算系统实验内容

软硬件技术栈

分阶段实验

综合实验





微软课程大纲 – 基础知识

课程	讲义名称	备注
1	课程介绍	Overview and system/AI basics
2	人工智能系统概述 System perspective of System for AI	System for AI: a historic view; Fundamentals of neural networks; Fundamentals of System for AI
3	深度神经网络计算框架基础 Computation frameworks for DNN	Backprop and AD, Tensor, DAG, Execution graph Papers and systems: PyTorch, TensorFlow
4	矩阵运算与计算机体系结构 Computer architecture for Matrix computation	Matrix computation, CPU/SIMD, GPGPU, ASIC/TPU Papers and systems: Blas, TPU
5	分布式训练算法 Distributed training algorithms	Data parallelism, model parallelism, distributed SGD Papers and systems:
6	分布式训练系统 Distributed training systems	MPI, parameter servers, all-reduce, RDMA Papers and systems: Horovod
7	异构计算集群调度与资源管理系统 Scheduling and resource management system	Running DNN job on cluster: container, resource allocation, scheduling Papers and systems: KubeFlow, OpenPAI, Gandiva, HiveD
8	深度学习推导系统 Inference systems	Efficiency, latency, throughput, and deployment



微软课程大纲（2） - 高级技术

课程	讲义名称	备注
9	计算图编译优化 Computation graph compilation and optimization	IR, sub-graph pattern match, Matrix multiplication and memory optimization Papers and systems: XLA, MLIR, TVM, NNFusion
10	模型压缩和稀疏化处理 Efficiency via compression and sparsity	Model compression, Sparsity Pruning
11	自动机器学习系统 AutoML systems	Hyper parameter tuning, NAS Papers and systems: Hyperband, SMAC, ENAS, AutoKeras, NNI
12	强化学习系统 Reinforcement learning systems	Theory of RL, systems for RL Papers and systems: AC3, RLlib, AlphaZero
13	模型安全与隐私保护 Security and Privacy	Federated learning, security, privacy Papers and systems: DeepFake
14	用AI技术优化计算机系统 AI for systems	AI for traditional systems problems, for system algorithms Papers and systems: Learned Indexes, Learned query path



微软课程大纲 (3) - 实验

课程	讲义名称	备注
Lab 1 (for week 1,2)	框架及工具入门示例 A simple throughout end-to-end AI example, from a system perspective	Understand the systems from debugger info and system logs
Lab 2 (for week 3)	定制一个新的张量运算 Customize operators	Design and implement a customized operator (both forward and backward): in python
Lab 3 (for week 4)	CUDA实现和优化 CUDA implementation	Add a CUDA implementation for the customized operator
Lab 4 (for week 5,6)	AllReduce实现和优化 AllReduce	Improve one of AllReduce operators' implementation on Horovod
Lab 5 (for week 7, 8)	配置Container来进行云上训练或推理准备 Configure containers for customized training and inference	Configure containers
Lab 6	学习使用调度管理系统 Scheduling and resource management system	Get familiar with OpenPAI or KubeFlow
Lab 7	分布式训练任务练习 Distributed training	Try different kinds of all reduce implementations
Lab 8	自动机器学习系统练习 AutoML	Search for a new neural networkNN structure for Image/NLP tasks
Lab 9	强化学习系统练习 RL Systems	Configure and get familiar with one of the following RL Systems: RLlib, ...



MIT



Song Han

Associate Professor, MIT
Distinguished Scientist, NVIDIA

<https://efficientml.ai>

@SongHan_MIT

9/7 Lecture 1: **Introduction**
[slides] [video] [video (live)]

9/12 Lecture 2: **Basics of Deep Learning**
[slides] [video] [video (live)]

Efficient Inference

9/14 Lecture 3: **Pruning and Sparsity (Part I)**
[slides] [video] [video (live)]

9/19 Lecture 4: **Pruning and Sparsity (Part II)** Lab 1 out
[slides] [video] [video (live)]

9/21 Lecture 5: **Quantization (Part I)**
[slides] [video] [video (live)]

9/26 Lecture 6: **Quantization (Part II)**
[slides] [video] [video (live)]

9/28 Lecture 7: **Neural Architecture Search (Part I)** Lab 1 due, Lab 2 out
[slides] [video] [video (live)]

10/3 Lecture 8: **Neural Architecture Search (Part II)**
[slides] [video] [video (live)]

10/5 Lecture 9: **Knowledge Distillation** Lab 3 out
[slides] [video] [video (live)]

10/10 Student Holiday — No Class

10/12 Lecture 10: **MCUNet** Lab 2 due
[slides] [video] [video (live)]

10/17 Lecture 11: **TinyEngine**
[slides] [video] [video (live)]

Efficient Transformer and Generative AI

10/19 Lecture 12: **Transformer and LLM (Part I)** Lab 3 due, Lab 4 out
[slides] [video] [video (live)]

10/24 Lecture 13: **Transformer and LLM (Part II)**
[slides] [video] [video (live)]

10/26 Lecture 14: **Vision Transformer**
[slides] [video] [video (live)]

10/31 Lecture 15: **GAN, Video, and Point Cloud** Lab 4 due, Lab 5 out
[slides] [video] [video (live)]

11/2 Lecture 16: **Diffusion Model**
[slides] [video] [video (live)]

Efficient Training and System Support

11/7 Lecture 17: **Distributed Training (Part I)**
[slides] [video] [video (live)]

11/9 Lecture 18: **Distributed Training (Part II)** Lab 5 due
[slides] [video] [video (live)]

11/14 Lecture 19: **On-Device Training (Part I)** Project proposal due
[slides] [video] [video (live)]

11/16 Lecture 20: **On-Device Training (Part II)**
[slides] [video] [video (live)]

Quantum ML

11/21 Lecture 21: **Basics of Quantum Computing**
[slides] [video] [video (live)]

11/23 Thanksgiving — No Class

11/28 Lecture 22: **Quantum Machine Learning**
[slides] [video] [video (live)]

11/30 Lecture 23: **Noise Robust Quantum ML**
[slides] [video] [video (live)]

12/5 Lecture 24: **Final Project Presentation**
[slides] [video] [video (live)]

12/7 Lecture 25: **Final Project Presentation**
[slides] [video] [video (live)]

12/12 Lecture 26: **Course Summary & Guest Lecture** Final report due
[slides] [video] [video (live)]



UC Berkeley

Big Data Systems

Guest Speaker: Reynold Xin (Databricks)

- Submit your review before 1:00PM.
- Lecture slides: PDF, PPTX
- Guest Lecture slides: PDF

Required Reading

- Towards a Unified Architecture for in-RDBMS Analytics
- Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing
- Lakehouse: A New Generation of Open Platforms that Unify Data Warehousing and Advanced Analytics

Additional Optional Reading

Hardware for Machine Learning

Guest Speaker: Prof. Sophia Shao (UC Berkeley)

- Submit your review before 1:00PM.
- Lecture slides: PDF, PPTX
- Guest Lecture slides: PDF

Required Reading

- Mixed precision training, [ICLR'18]
- Eyeriss: A Spatial Architecture for Energy-Efficient Dataflow for Convolutional Neural Networks [SIGGRAPH, 2016]
- Interstellar: Using Halide's Scheduling Language to Analyze DNN Accelerators (formerly: DNN Dataflow Choice Is Overrated)
- Gemmini: Enabling Systematic Deep-Learning Architecture Evaluation via Full-Stack Integration [Best Paper Award, DAC'21]

Additional Optional Reading

Distributed deep learning, Part I: Systems

Guest Speaker: Microsoft DeepSpeed Team

- Submit your review before 1:00PM.
- Lecture slides: PDF, PPTX
- Guest Lecture slides: PDF

Required Reading

- Chimera: Efficiently Training Large-Scale Neural Networks with Bidirectional Pipelines [SC'21, Best Paper finalist]
- Efficient Large-Scale Language Model Training on GPU Clusters Using Megatron-LM [SC'21, Best Student Paper]
- ZeRO-Infinity: Breaking the GPU Memory Wall for Extreme Scale Deep Learning [SC'21]

Additional Optional Reading

Distributed deep learning, Part II: Scaling Constraints

Guest Speaker: Michael Houston (Nvidia)

- Submit your review before 1:00PM.
- Lecture slides: PDF, PPTX
- Guest Lecture slides: PDF

Required Reading

- Measuring the Effects of Data Parallelism on Neural Network Training
- Scaling Laws for Neural Language Models [OpenAI, 2020]
- Deep Learning Training in Facebook Data Centers: Design of Scale-up and Scale-out Systems

Additional Optional Reading

Project Proposals

- Lecture slides: PDF, PPTX

Machine learning Applied to Systems

Guest Speaker: Prof. Tim Kraska (MIT)

- Submit your review before 1:00PM.
- Lecture slides: [PDF], [PPTX]

Required Reading

- The Case for Learned Index Structures [ICMD'18]
- Device Placement Optimization with Reinforcement Learning [ICML'17]
- Neural Adaptive Video Streaming with Pensieve [SIGCOMM'17]

Spring Break

Machine Learning Frameworks and Automatic Differentiation

Guest Speaker: Prof. Tianqi Chen (OctoML and CMU)

- Submit your review before 1:00PM.
- Lecture slides: PDF, PPTX
- Joey's Lecture slides: PDF, PPTX

Required Reading

- Automatic differentiation in ML: Where we are and where we should be going
- TensorFlow: A System for Large-Scale Machine Learning
- TVM: An Automated End-to-End Optimizing Compiler for Deep Learning

Efficient Machine Learning

Guest Speaker: Vikas Chandra (Facebook)

- Submit your review before 1:00PM.
- Lecture slides: PDF, PPTX

Required Reading

- Linear Mode Connectivity and the Lottery Ticket Hypothesis
- Integer-only Quantization of Neural Networks for Efficient Integer-Arithmetic-Only Inference
- FBNet: Hardware-Aware Efficient ConvNet Design via Differentiable Neural Architecture Search

Additional Optional Reading

Fundamentals of Machine Learning in the Cloud, the Modern Data Stack

Guest Speaker: Prof. Matei Zaharia (Databricks and Stanford)

- Submit your review before 1:00PM.
- Lecture slides: PDF

Required Reading

- The Sky Above The Clouds
- FrugalML: How to Use ML Prediction APIs More Accurately and Cheaply
- Pollux: Co-adaptive Cluster Scheduling for Goodput-Optimized Deep Learning

Additional Optional Reading

Benchmarking Machine Learning Workloads

Guest Speaker: Prof. Vijay Reddi (Harvard)

- Submit your review before 1:00PM.
- Lecture slides: PDF, PPTX

Required Reading

- MLPerf Training Benchmark
- MLPerf Inference Benchmark
- Benchmark Analysis of Representative Deep Neural Network Architectures

Machine learning and Security

- Submit your review before 1:00PM.
- Lecture slides: PDF

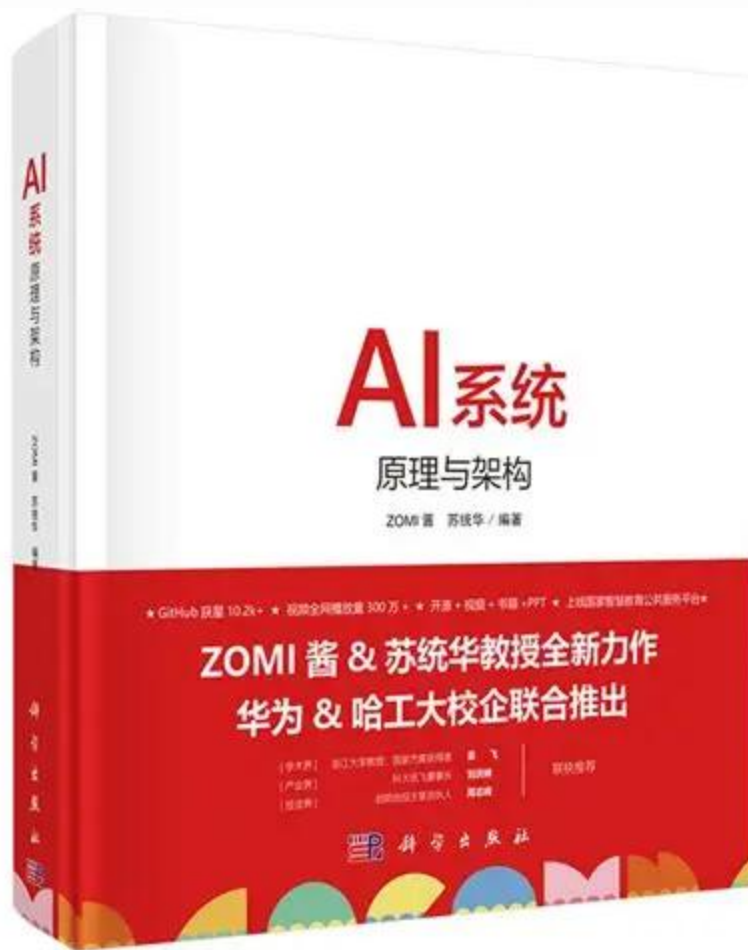
Required Reading

- Communication-Efficient Learning of Deep Networks from Decentralized Data



软硬协同：协同思维（GPU计算课程）

序号	教学内容	学时
1	1. GPU计算概述 (1)GPU概念与Flynn分类体系 (2)GPU计算发展历史 (3)GPU硬件简介 (4)GPU程序结构 (5)CPU与GPU程序区别	2
2	2. GPU硬件架构 (1)GPU连接模型 (2)现有GPU硬件架构分析 (3)流处理器SMX结构	2
3	3. GPU计算的软件抽象 (1)CPU与GPU设计理念对比 (2)多线程原理与GPU细粒度多线程技术 (3)延迟隐藏原则 (4)CUDA抽象原理 (5)GPU内核函数 (6)线程与数据索引方法 (7)设备占用率 (8)线程束调度原理与过程 (9)CUDA软件生态圈 (10)CUDA程序编译与连接	6





提纲

01 课程建设背景

02 国内外课程情况

03 AI技术栈和课程建设思路

04 硬件体系结构与 AI 芯片

05 AI 编译和算子优化

06 并行计算和分布式训练

07 课程思政



AI 系统架构

算法应用

CV

NLP

Audio

...

开发体系

编程语言 (AI与主流语言集成)



AI 训练&推理框架

AI框架



MindSpore



PyTorch



JAX



TensorFlow

AI推理引擎



MindIE



TensorRT



MNN

AI编译器 & 编程体系

传统编译器

AI 编译&计算架构



CANN



CUDA



TVM



XLA



LLVM



GCC

AI 硬件&结构体系



Ascend NPU



TPU



IPU



GPU



NPU



CPU



应用层

- 为不同应用场景提供已经建模好和优化好的不同AI算法
- 计算机视觉、语音语义、机器学习、自然语言处理

AI系统全栈架构图





开发层

- 提供用户前端的编程语言，接口和工具链。
- 尽可能让用户表达 AI 任务与算法，尽量少让用户关注底层实现（如声明式编程、命令式编程）是提升开发体验的较好的手段。

AI系统全栈架构图





框架层

- 负责静态程序分析与计算图构建，编译优化等。
 - 提供用户编程的 API
 - 获取用户表达 AI 模型，数据读取等意图
 - 在静态程序分析阶段完成尽可能的自动前向计算图构建
 - 自动求导补全反向传播计算图
 - 计算图整体编译优化
 - 算子内循环编译优化





AI 开发框架

算法应用

CV

NLP

Audio

...

开发体系

编程语言 (AI与主流语言集成)



AI 训练&推理框架

AI框架



AI推理引擎



AI 编译&计算架构

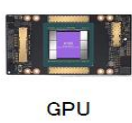
AI编译器 & 编程体系



传统编译器



AI 硬件&结构体系



AI 框架基础

AI 框架的作用

AI 框架之争 (框架发展)

编程范式 (声明式&命令式)

自动微分

微分原理

正反向计算模式

自动微分三种实现方法

实现正向微分框架

实现反向微分框架 (PyTorch)

自动微分的挑战&未来

计算图

什么是计算图

计算图与自动微分关系

图优化与图执行调度

计算图控制流实现

计算图实现动静统一

计算图的挑战与未来

分布式并行

数据并行

模型并行之张量并行

模型并行之流水并行

多维混合并行



AI 推理引擎&推理系统

算法应用

CV

NLP

Audio

...

开发体系

编程语言 (AI与主流语言集成)

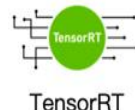


AI 训练&推理框架

AI框架



AI推理引擎



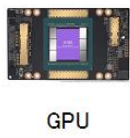
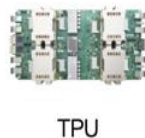
AI 编译&计算架构

AI编译器 & 编程体系



传统编译器

AI 硬件&结构体系



推理系统

推理系统

推理流程

推理系统 & 推理引擎架构

模型轻量化

推理参数

CNN 模型小型化

Transformer 小型化

模型压缩

低比特量化

感知量化训练 QTA & 训练后量化 PTQ

模型剪枝

知识蒸馏

模型转换

转换模块架构 & 模型转换流程

转换文件格式

自定义计算图 IR

图优化

计算图优化策略

常量折叠&冗余节点消除

算子融合/替换/前移

数据布局转换&内存优化

Kernel 优化

Kernel 优化整体方案

CNN & Transformer 优化

推理内存布局 & nc4hw4 内存排布

汇编与循环优化



AI 编译&计算架构

算法应用

CV

NLP

Audio

...

开发体系

编程语言 (AI与主流语言集成)



AI 训练&推理框架

AI框架



AI推理引擎



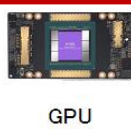
AI 编译&计算架构

AI编译器 & 编程体系



传统编译器

AI 硬件&结构体系



传统编译器

GCC编译过程和原理
LLVM设计架构与详解

AI 编译器

AI编译器的发展阶段
AI编译器的通用架构

前端优化

计算图层IR
算子融合策略
布局转换与内存分配算法
常量折叠与公共表达式消除
死代码消除与代数简化

后端优化

算子的计算与调度
算子优化手工方式
算子循环优化
指令和内存优化
Auto-Tuning原理

多面体技术

PyTorch图模式

TorchScript 静态图尝试
Torch FX 与 LazyTensor 特性
AOTAutograd 与 Dispatch 原理



编译编程-功能

- 编译编程：
 - 提供硬件编程模型
 - 框架 IR 编译优化
- 编程模型
 - 通过层次结构来组织软件执行线程
 - 通过层次结构来组织硬件内存的访问
 - 如 CUDA 编程模型

AI系统全栈架构图





编译编程-性能

- 循环优化等传统编译器的优化策略
- 运行时动态调度与优化。
 - 计算图部署于单卡，多卡或分布式环境
 - 计算图按照执行顺序调度算子与任务的执行
 - 多路复用资源，硬件驱动，内存等资源的分配与释放。
- 编译优化
 - 编译器或框架
 - 根据计算语义优化算子执行方式
 - 降低内核启动与访存代价。

AI系统全栈架构图





AI 硬件&结构体系

算法应用

CV

NLP

Audio

...

开发体系

编程语言 (AI与主流语言集成)



AI 训练&推理框架

AI框架



MindSpore



PyTorch



JAX

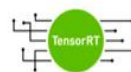


TensorFlow

AI推理引擎



MindIE



TensorRT



MNN

AI编译器 & 编程体系



CANN



CUDA



TVM



XLA



LLVM

传统编译器



GCC

AI 硬件&结构体系



Ascend NPU



TPU



IPU



GPU



NPU



CPU

AI 计算体系

AI计算模式与关键设计指标

核心计算：矩阵乘

数据单位：比特位

AI 芯片基础

CPU 指令集架构

CPU 计算本质

GPU/NPU 基础原理

图形处理器 GPU

GPU 与 AI 关系

GPU 架构与 CUDA 关系

GPU 十年架构回顾

英伟达 GPU 详解

TensorCore原理

分布式通信与NVLink

国外 AI 芯片

特斯拉 DOJO

谷歌 TPU

国内 AI 芯片

壁仞BR100

燧原科技

寒武纪AI芯片

AI 芯片黄金十年

芯片的编程体系

SIMD和SIMT跟AI芯片关系

CUDA/SIMD/SIMT/DSA关系



硬件体系结构

- 芯片和网络加速器
- 硬件接口抽象：
 - GPU\TPU\NPU和各种DSA。
 - 统一硬件接口抽象复用编译优化策略
 - 让优化与底层设备和体系结构解耦。
- 可扩展网络
 - RDMA，InfiniBand，NVLink 等提供高效NPU到NPU 互联，
 - 高带宽，更灵活通信原语与高效的通信聚合算法





硬件体系结构

- 存储、组网和资源管理
- 集群硬件体系结构
 - 存储、网络 and 程序执行
 - 互联与加速
- 作业调度
 - 结合任务特点和异构硬件拓扑进行高效调度
- 资源管理
 - 将服务器资源池化
 - 运行期资源分配与环境隔离
 - 异构资源集群管理





广泛的 AI 系统生态

核心 AI 系统软硬件

强化学习、联邦学习

可解释性AI

AI 安全与隐私

模型压缩、小型化

AI 算法和框架

新 AI 算法
新 AI 范式支持

AI 框架、AI 推理引擎支持与演进

AI 编译架构及优化

核心 AI 系统软硬件

AI 任务
执行和编程模型

AI 集群资源
管理和调度

AI 加速器
及高性能网络



AI系统技术栈

应用算法 (Algorithm)	领域算法	视觉、自然语言处理、语音语义：CNN、RNN、LSTM、GAN、Transformer、Stable Diffusion、Reinforce Learning			
开发编程 (Development)	集成开发环境	编程环境：VS Code, Jupyter Notebook, PyCharm			
	编程语言	与主流语言集成：PyTorch and Tensorflow etc. inside Python			
框架 (Frameworks)	AI框架、推理引擎	中间表达（IR）、前端 API	编译优化（Compilation）	图优化与图执行调度	推理优化
		基本数据结构：张量（Tensor）	词法、语法、语义分析	计算图动静统一	常量折叠&冗余节点消除等
		基本计算单元：有向无环图（DAG）	自动微分	计算图控制流	模型压缩：蒸馏剪枝与量化
异构与编译 (Compiler)	编程编译	前端优化	数据格式布局转换	内存分配	公共表达式消除
		后端优化	代码优化、代码生成	算子循环优化	令和内存优化
		编程模型：CUDA/Ascend C/Band C		AI编译器：TVM、XLA、TC	
		LLVM/GCC			
	底层通信	HCCL/NCCL、通信原语、集合通信算法			
体系结构 (Architecture)	底层硬件	I芯片：CPU/GPU/AISC/FPGA/TPU/NPU			



自底向上全栈贯通的教学方案

❖ 2026年入选北京高校优质本科基本教学资源

AI 技术栈理论知识点

体系结构		底层硬件				国产/异构芯片: CPU / GPU / 昇腾 NPU / 海光 DCU / 寒武纪 MLU / FPGA			
 异构与编译 (Compiler) 		理论联系实践							
		编程编译		数据格式 布局转换		内存分配		公共表达式 消除	
		后端优化		代码优化 代码生成		算子循环优化		令与内存 优化	
		底层通信		编层模型: CUDA / Ascend C / CANN		AI 编译器: TVM / XLA / TC			
		LLVM / GCC / HCCL / NCCL / 通信原语 / 集合通信算法							

理论联系实践

实践方案创新

图形化 AI 加速单元硬件设计



4Bit 量化矩阵乘法
算子设计与实现



$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \div \begin{bmatrix} e & f \\ g & h \end{bmatrix}_x$$

面向矩阵乘与激活函数的
融合算子设计与实现



理论联系实践

 框架与算法 (Frameworks) AI Model	AI 框架 推理引擎 ✓ Optimization ✓ Optimization	中间表示 (IR) 前缀 API	编译优化 (Compilation)	图优化与 图块行调度	推理优化
		基本数据结构 张量 (Tensor)	词法 / 语法 语义分析	计算图动静统一	常量折叠 冗余节点消除
		基本计算单元 有向无环图 (DAG)	自动微分	计算图控制流	模型压缩 蒸馏 / 剪枝 / 量化



LLM

KV Cache 与 FlashAttention
联合优化

大模型推理效率提升实验



提纲

01 课程建设背景

02 国内外课程情况

03 AI技术栈和课程建设思路

04 硬件体系结构与 AI 芯片

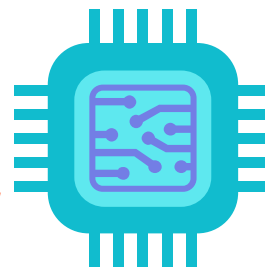
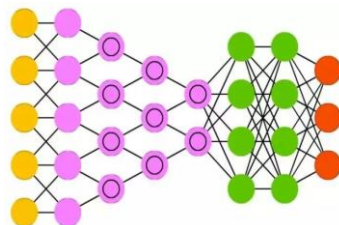
05 AI 编译和算子优化

06 并行计算和分布式训练

07 课程思政



人工智能的运行环境



架构基础



架构设计



标准与评测



芯片分类

- 强大的调度、管理、协调能力
- 应用范围广
- 开发方便灵活

- 并行架构
- 计算单元多
- 合适大量逻辑确定的重复计算

- 针对AI定制架构
- 成本低、能耗低

CPU

GPU

NPU

Matrix unit

Vector unit

Cache/Buffer

CPU

System Memory

XPU

灵活性

能效性

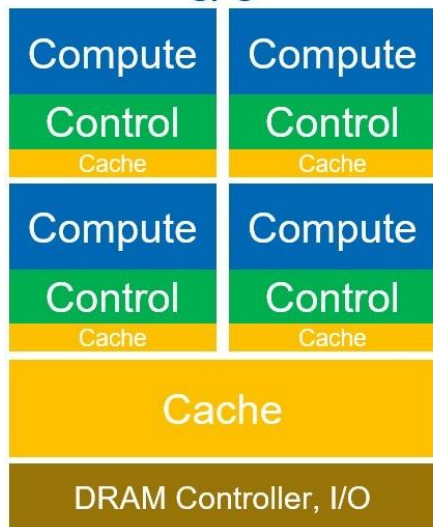


从专用到通用

通用：高速，低吞吐



CPU



专用：中速，高吞吐



GPU



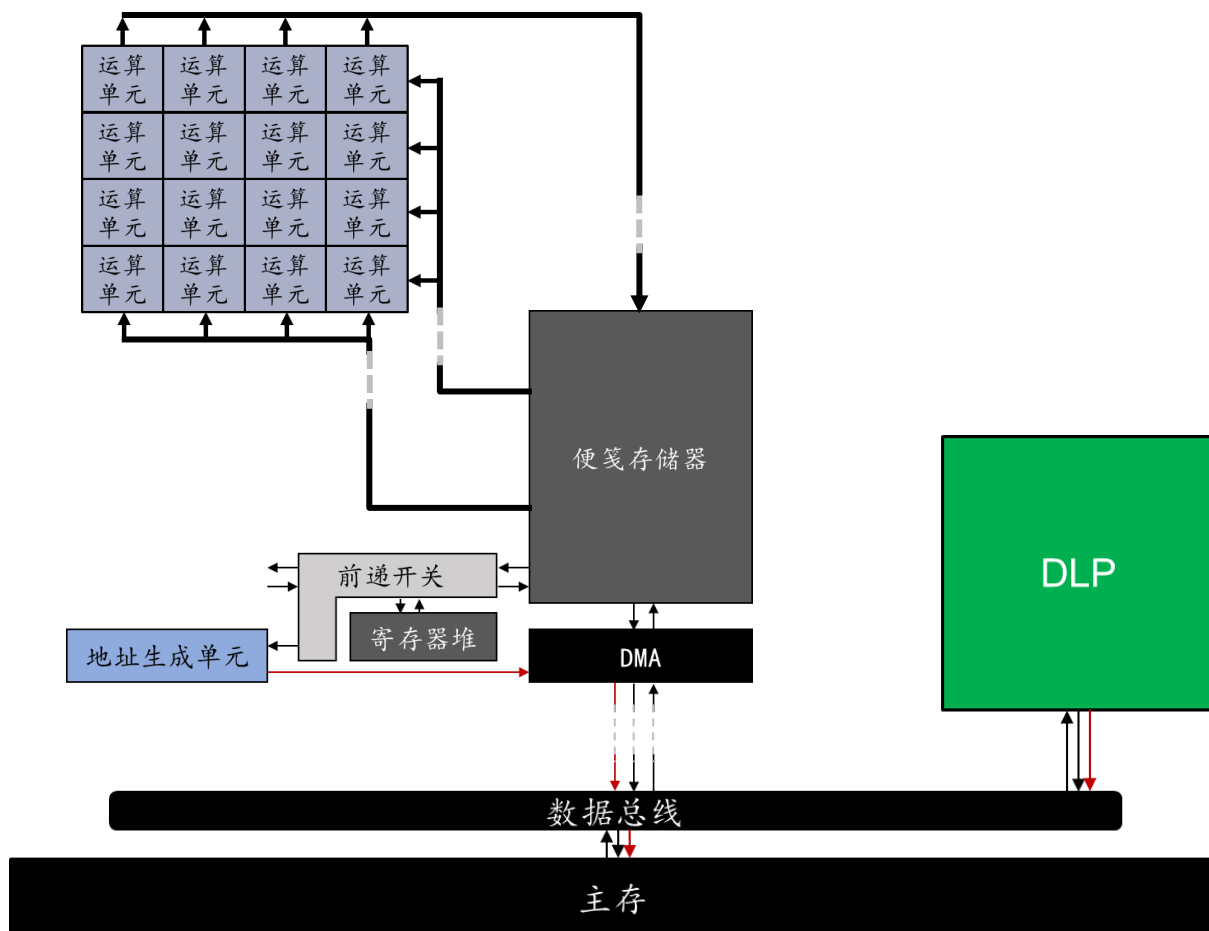


架构设计

▶ 计算

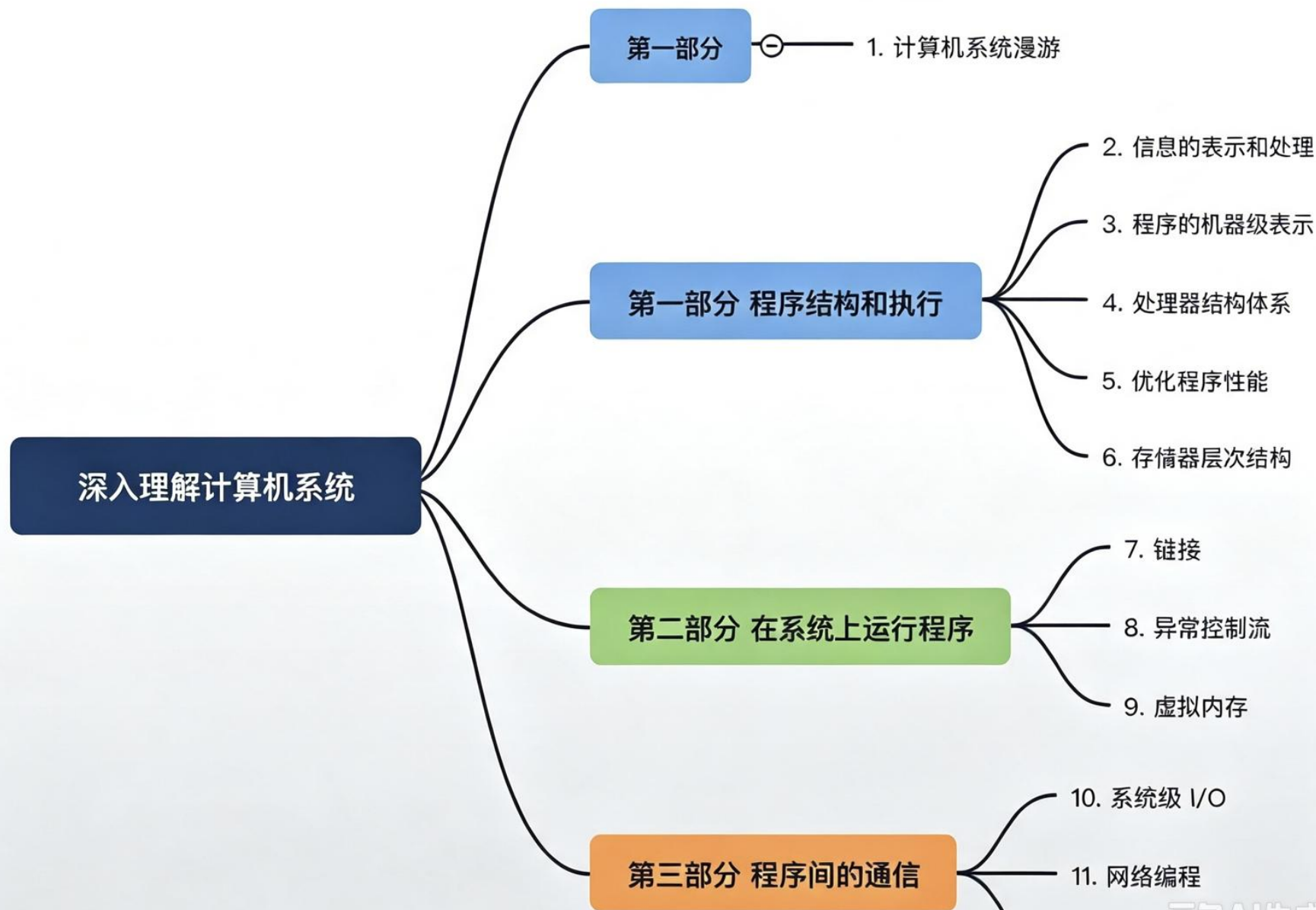
▶ 访存

▶ 通信



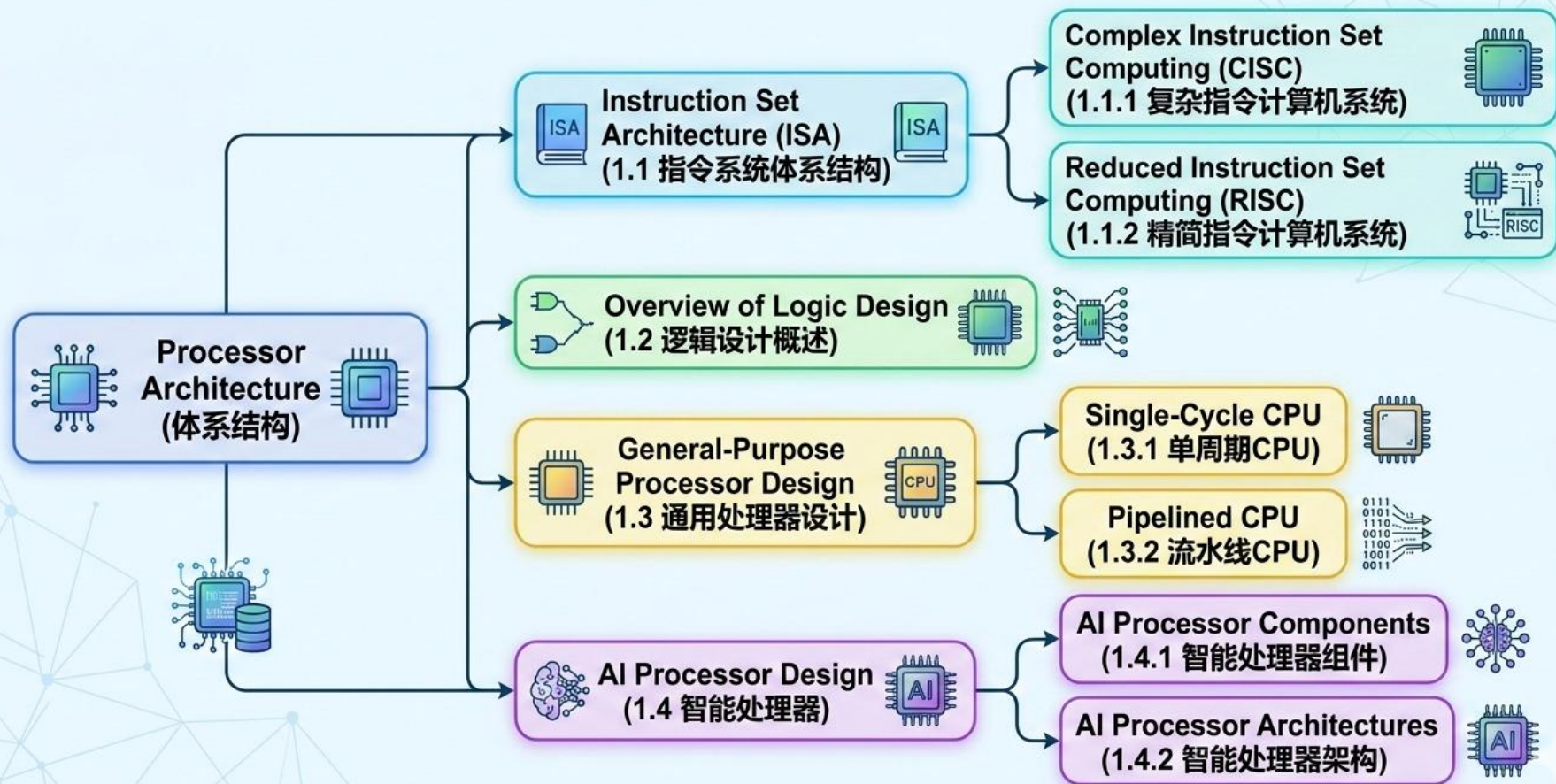


计算机系统基础（CSAPP）



硬件体系结构

Technical Hierarchy Mind Map: Computer Architecture





课程安排

教学周	章节名称	讲授内容及掌握程度
第1周	智能计算机系统漫游	课程概述，智能计算机系统概述
第2周	处理器体系结构	数字逻辑基础（寄存器、门电路）、指令系统体系结构，CISC和RISC
第3周	处理器体系结构	单周期CPU的数据通路，控制信号和控制器设计
第4周	处理器体系结构	流水线CPU基本原理，相关和冒险，超标量流水线
第5周	深度学习处理器	深度学习处理器架构概况：并行思想与弗林架构，英伟达架构发展
第6周	深度学习处理器	深度学习处理器设计：脉动阵列、便笺存储器，多核互联



计算机专业培养计划



大一课程



大二课程



大三课程



大四课程

Java程序设计 +

C语言程序设计 +

Python程序设计

C++程序设计

数据结构

C#程序设计 +

计算机组成原理 +

算法设计与分析 +

离散数学基础

概率论与数理统计

软件工程

编译原理

操作系统 +

软件测试方法和技术实践

面向对象软件开发实践 +

计算机系统结构 +

Linux内核分析

软件安全 +

信息安全数学基础

近世代数

计算机组成与设计

硬件/软件接口

[美] 戴维·A. 帕特森 (David A. Patterson)
约翰·L. 亨尼斯 (John L. Hennessy) 著
易江芳 刘先华 等译

计算机体系结构新黄金时代必读之作，理解专用硬件协同设计，共建开源生态



Computer Organization and Design
The Hardware/Software Interface, RISC-V Edition

机械工业出版社
China Machine Press

计算机体系结构

量化研究方法

[美] 约翰·L. 亨尼斯 戴维·A. 帕特森 著
John L. Hennessy David A. Patterson

COMPUTER
ARCHITECTURE

A Quantitative Approach

(英文版·原书第6版)

机械工业出版社
China Machine Press



国内课程

coursera

探索 ▾

学位

您想学习什么?



🏠 > 浏览 > Computer Science > Computer Security and Networks



北京大学
PEKING UNIVERSITY

计算机组成 Computer Organization



位教师: 陆俊林 (Lu Junlin)

计算机组成原理

第2版

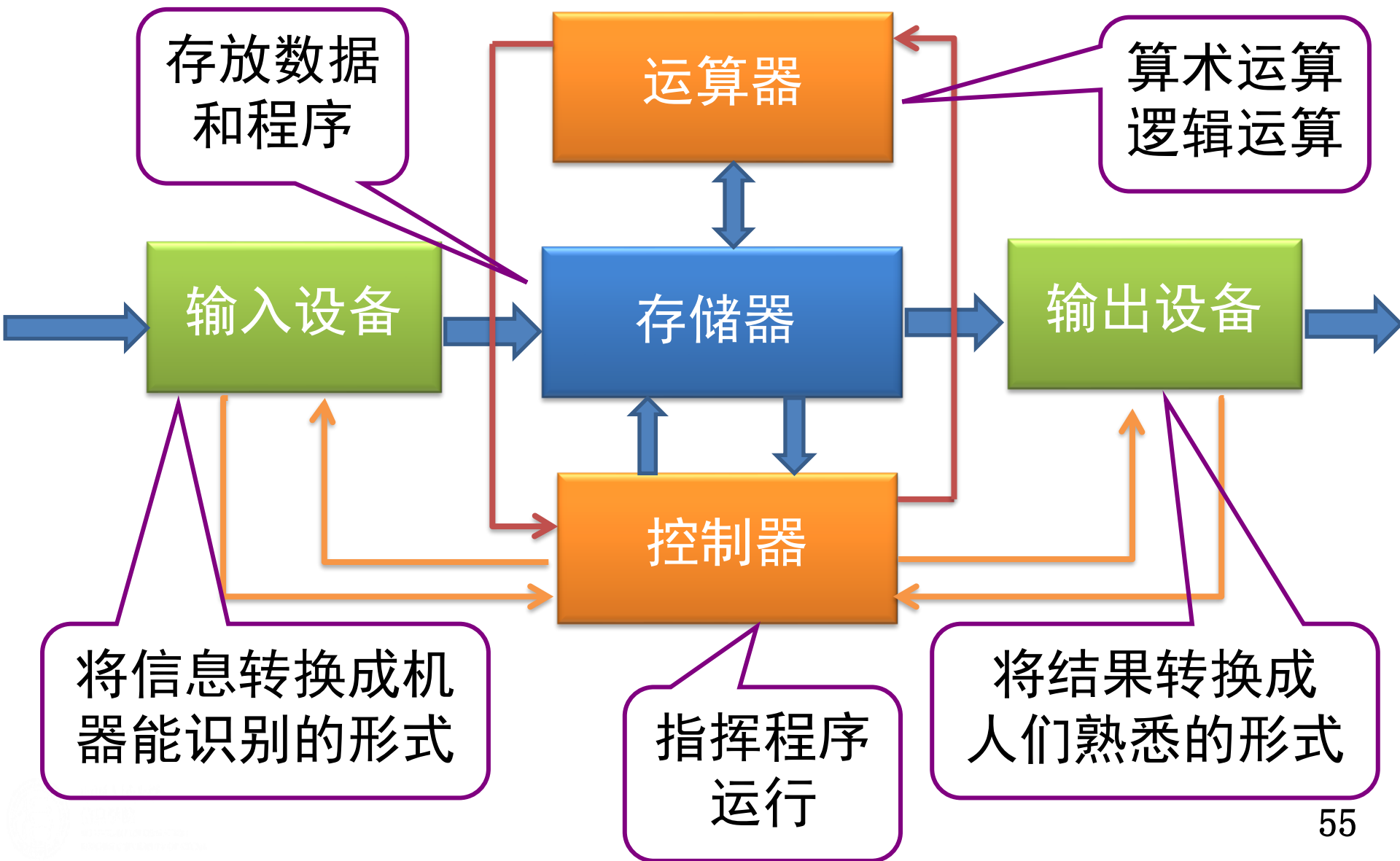
唐朔飞



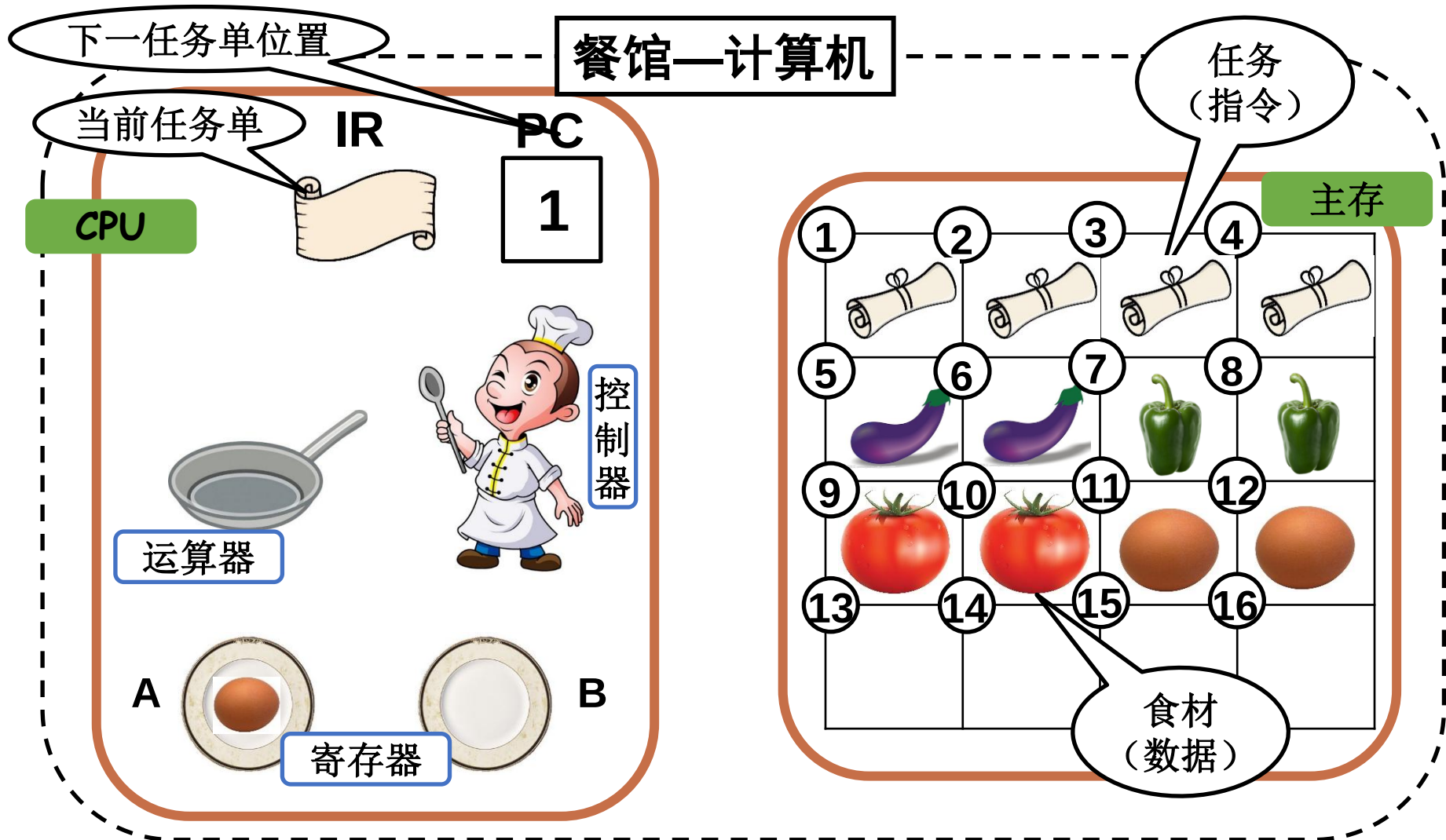
高等教育出版社
高等教育电子音像出版社



冯·诺依曼结构的要点



计算机的硬件

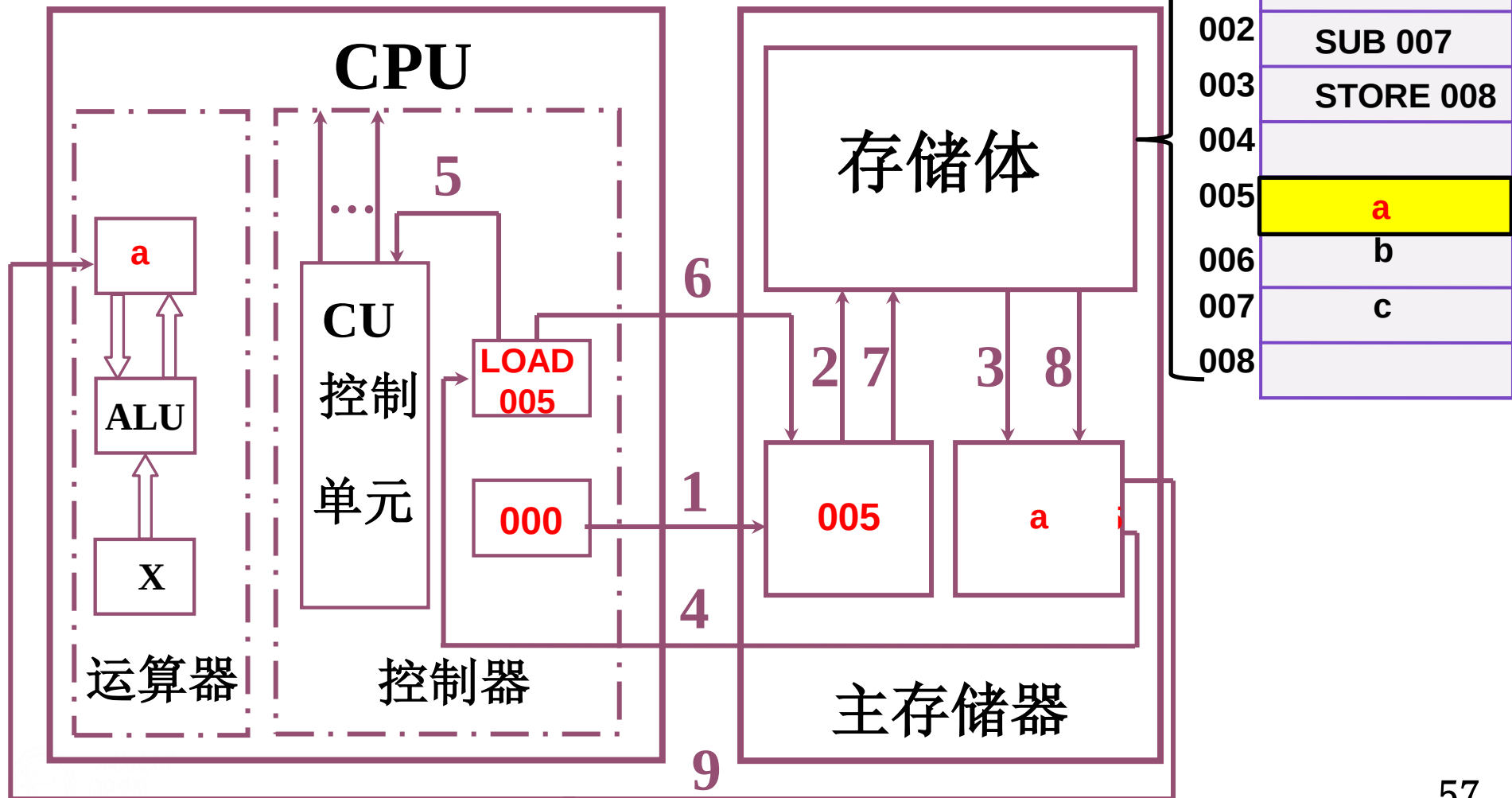




主机完成一个程序的过程

取数指令

主存





指令系统体系结构





指令系统体系结构

- CPU 于上世纪 60 年代问世，已发展几十年，有几十种不同的指令系统架构相继诞生或消亡。
- 依据硬件系统构成的复杂程度、指令执行的效率和并行性划分，指令系统设计实现有两种思路：



复杂指令计算机 CISC:
Complex Instruction Set
Computer

精简指令计算机 RISC:
Reduced Instruction Set
Computer



CSAPP教材：Y86-64

Byte	0	1	2	3	4	5	6	7	8	9
halt	0	0								
nop	1	0								
cmovXX rA, rB	2	fn	rA	rB						
irmovq V, rB	3	0	F	rB	V					
rmmovq rA, D(rB)	4	0	rA	rB	D					
mrmmovq D(rB), rA	5	0	rA	rB	D					
OPq rA, rB	6	fn	rA	rB						
jXX Dest	7	fn	Dest							
call Dest	8	0	Dest							
ret	9	0								
pushq rA	A	0	rA	F						
popq rA	B	0	rA	F						

- 从内存中读取**1到10**字节的信息
- 从第一个字节确定指令长度



RISC架构



John Hennessy

斯坦福大学前校长, 现任 **Alphabet** 董事会主席、深度参与 **AI** 治理与产业战略。



David Patterson

加州大学伯克利分校教授, **RISC-V RIOS Lab** 主任与 **Google DeepMind** 杰出工程师



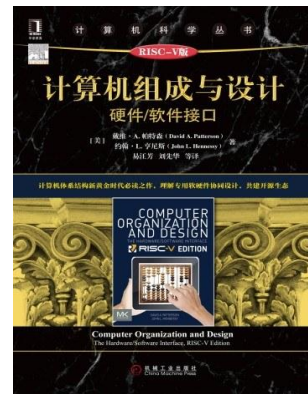
MIPS 架构商业化

由 John Hennessy 主导并创立 MIPS 公司, 成功将 RISC 架构从学术研究转化为成熟的商业产品, 奠定了行业基础。



SPARC 原型与 RISC-V

David Patterson 团队首创 RISC 术语并实现 SPARC 原型。如今, RISC-V 等开源架构继承衣钵, 全球99%的移动设备采用 RISC 架构。



2017年图灵奖得主:
联合表彰其对计算机架构的定量方法





处理器的设计步骤

- ① 分析指令系统，得出对数据通路的需求
- ② 为数据通路选择合适的组件
- ③ 连接组件建立数据通路
- ④ 分析每条指令的实现，以确定控制信号
- ⑤ 集成控制信号，形成完整的控制逻辑



MIPS指令系统的简化版本

- 无符号加法和减法

- `addu rd,rs,rt`
- `subu rd,rs,rt`

- 立即数的逻辑或

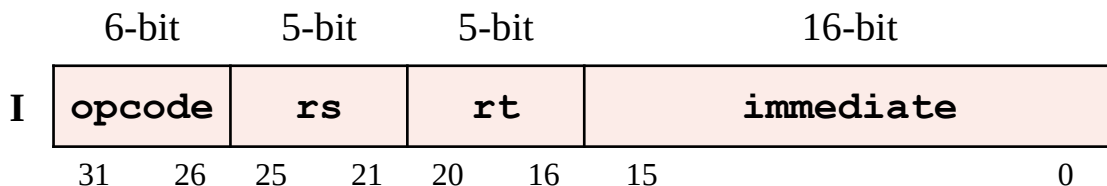
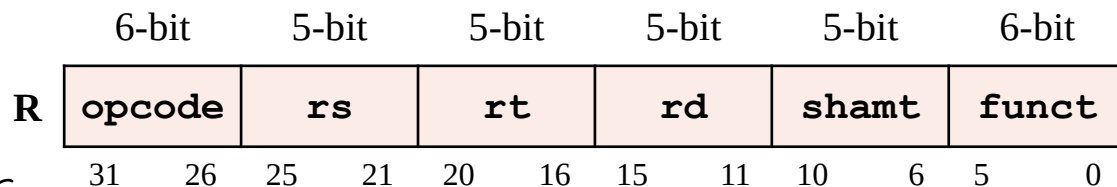
- `ori rt,rs,imm16`

- 装载和存储一个字（32位）

- `lw rt,imm16(rs)`
- `sw rt,imm16(rs)`

- 条件分支

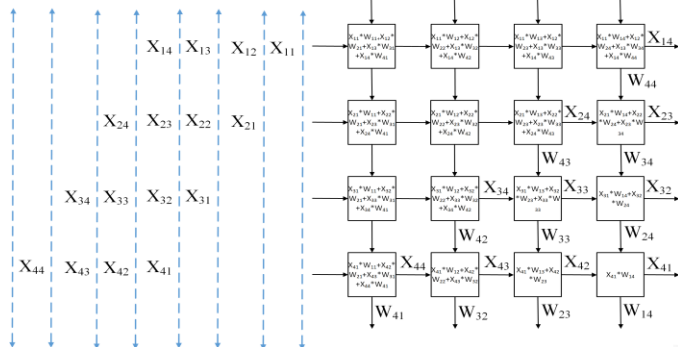
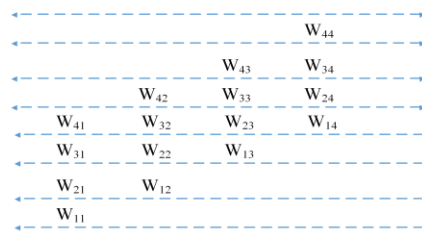
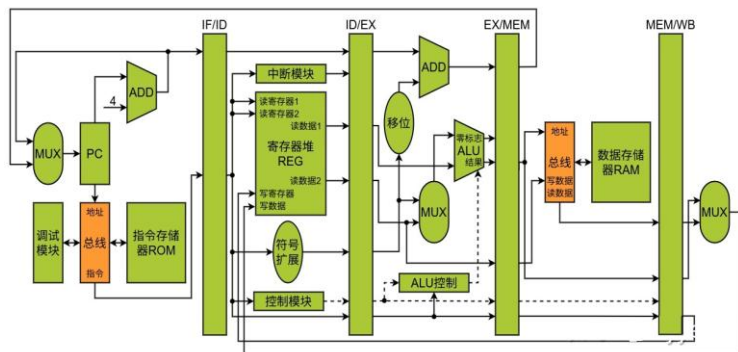
- `beq rs,rt,imm16`





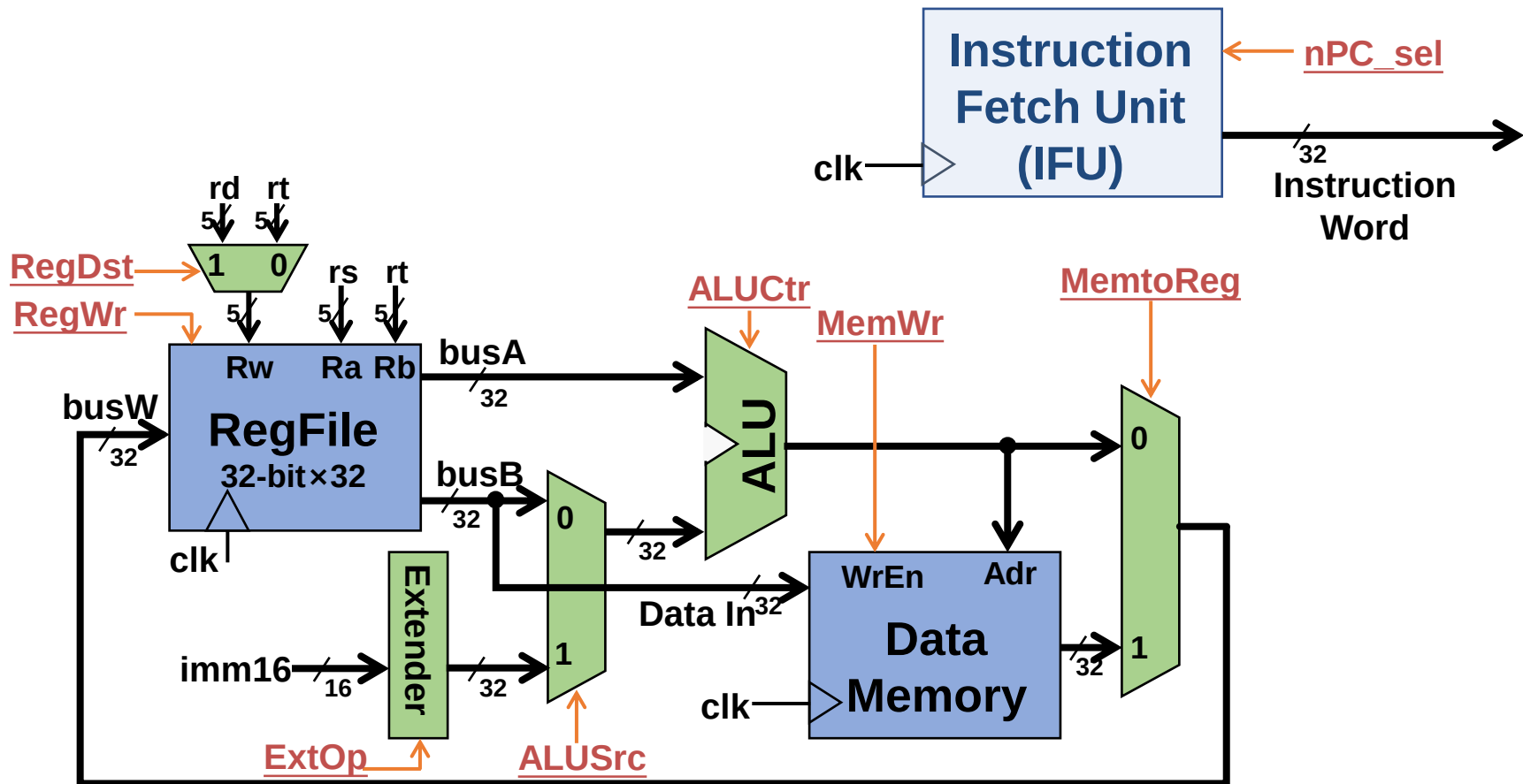
实验：大模型辅助的图形化异构处理器设计

- 图形编码与开发板实践双轮驱动
- 图形化设计
 - 解决硬件描述语言学习难度大的问题
 - 解决大模型直接生成代码缺乏思考的问题
- 开发板实践
 - 嵌入式开发板创新智能应用：养老、教育、医疗等领域
 - 基于华为昇腾310，熟悉国产硬件和生态





数据通路初步完成

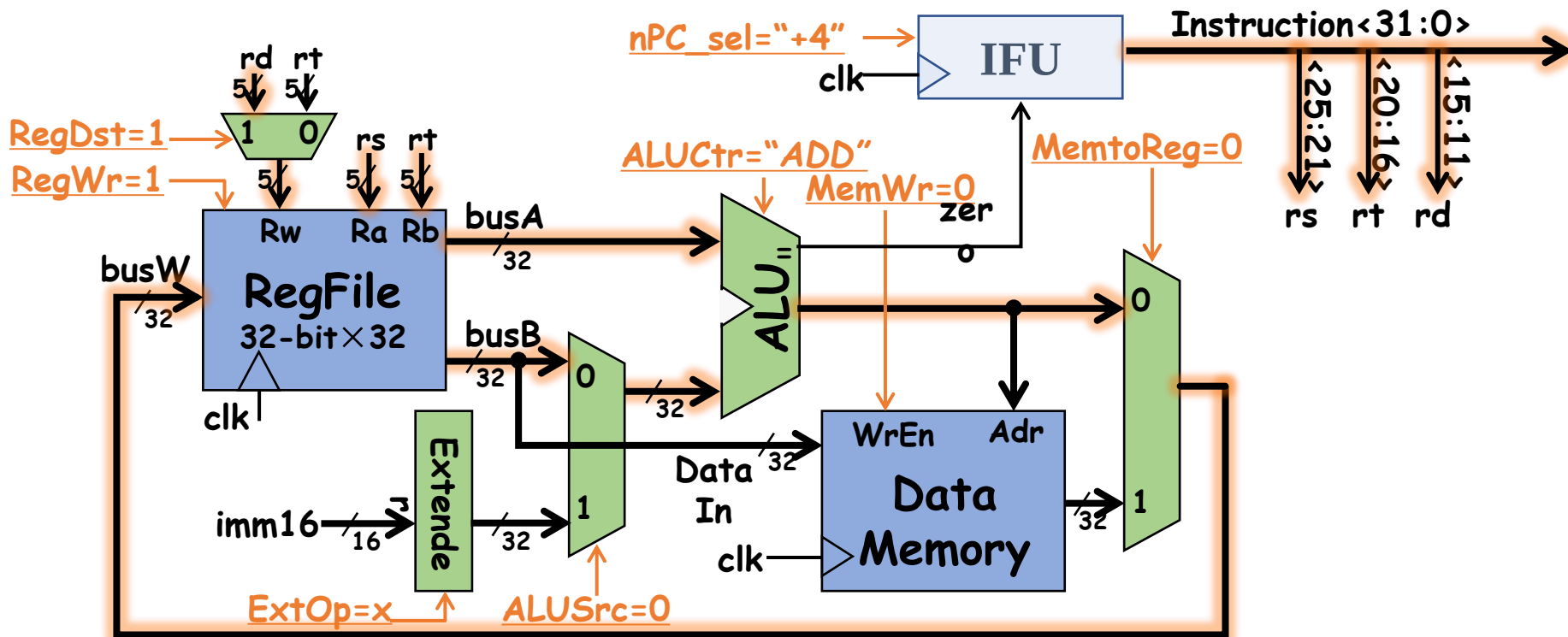




控制信号的汇总

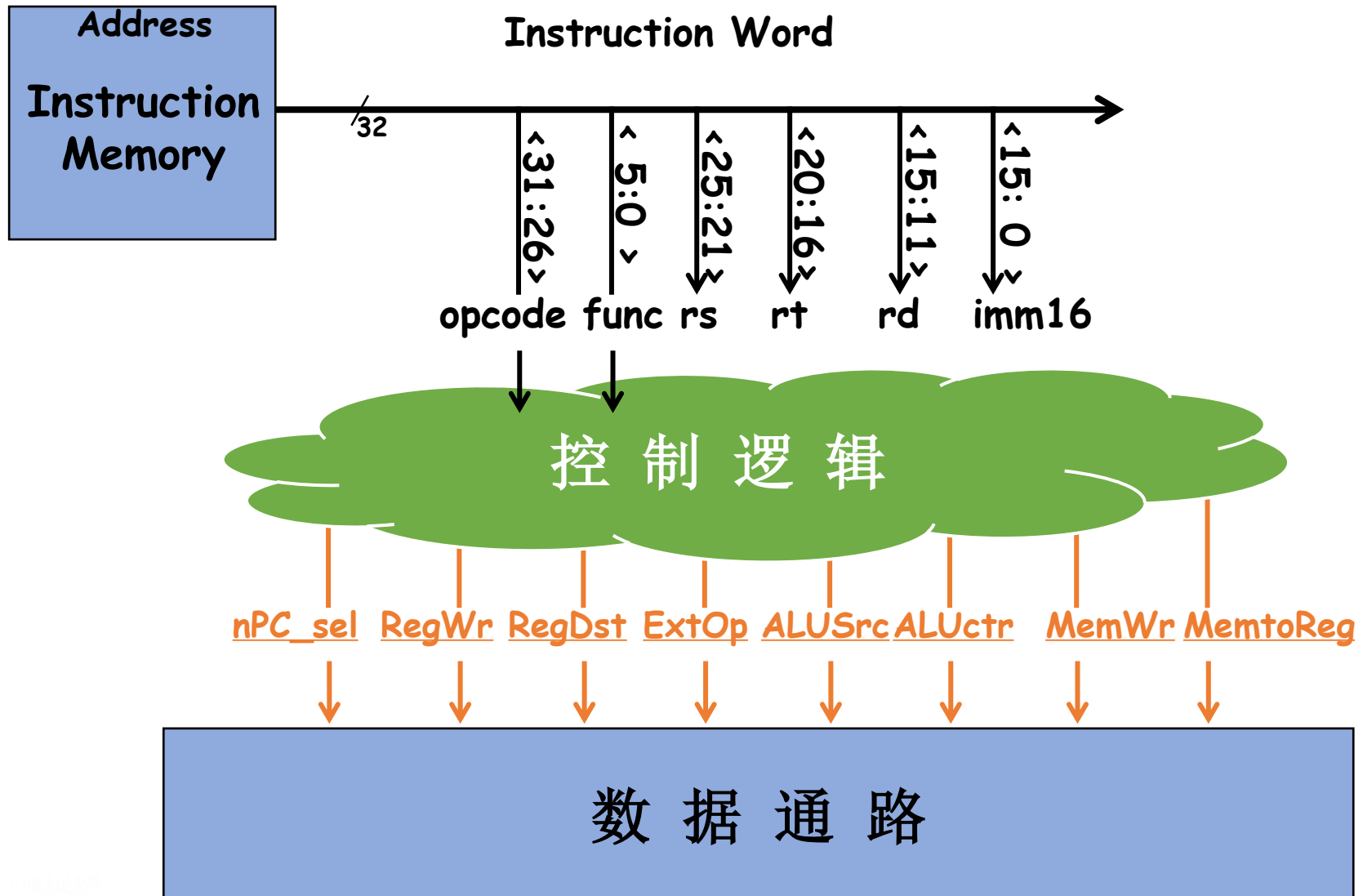
(以add指令为例)

add: $R[rd] \leftarrow R[rs] + R[rt]; PC \leftarrow PC + 4$





控制逻辑与数据通路



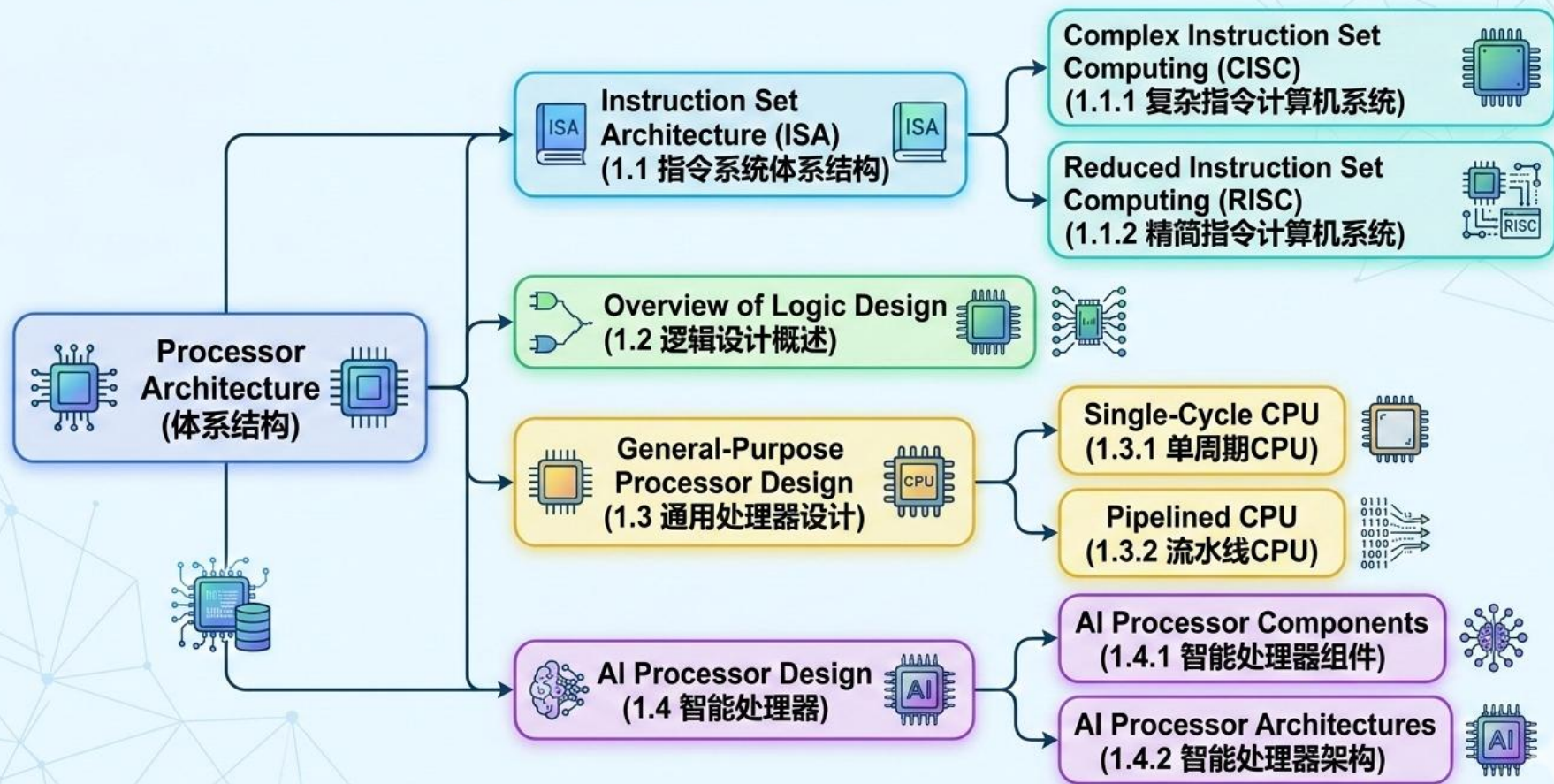


控制信号的逻辑表达式

func opcode (op)	100000	100010	/			
	000000	000000	001101	100011	101011	000100
	add	sub	ori	lw	sw	beq
RegDst	1	1	0	0	x	x
ALUSrc	0	0	1	1	1	0
MemtoReg	0	0	0	1	x	x
RegWr	1	1	1	1	0	0
MemWr	0	0	0	0	1	0
nPC_sel	0	0	0	0	0	1
ExtOp	x	x	0	1	1	x
ALUctr<1:0>	00 (ADD)	01 (SUB)	10 (OR)	00 (ADD)	00 (ADD)	01 (SUB)

硬件体系结构

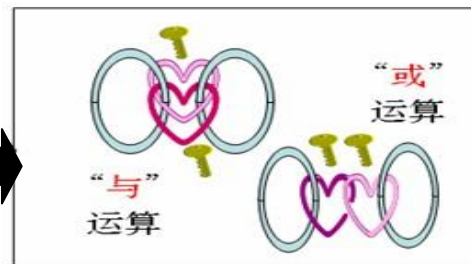
Technical Hierarchy Mind Map: Computer Architecture



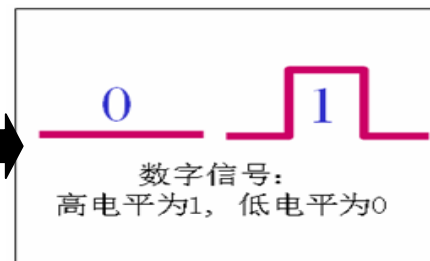
计算机的本质是逻辑



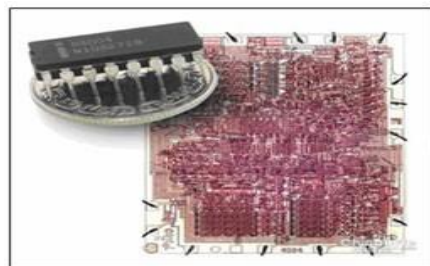
现象和思维均可表达成0和1



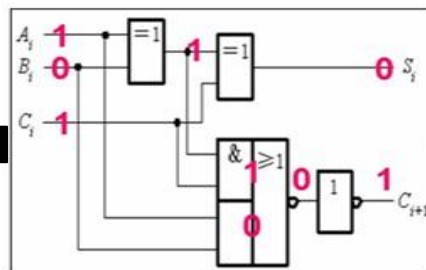
用0和1可进行算术与逻辑运算



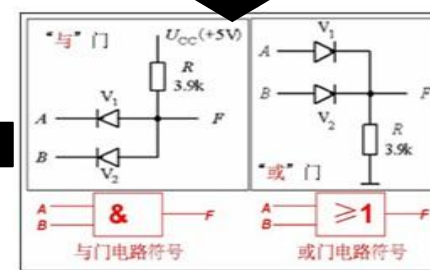
0和1可用电子技术实现



芯片--集成的复杂组合逻辑电路



分层构造实现复杂运算



电子技术实现逻辑运算

语义符号化 → 符号计算化 → 计算0(和)1化 → 0(和)1自动化 → 分层构造化 → 构造集成化



计算机的本质是逻辑

0和1的计算：基本逻辑运

注: 1表示 真, 0表示 假

用0和1来表示逻辑运算

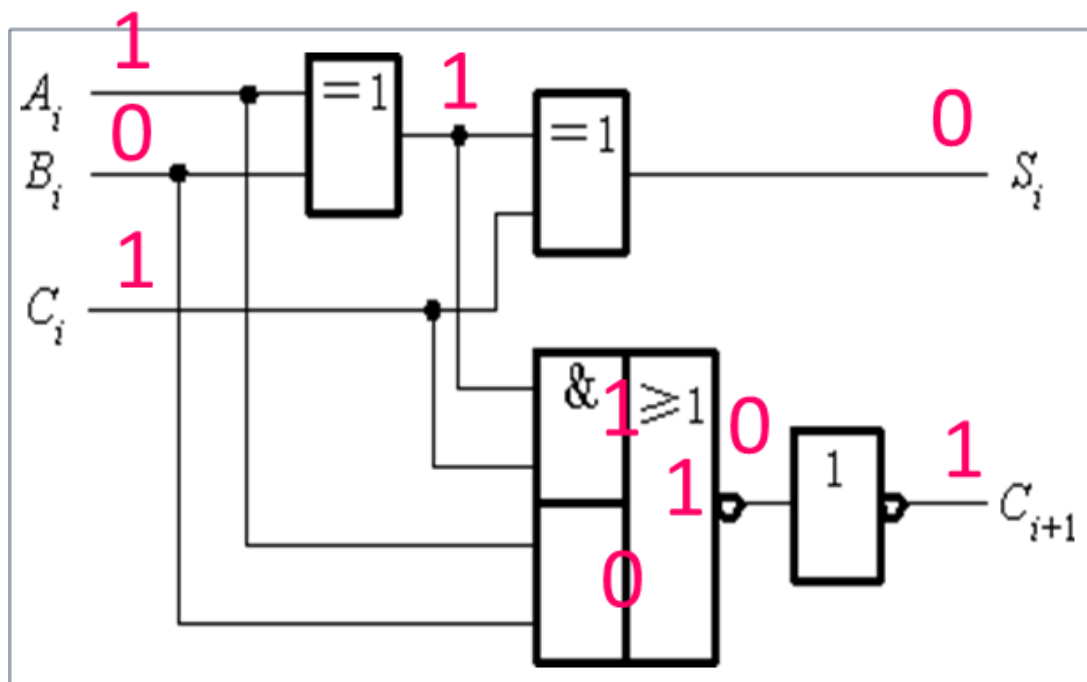
- **【AND】**：“与” 运算
有0为0，全1为1
- **【OR】**：“或” 运算
有1为1，全0为0
- **【NOT】**：“非” 运算
非0则1，非1则0
- **【XOR】**：“异或” 运算
相同为0，不同为1

0	0	1	1
AND 0	AND 1	AND 0	AND 1
0	0	0	1
0	0	1	1
OR 0	OR 1	OR 0	OR 1
0	1	1	1
NOT 0		NOT 1	
1		0	
0	0	1	1
XOR 0	XOR 1	XOR 0	XOR 1
0	1	1	0



加法运算是这样实现的

一位加法器的示例



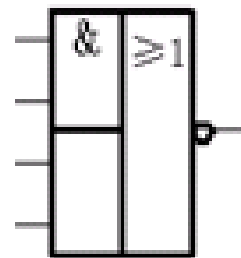
A_i
 B_i
 C_i
+
 C_{i+1} S_i

$\&$
与门电路符号

≥ 1
或门电路符号

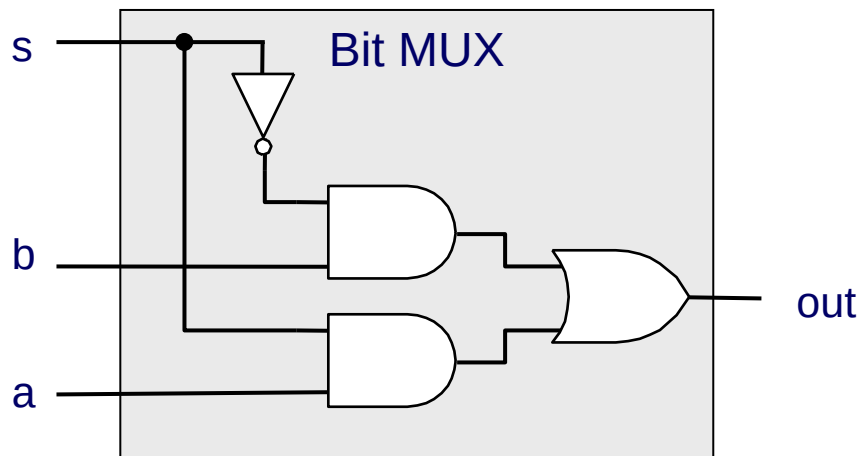
1
非门电路符号

$= 1$
异或门电路符号





位级多路选择器



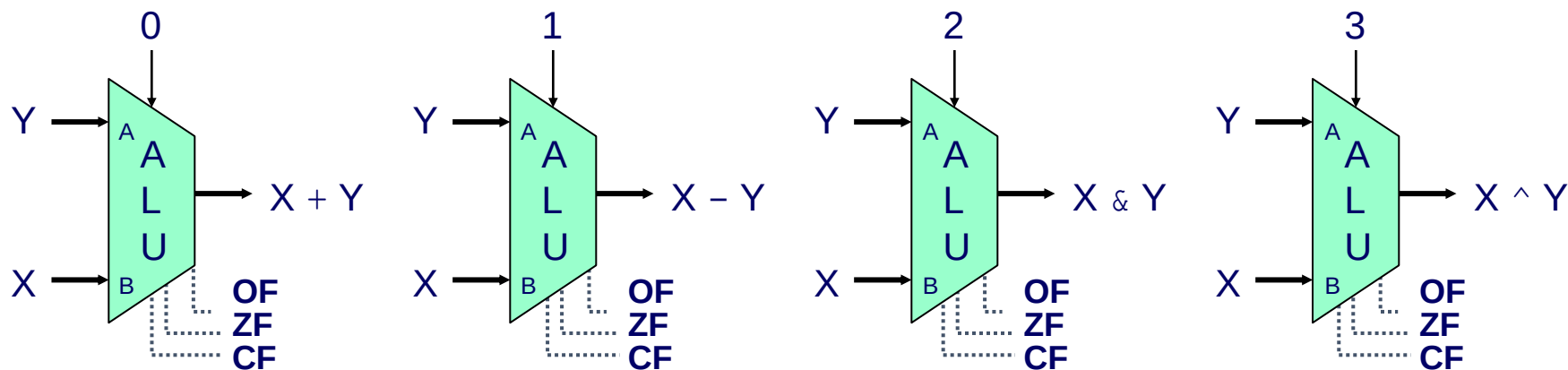
HCL Expression

```
bool out = (s&&a) || (!s&&b)
```

- 控制信号s
- 数据信号a and b
- 当s = 1时输出a，当s = 0时输出b



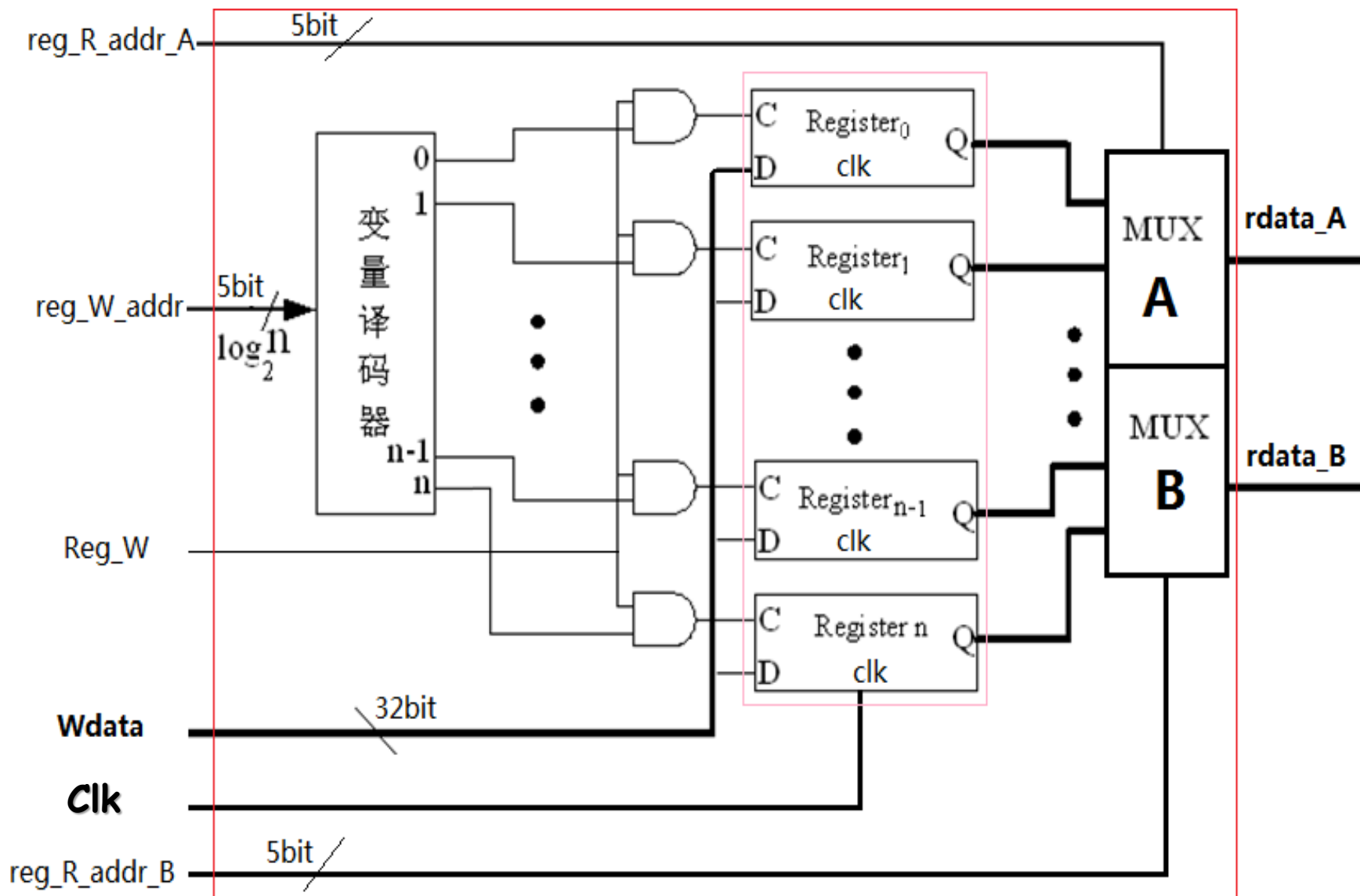
算术逻辑单元Arithmetic Logic Unit



- 组合逻辑
 - ✓ 持续响应输入
- 控制信号选择要计算的函数
 - ✓ 对应于4种算术/逻辑运算
- 也计算条件码的值



完整的寄存器组



什么是译码器

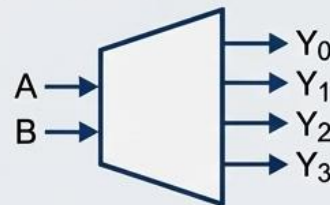
组合逻辑电路，主要用于将输入的二进制代码“翻译”为特定的输出信号。

- **多对多转换**：通常具有 n 个输入端和 2^n 个输出端。
- **唯一性激活**：在任何时刻，输入的二进制组合，只有一个对应的输出端会被激活。
- **关键应用**：广泛用于**指令译码**（指令操作码）、**存储器寻址**（片选信号生成）。

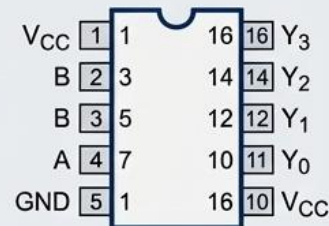
(1) Truth Table

Inputs			Outputs					
A	B	Binary Value	Y ₀	Y ₁	Y ₂	Y ₃	Selected Output	Description
0	0	00	0	0	1	0	Y ₀	Y ₀ active high
0	1	01	1	0	0	1	Y ₁	Y ₁ active low
1	0	10	0	1	0	0	Y ₂	Y ₂ active low
1	1	11	0	0	1	1	Y ₃	Y ₃ active low

(2) Logic Symbol



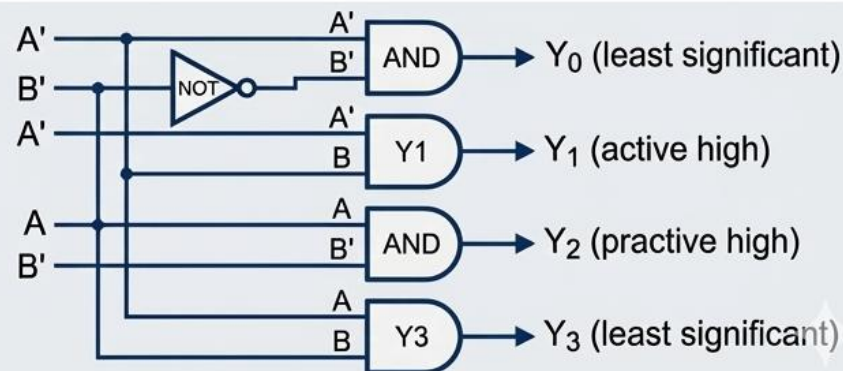
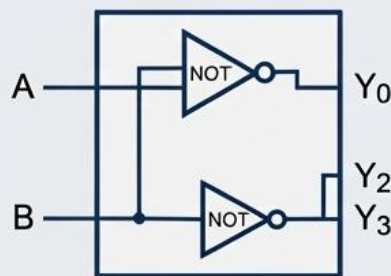
Logic Symbol



Typical 16-Pin DIP

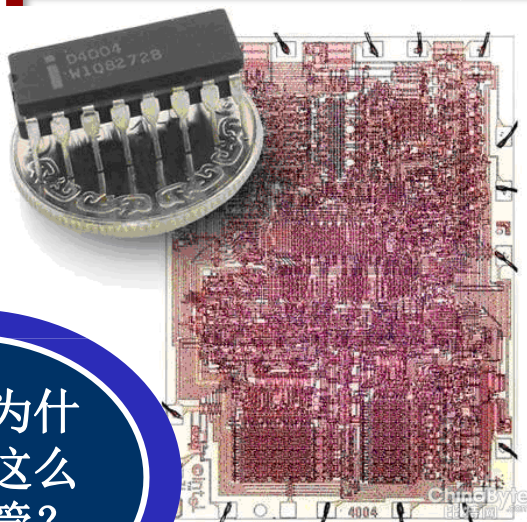
(3) Internal Logic Circuit Diagram

Overall logic circuit for 2-line-to-4-line output functionality



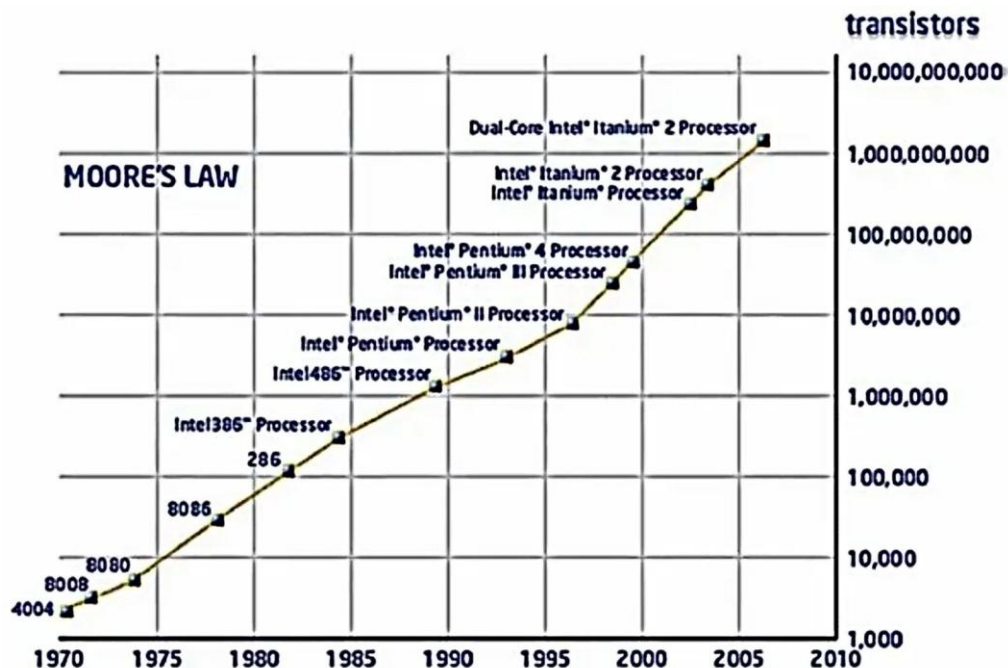
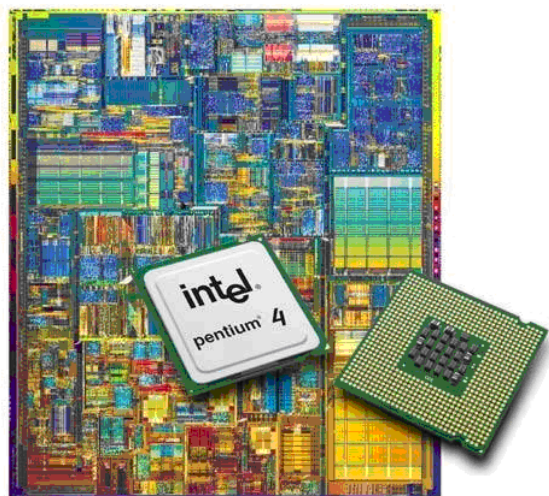


集成电路是这样复杂起来的



疑问：为什么需要这么多晶体管？

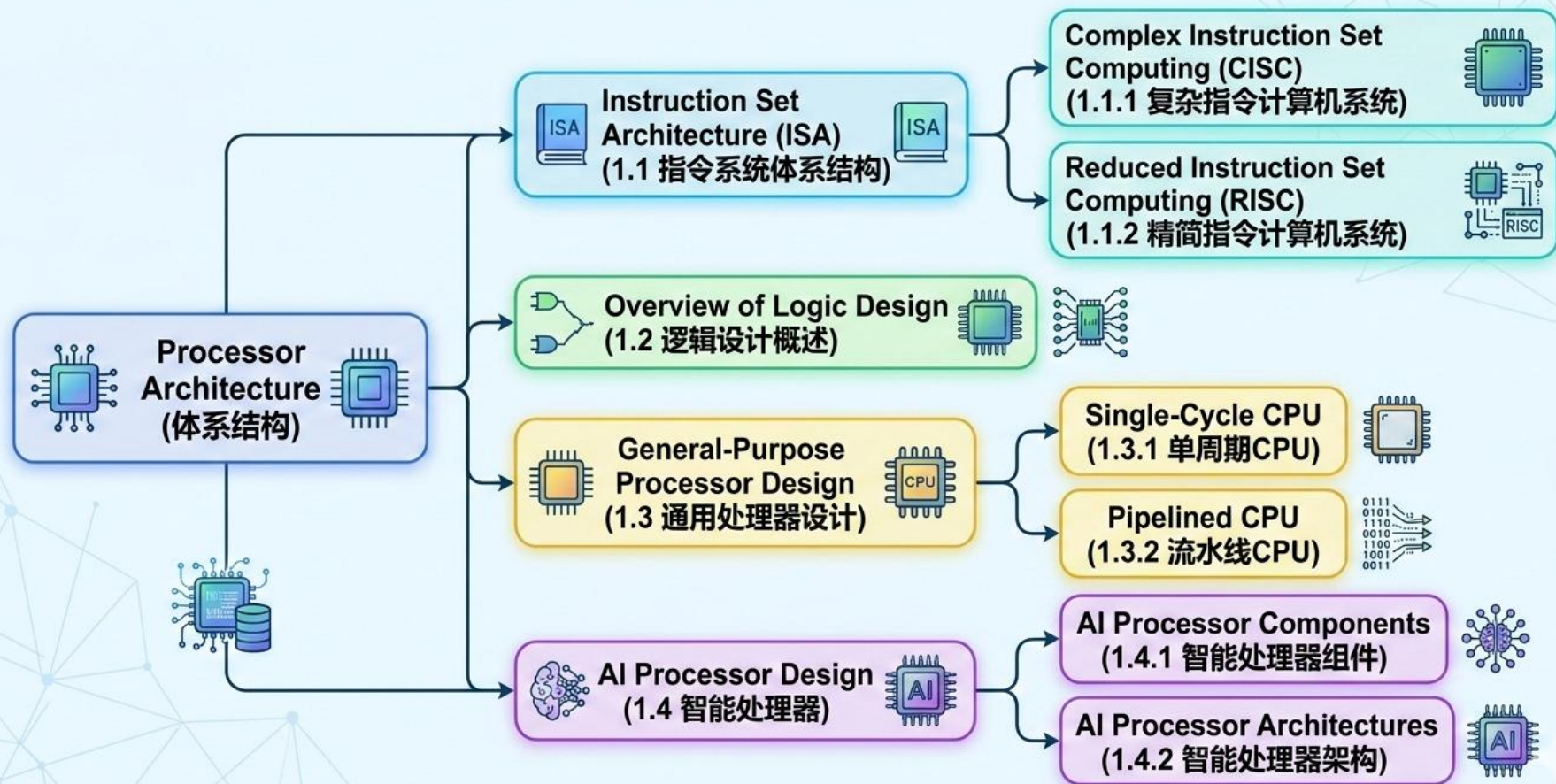
摩尔定律：芯片可容纳的晶体管数目大约每经过**18**个月便会增加一倍：**2021**年达到的规模一块芯片**4.6**万亿颗晶体管



从沙子到芯片-中芯国际

硬件体系结构

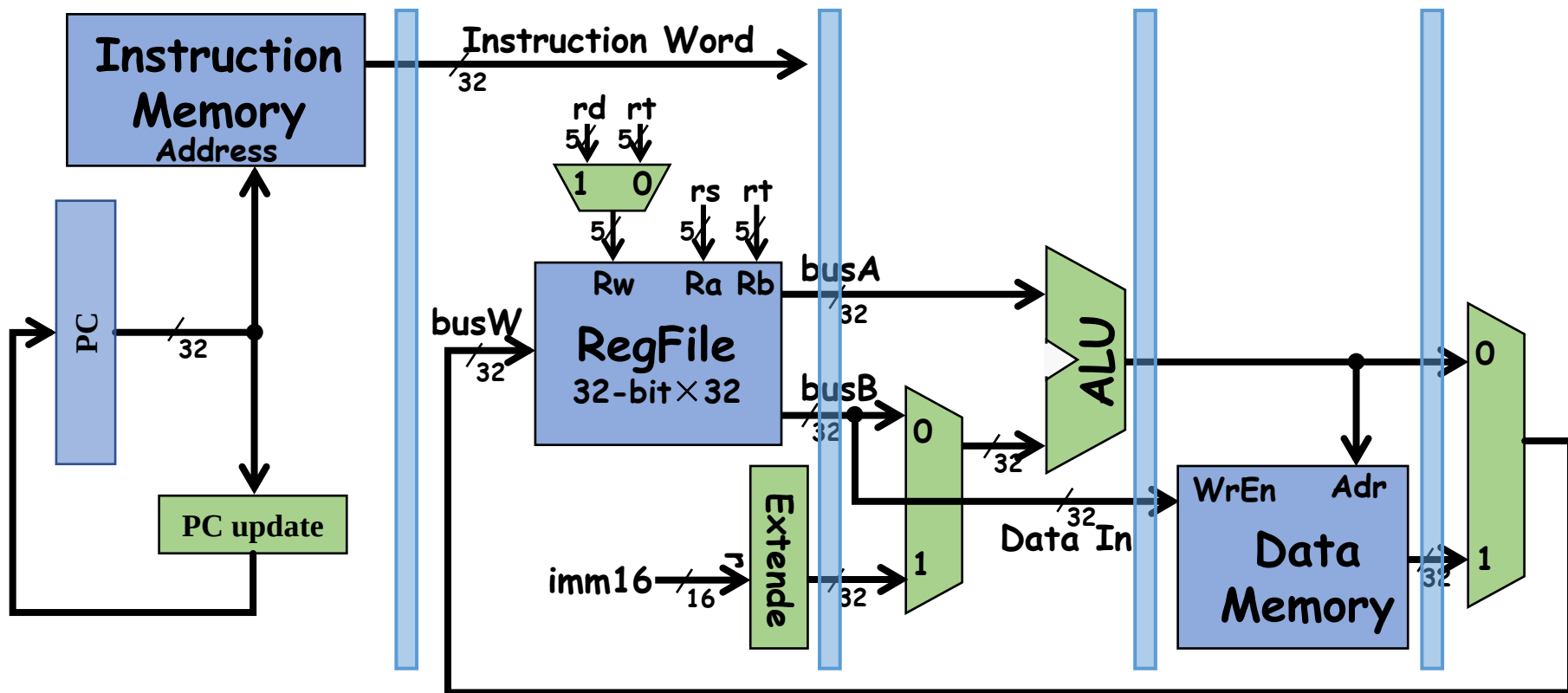
Technical Hierarchy Mind Map: Computer Architecture





添加流水线寄存器

取指 译码 执行 访存 写回



- 这不同阶段所用到的硬件资源基本上是相互独立的
- 拆分并增加级间寄存器，保存前一个阶段要向后一个阶段传送的所有的信息
- 从“输入变输出就变”的组合逻辑，变为时钟驱动的时序逻辑



LW 指令的执行过程

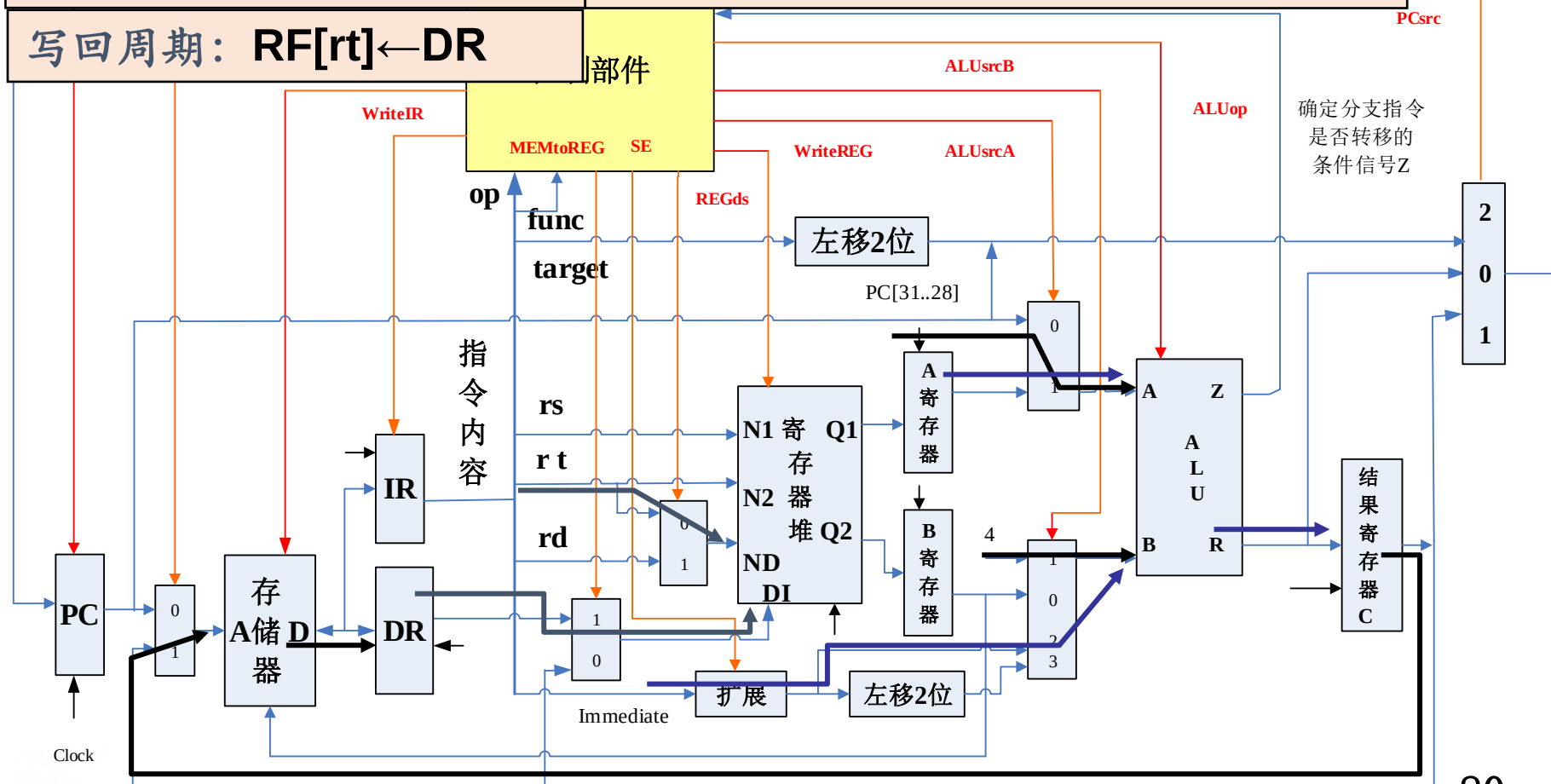
取指周期: $IR \leftarrow MEM[PC]$
 $PC \leftarrow PC + 4$

译码周期: $A \leftarrow RF[rs]$ $B \leftarrow RF[rt]$
 $C \leftarrow PC + SignExt(imm) \ll 2$

执行周期: $C \leftarrow A + SE(imm)$

访存周期: $DR \leftarrow MEM[C]$

写回周期: $RF[rt] \leftarrow DR$

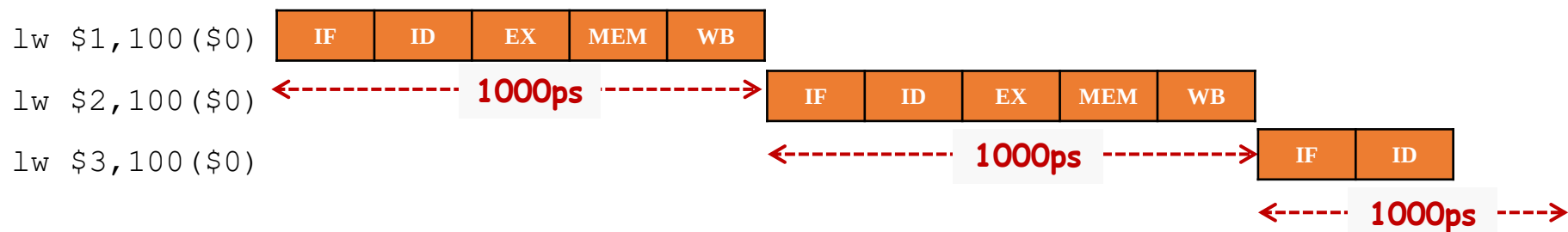




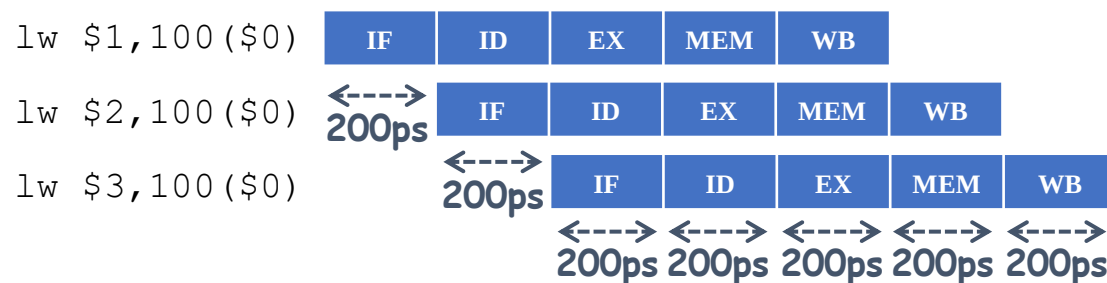
流水线处理器性能分析

时间 (ps) 0 200 400 600 800 1000 1200 1400 1600 1800 2000 2200 2400

单周期处理器 (时钟周期1000ps)



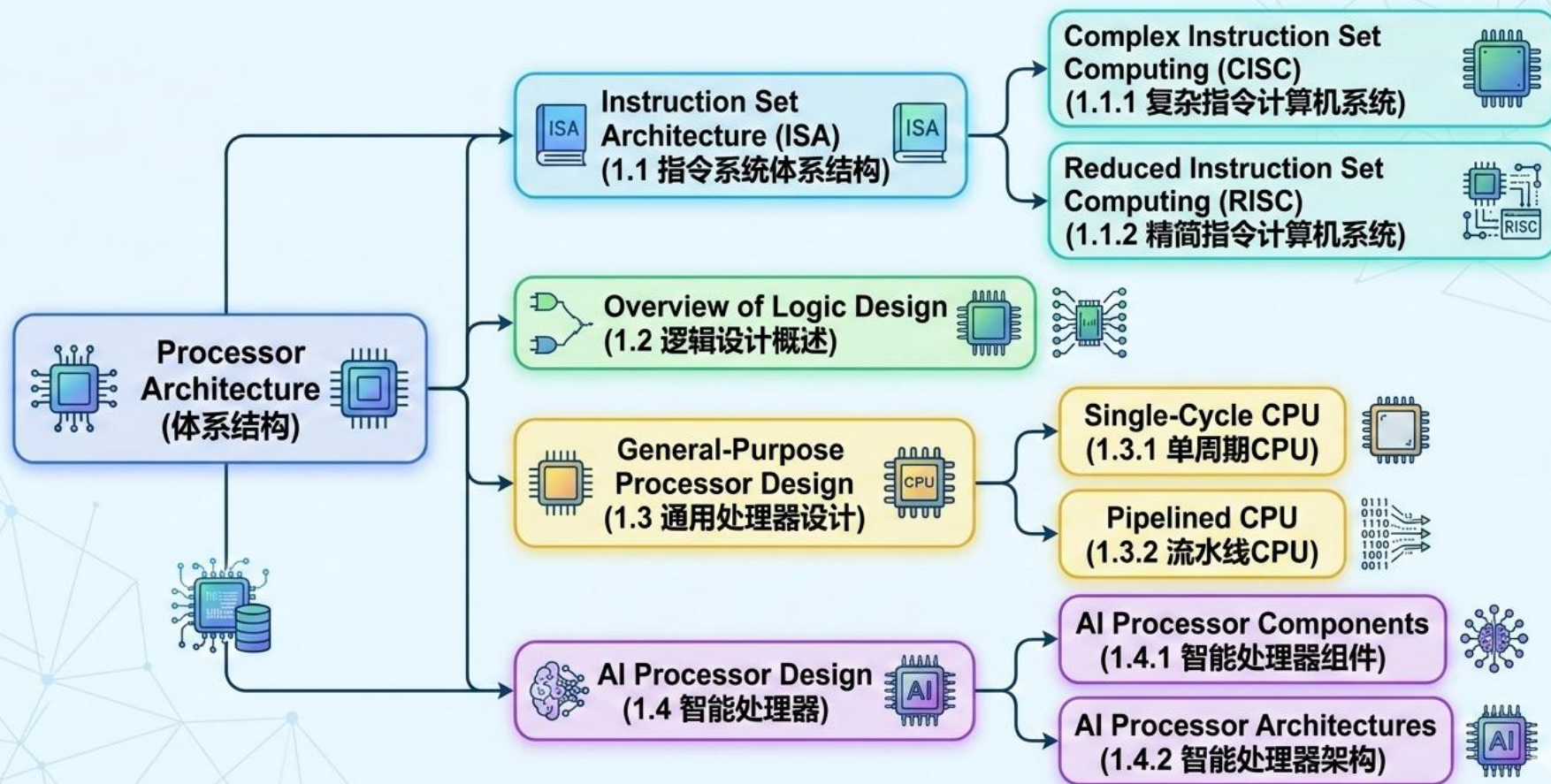
流水线处理器 (时钟周期200ps)





硬件体系结构

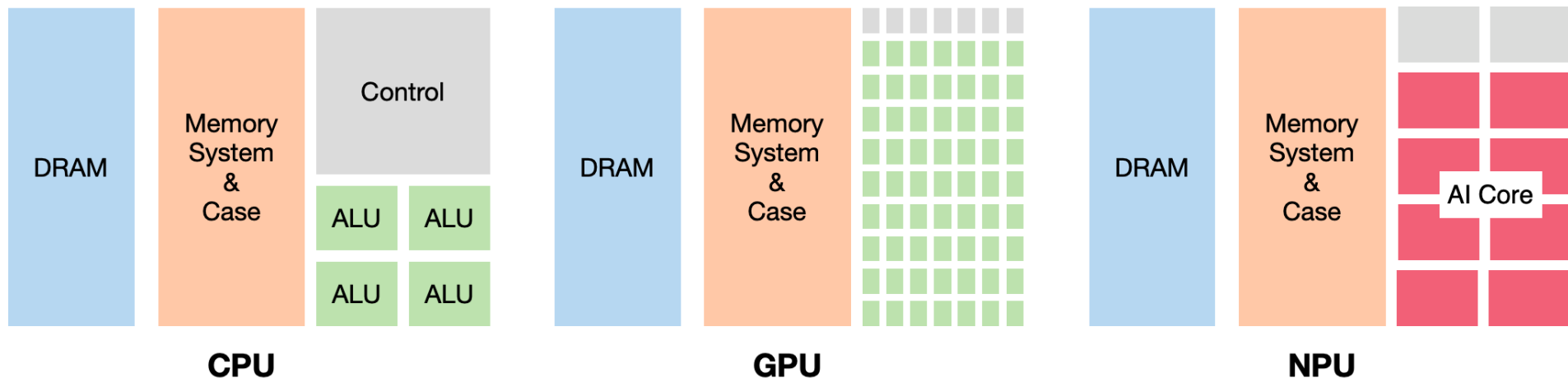
Technical Hierarchy Mind Map: Computer Architecture





什么是AI芯片

- **DSA**: 特定领域的体系结构，通常被称为加速器，因为与在通用 **CPU** 上执行整个应用程序相比，它们可以加速某些应用程序。
- **好处**: **DSA** 可以实现更好的性能，因为它们更贴近应用的实际需求；**DSA** 例子，包括图形加速单元（即**GPU**），用于深度学习的神经网络处理器（**NPU/TPU**），以及软件定义处理器（**SDN**）。





芯片场景

从**2017**年开始，苹果、华为海思、高通、联发科等主要芯片厂商相继发布支持**AI**加速功能的新一代芯片，**AI**芯片逐渐向中端产品渗透。

AI 应用和 **AI** 部署也将会呈现更多的结合性态。

AI 计算中心



自动驾驶



安防应用



IOT AI应用



芯片场景

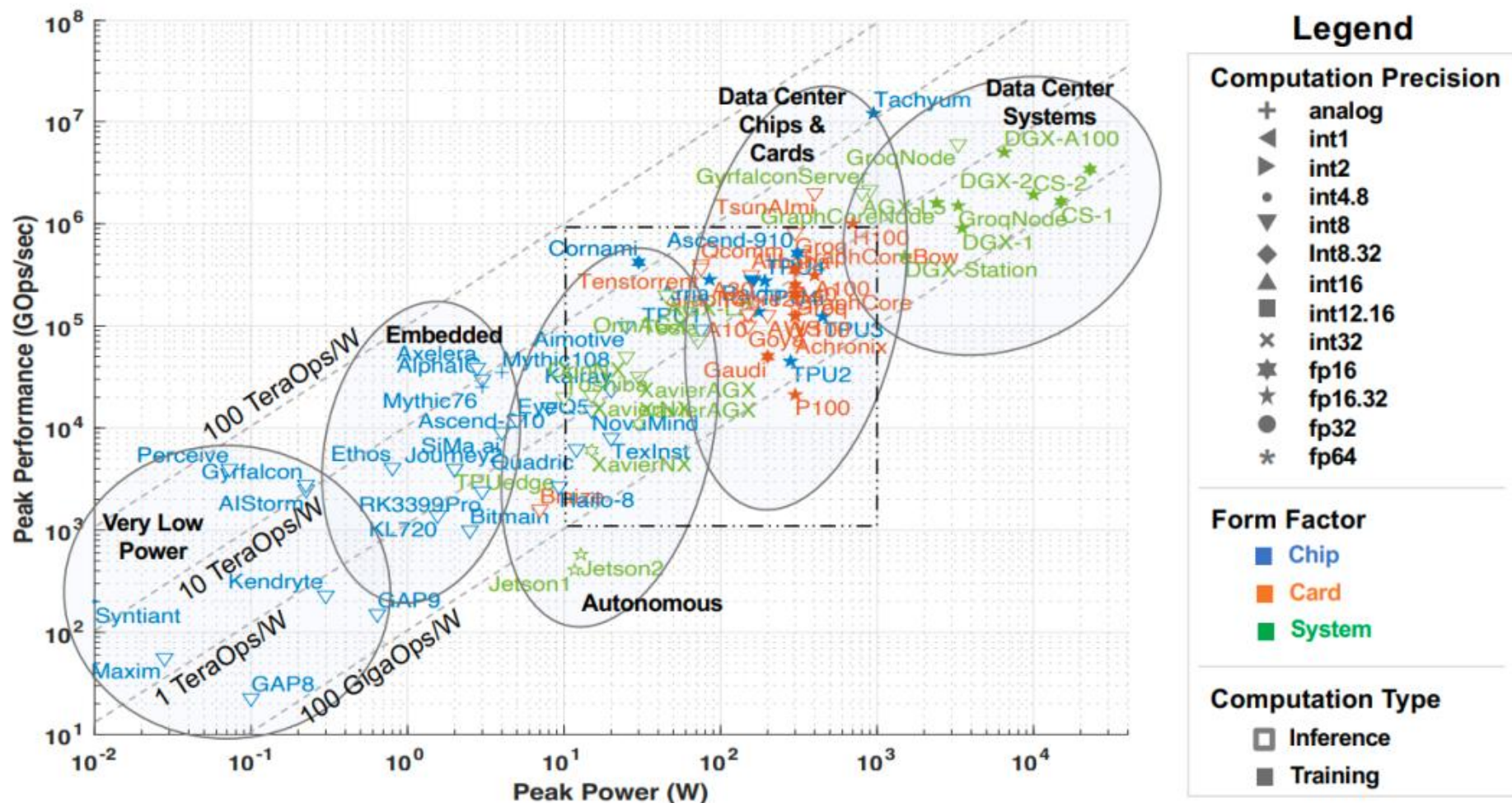
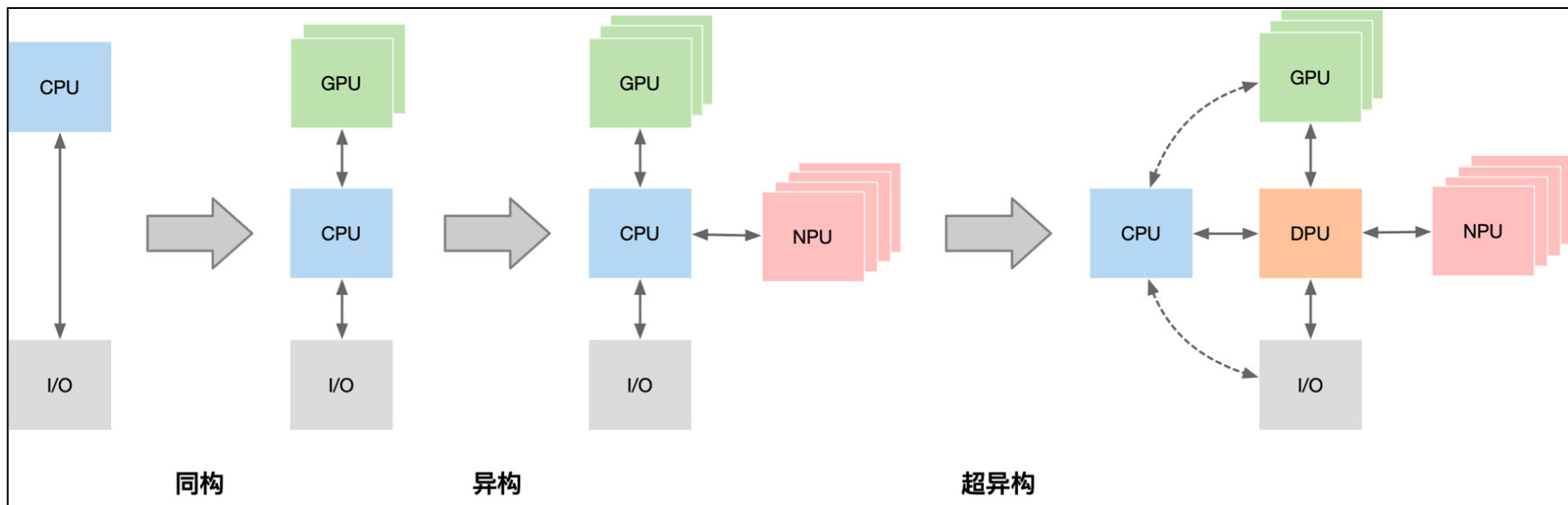


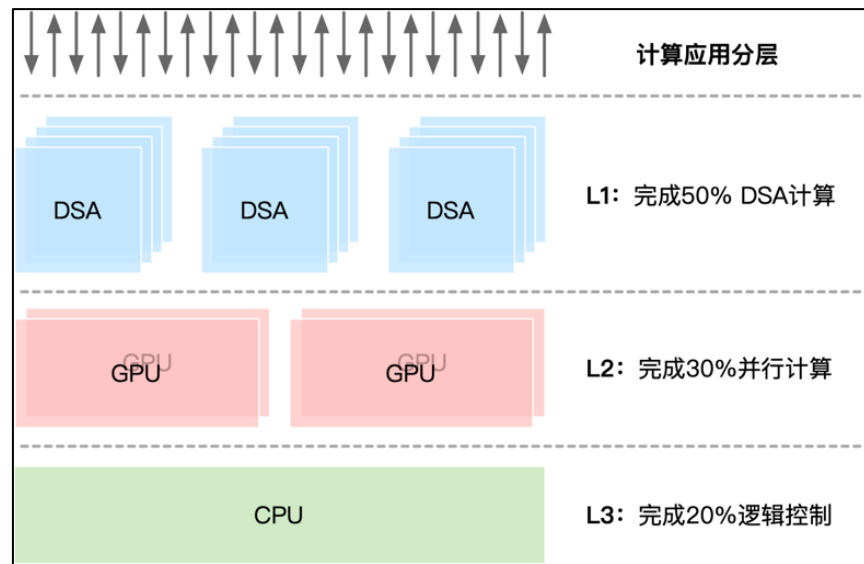
Fig. 2: Peak performance vs. power scatter plot of publicly announced AI accelerators and processors.



异构系统



- 计算从单核的串行走向多核的并行；又进一步从同构并行走向异构并行。
- 计算需要进一步从异构并行走向超异构并行。异构计算是 **CPU + XPU** 的两个层次的处理器类型，而超异构计算则是 **CPU + GPU + DSA** 三个层次的处理器类型。

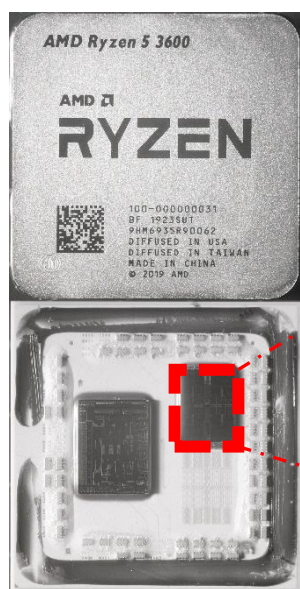




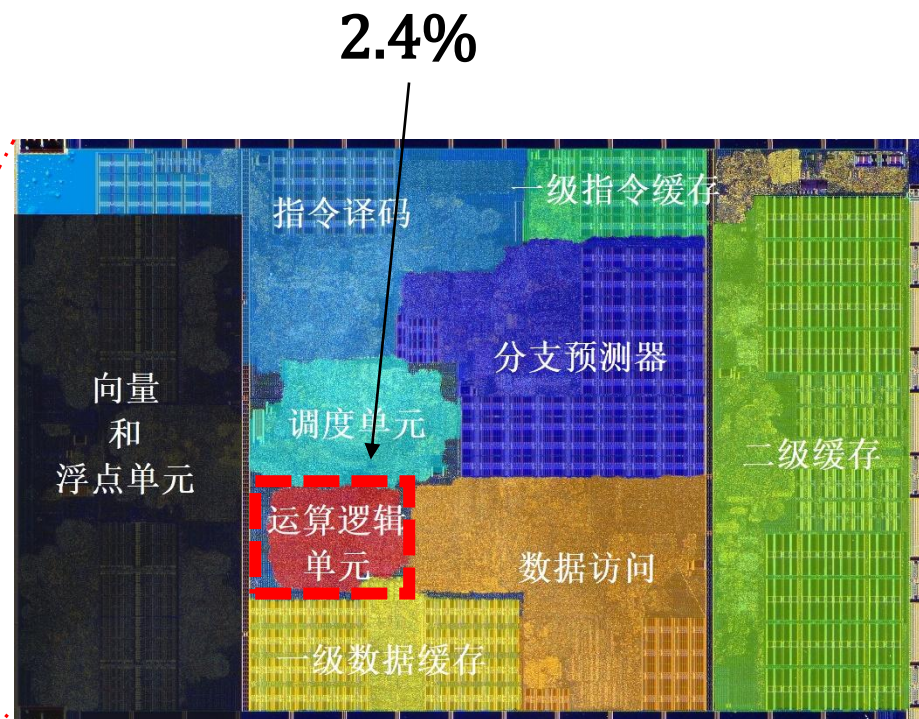
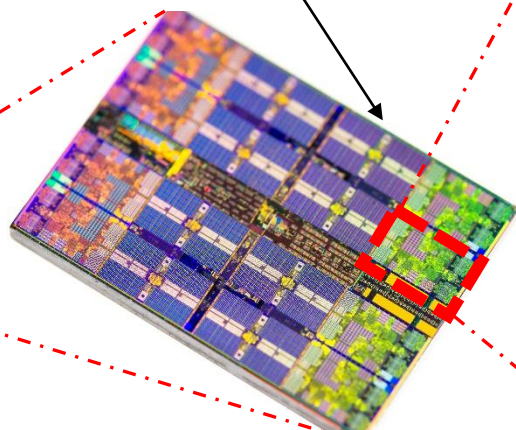
提纲

- 从通用处理器到智能加速器的演变
- 从向量到GPU的发展
- AI芯片设计
 - 计算
 - 存储
 - 互联
- 性能指标
- 未来展望

真实的处理器例子



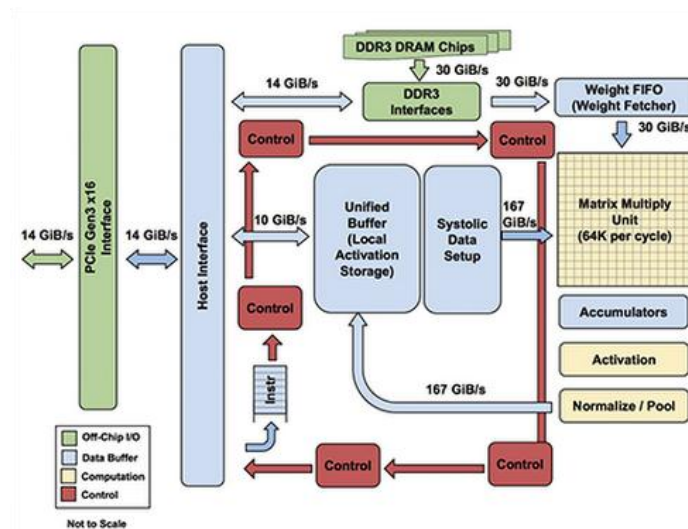
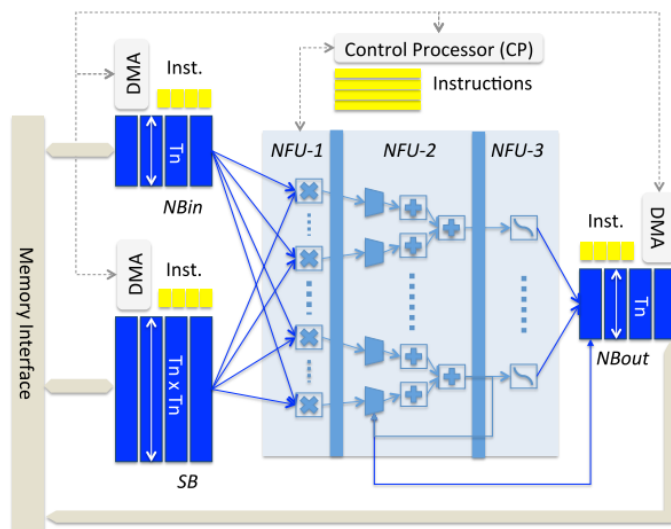
$$5\% \times 8 = 39\%$$



用于运算的芯片面积不到1%！

AI芯片样例

- TPU (Google, 2016)
- DOJO (特斯拉, 2019)
- Volta GPU (NVIDIA, 2017)
- 昇腾 NPU (华为, 2018)
- DianNao (寒武纪, 2016)
- 天数
- 曙光
- 摩尔线程

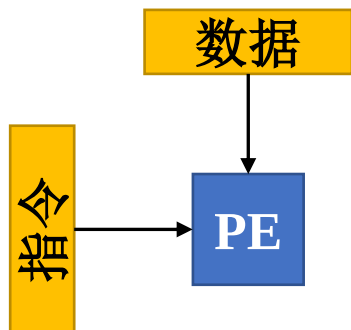




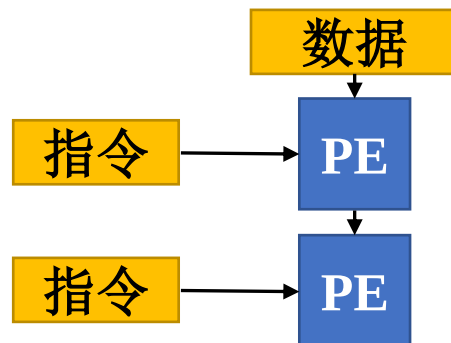
SIMD

- 来源于Flynn分类法（1972）

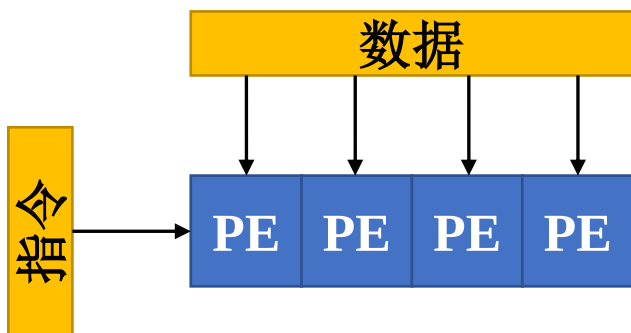
- 单指令流、单数据流（SISD）



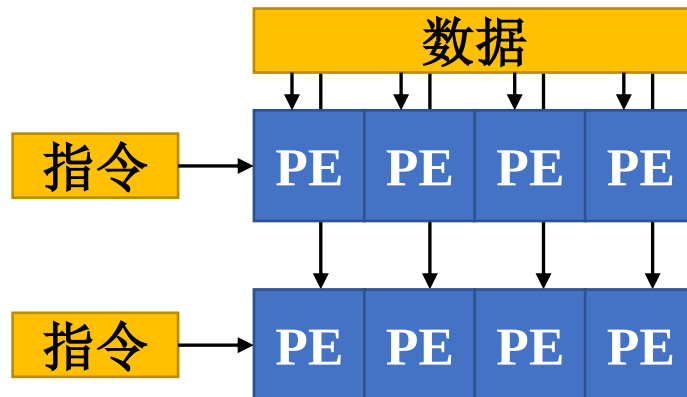
- 多指令流、单数据流（MISD）



- 单指令流、多数据流（SIMD）

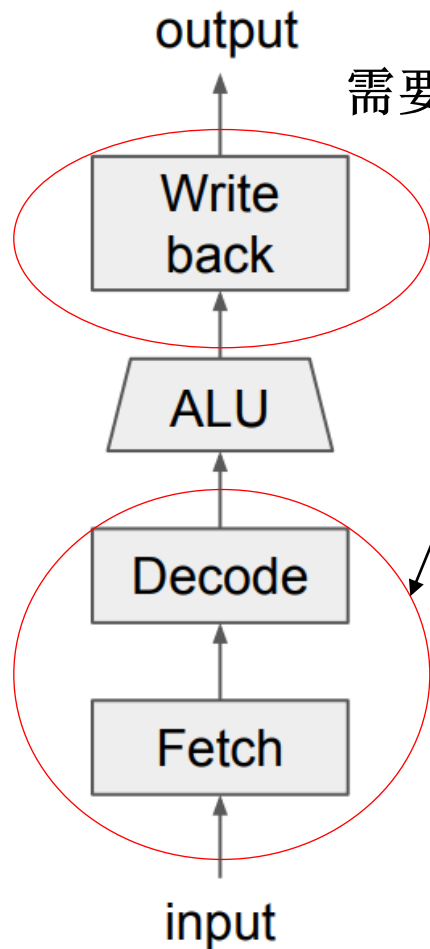


- 多指令流、多数据流（MIMD）

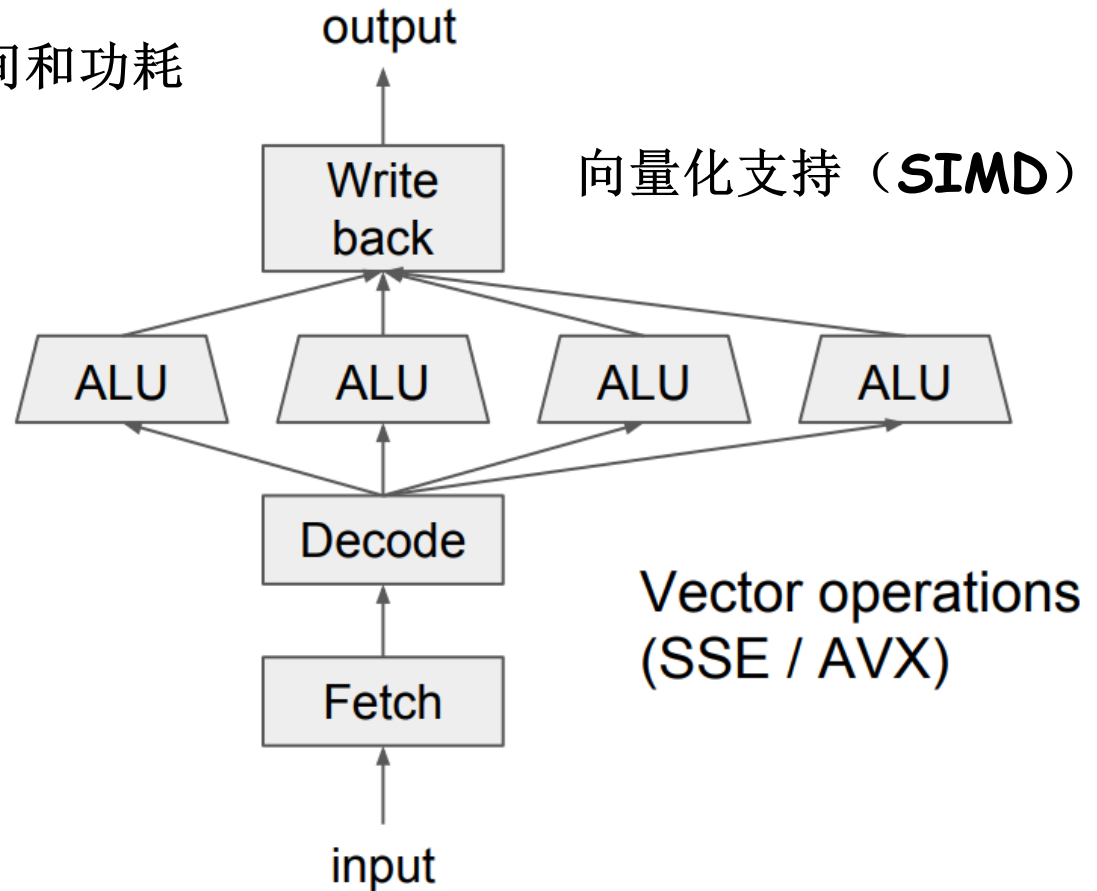




向量化执行



需要花费大量时间和功耗

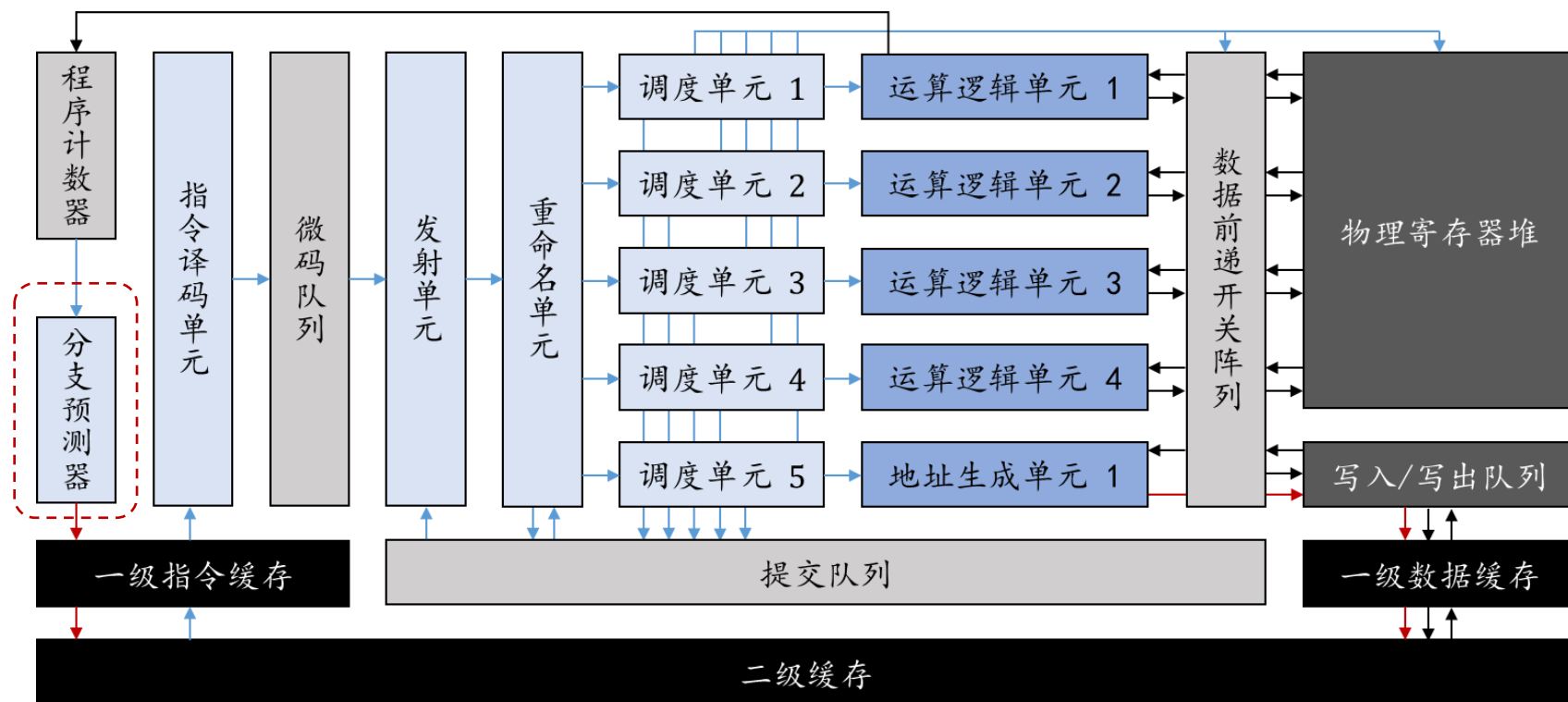




通用处理器结构

分支预测：未确定跳转方向时，按猜测方向投机执行

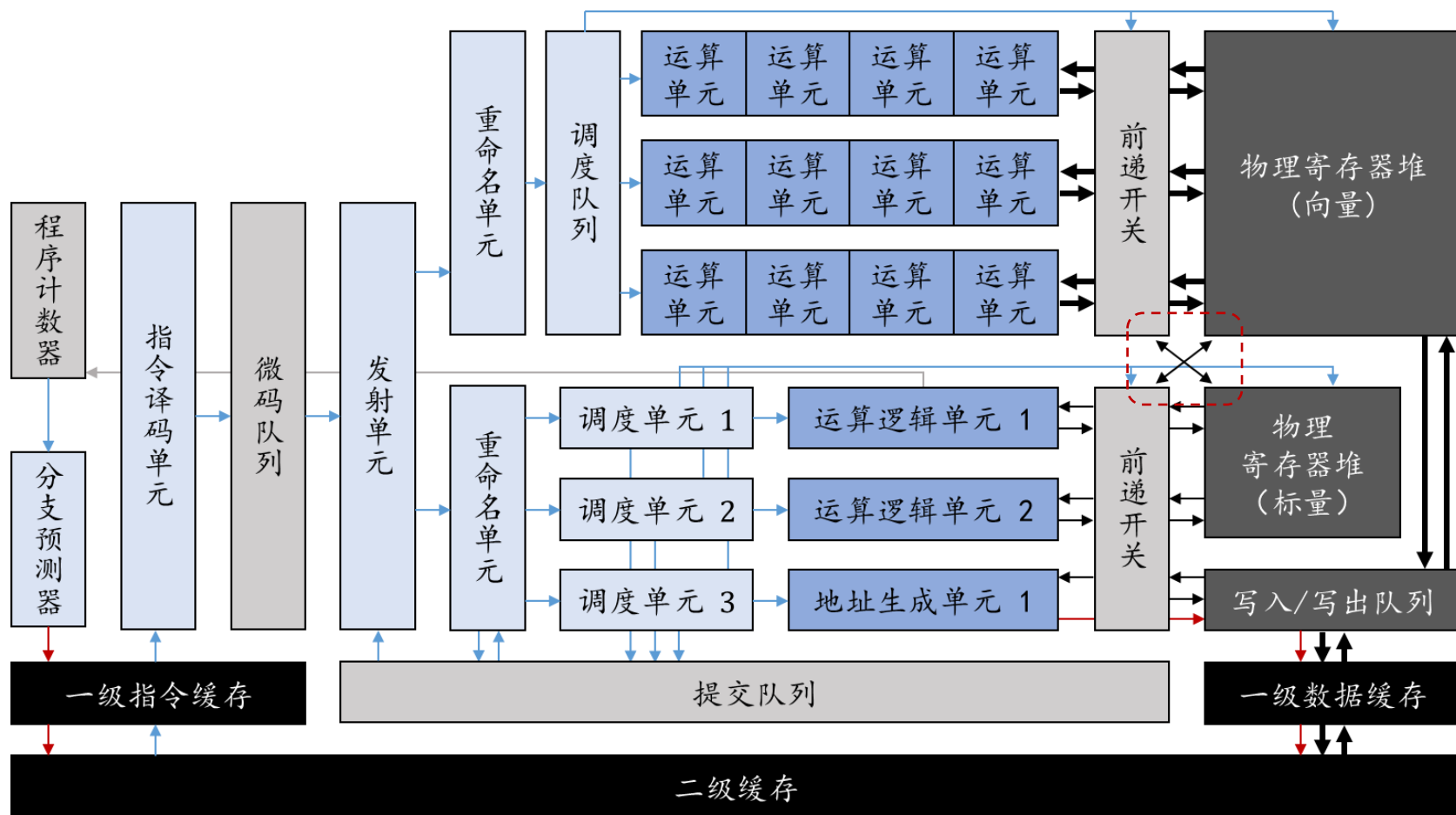
- 预测正确时：减少等待时间，提高了流水线效率
- 预测错误时：不予提交，撤销错误执行的指令





向量处理器结构

可以增加向量/标量数据交换通路，方便控制

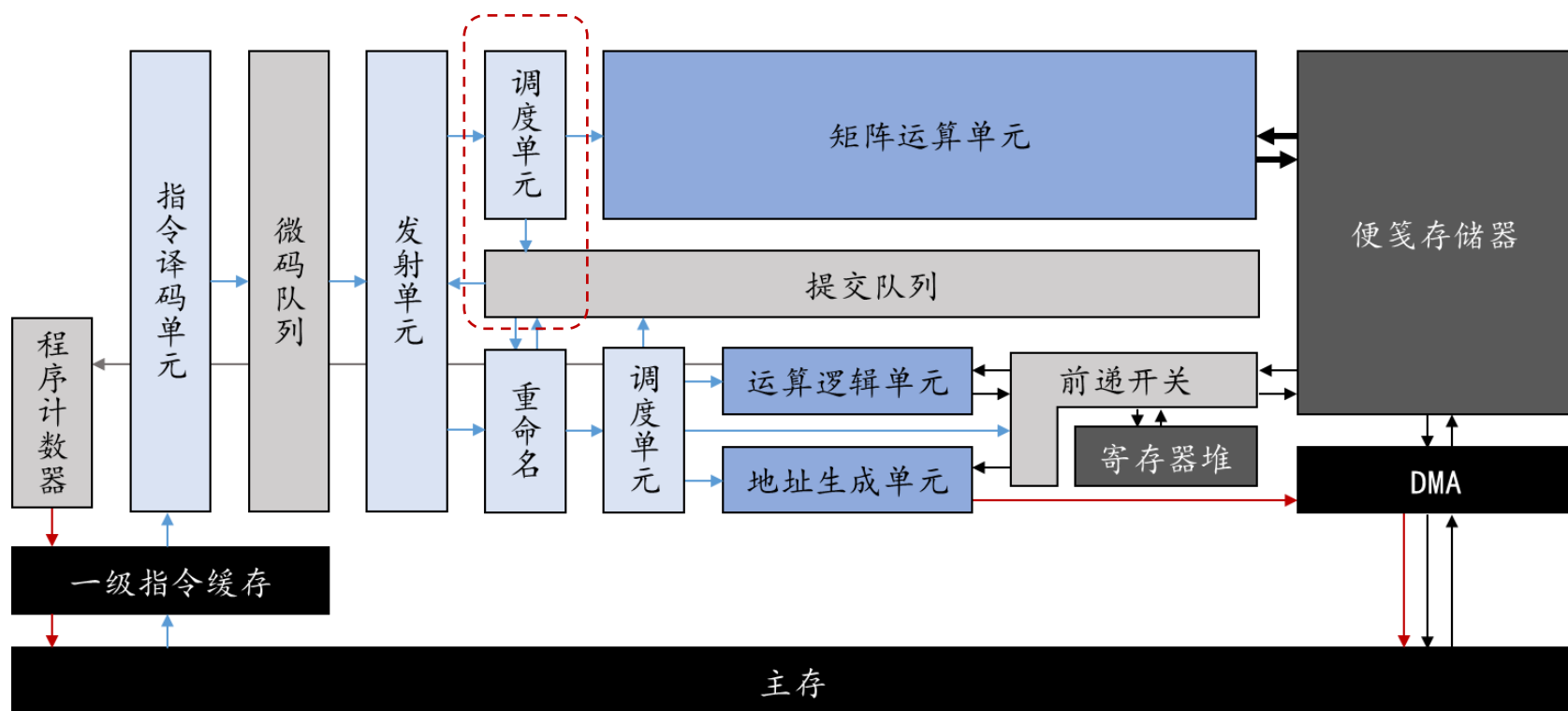




智能加速结构

添加矩阵指令的控制单元

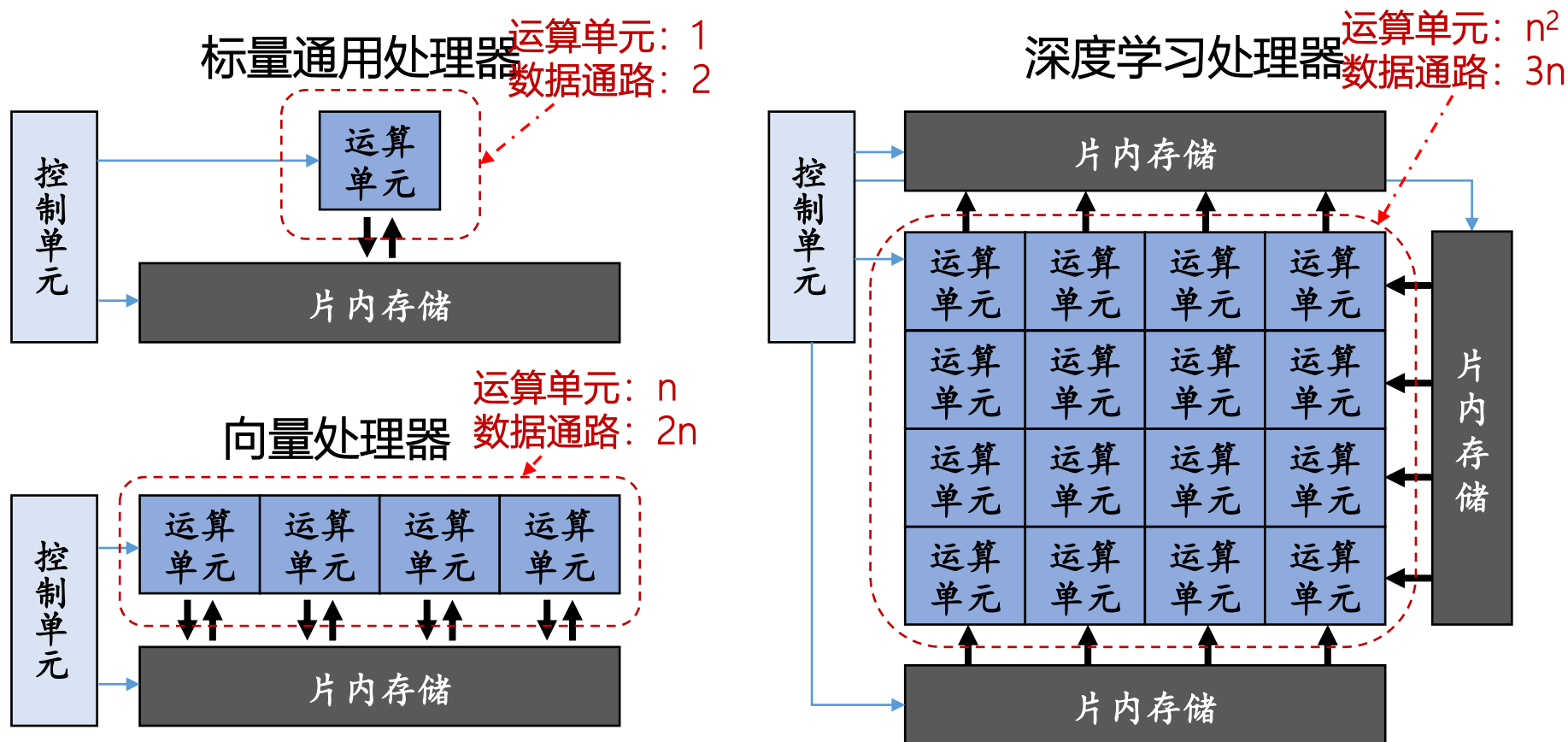
- 矩阵指令在提交队列中，与标量/访存指令同步





深度学习处理器（DLP）

- 基本运算单元从向量运算扩展到矩阵运算



- 深度学习处理器在相同带宽下，提供更高运算能力

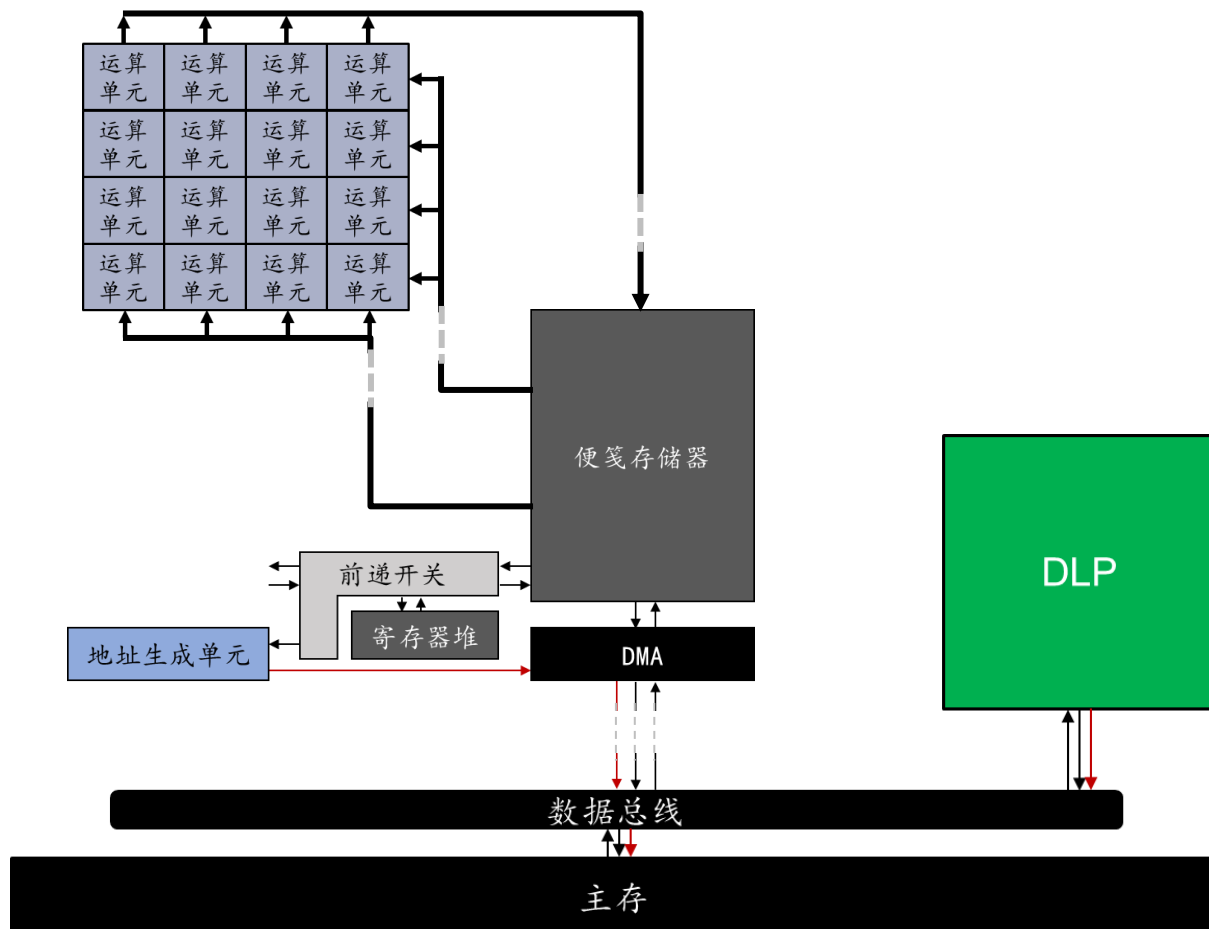


总体架构

• 计算

• 访存

• 通信

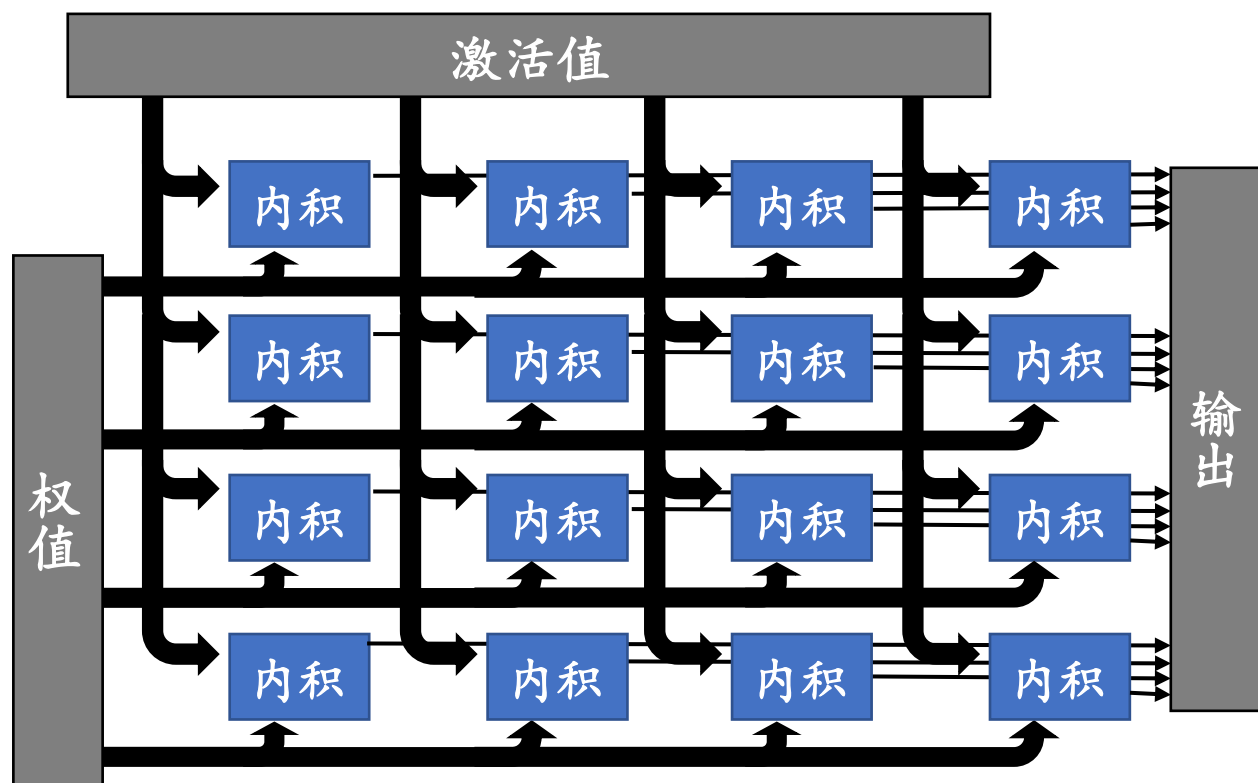




矩阵运算单元

- 扩大规模

矩阵乘矩阵单元

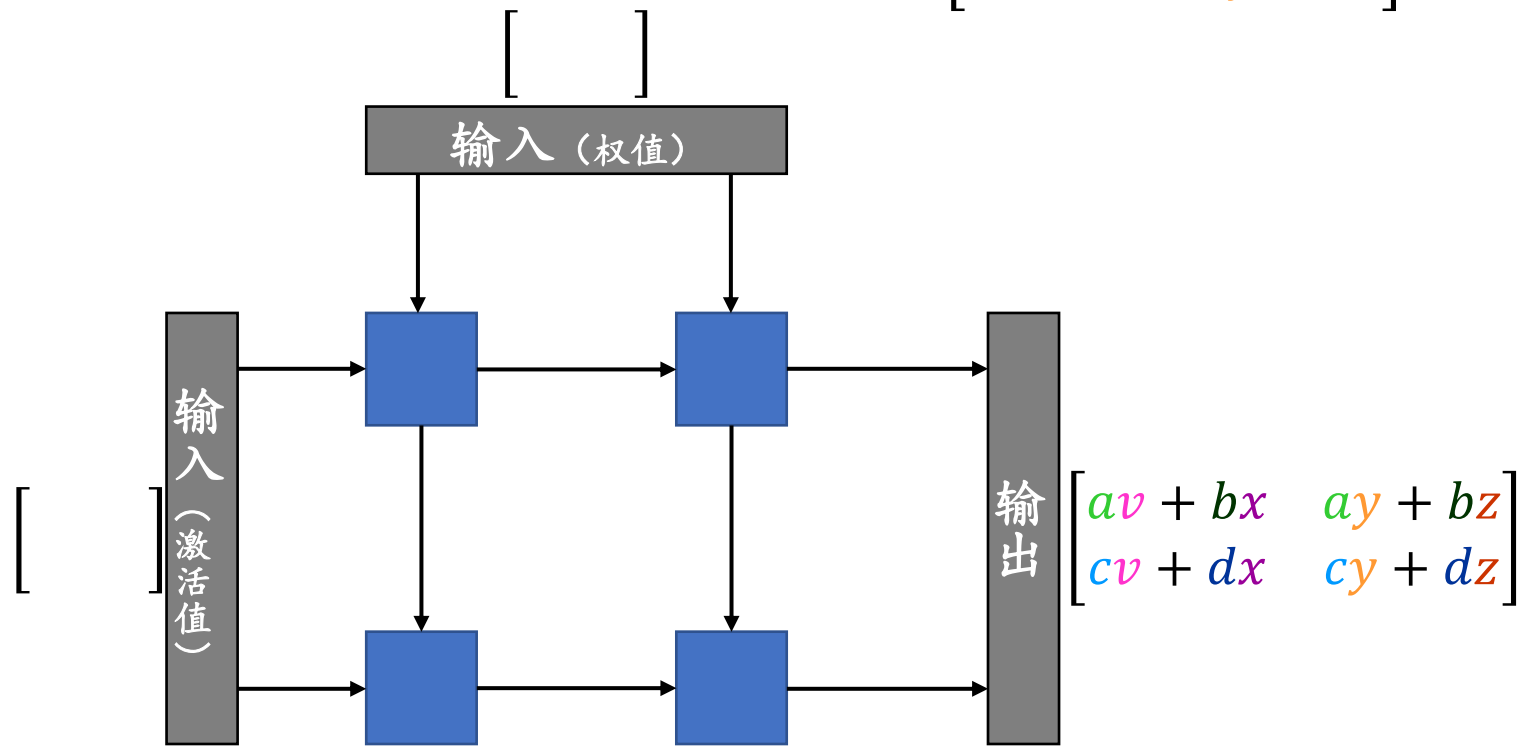


计算-I/O比例 = 112:48 = 1:0.4



脉动阵列机

如何完成矩阵运算? $\begin{bmatrix} a & b \\ c & d \end{bmatrix} \times \begin{bmatrix} v & y \\ x & z \end{bmatrix} = \begin{bmatrix} av + bx & ay + bz \\ cv + dx & cy + dz \end{bmatrix}$

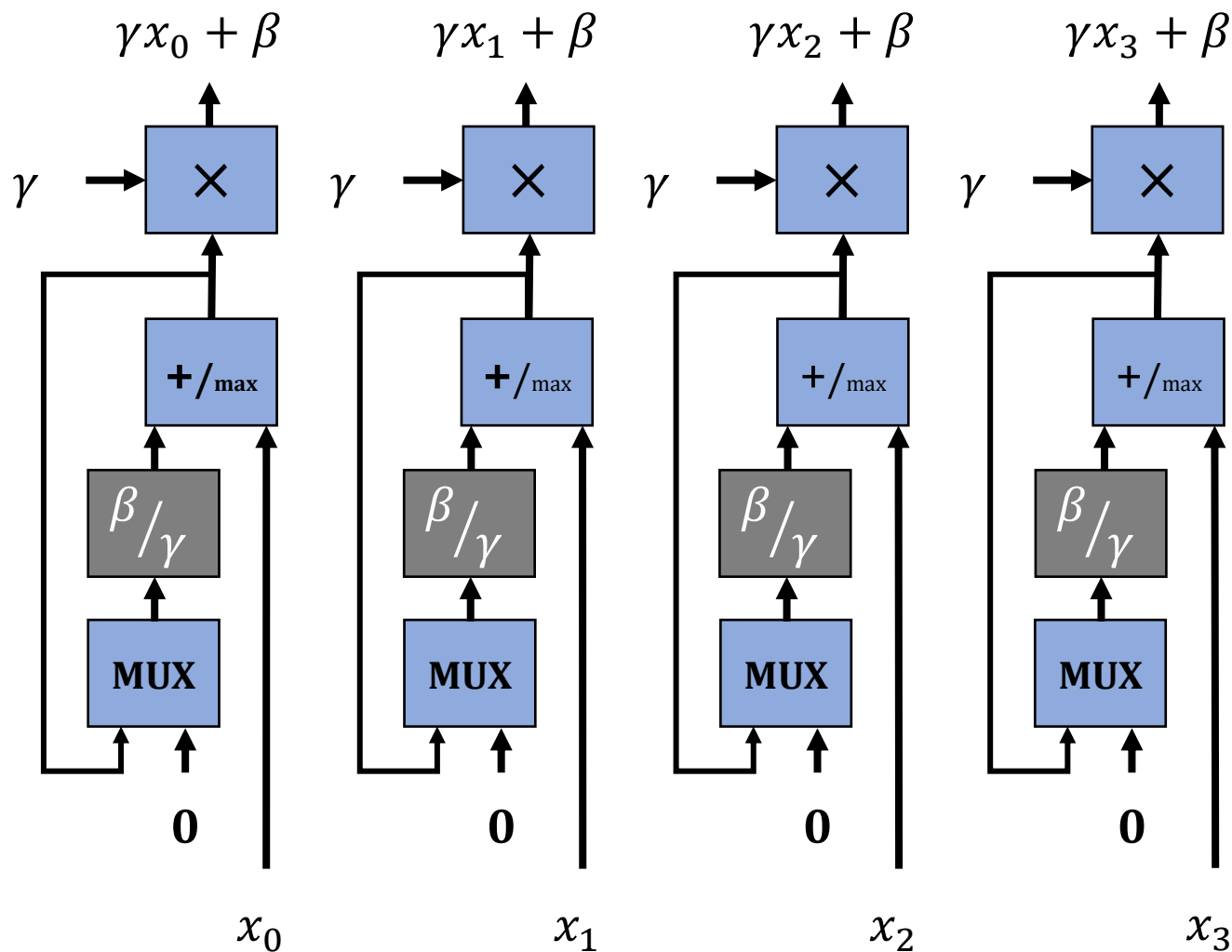


计算-I/O比例 = 1:0.7



池化/均一化

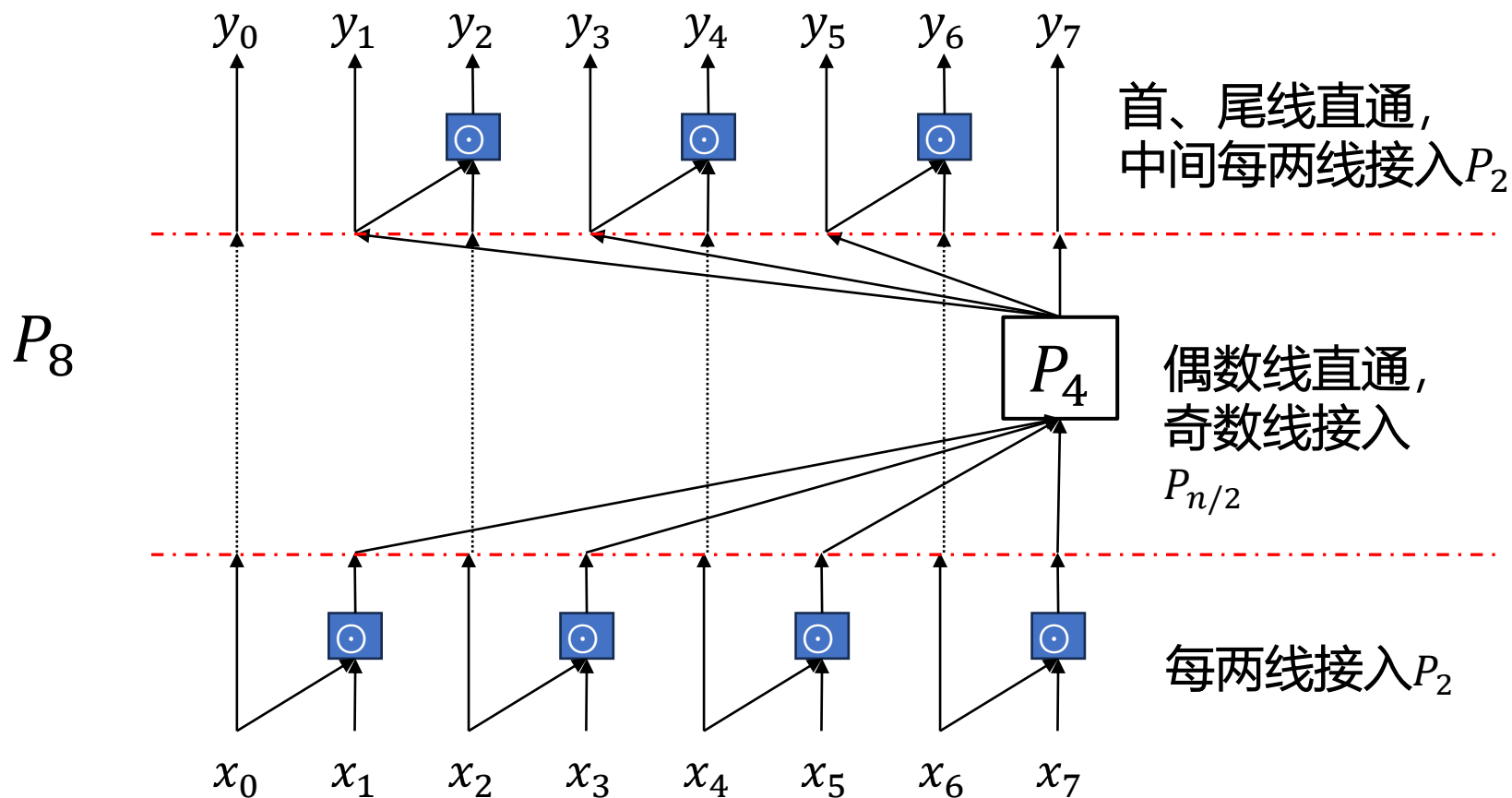
- 支持AvgPool、MaxPool、BatchNorm





前缀计算

- 实现很简单。有高效的方式吗？





访存

- 容量失效：循环访问数据超过缓存容量，LRU缓存失效

缓存容量32字

4×4方阵



高效

5×5方阵



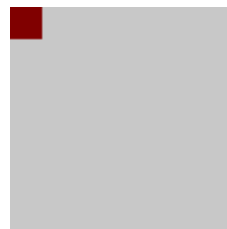
高效

6×6方阵



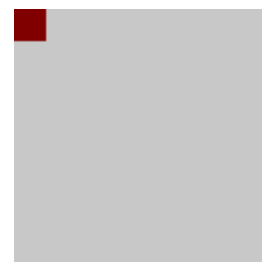
失效

7×7方阵



失效

8×8方阵

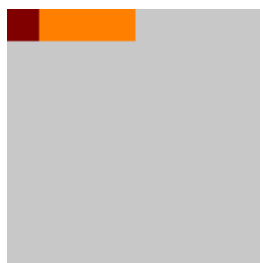


失效

- 访问放大：逐字访问不命中时，缓存以行为单位替换

缓存行4字

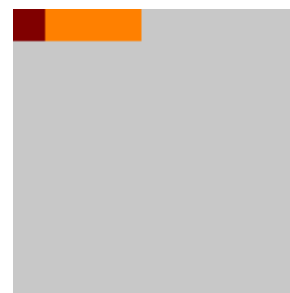
缓存容量32字



8×8方阵

缓存访问量 64字

内存访问量 64字



9×9方阵

缓存访问量 81字

内存访问量 324字



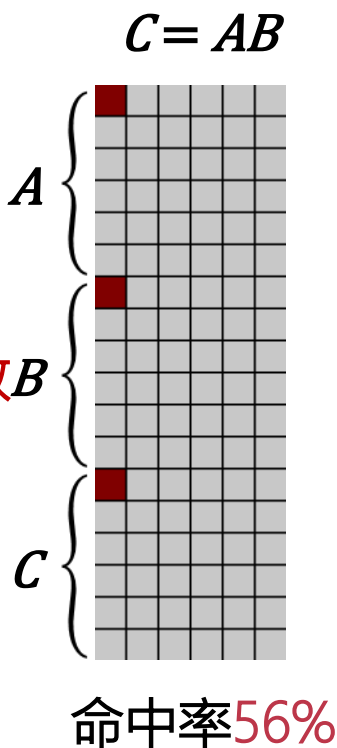
访存优化

- 分块 (tiling) : 将运算分解至固定尺寸的区域处理

不分块:

三重循环

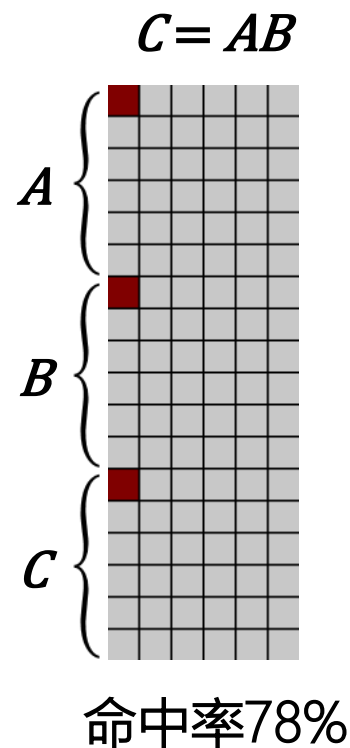
B 矩阵发生容量失效



分为 3×3 区块:

六重循环:

- 三重循环遍历各个分块
 - 分块内正常运算仍需三重
- 避免了前述各项缓存问题
程序更复杂

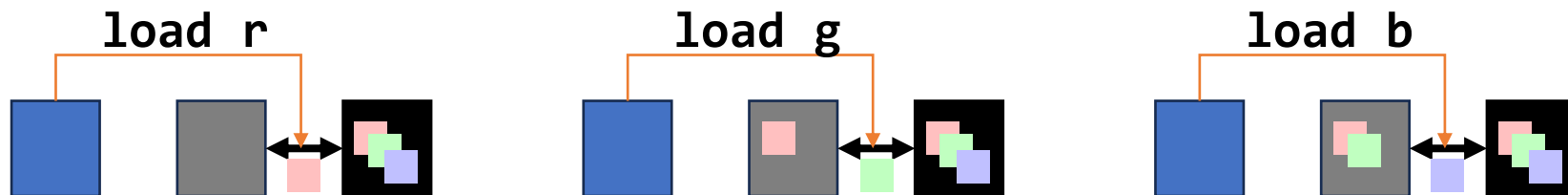




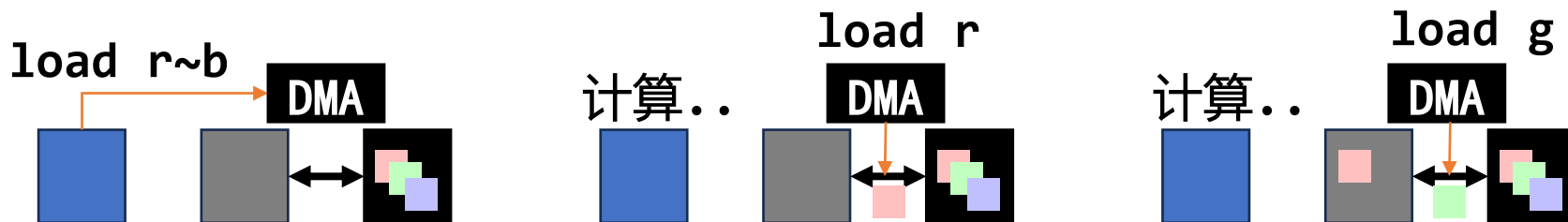
直接内存访问（DMA）

如何实现“1条load指令装载一整块图像”？

- 处理器控制：



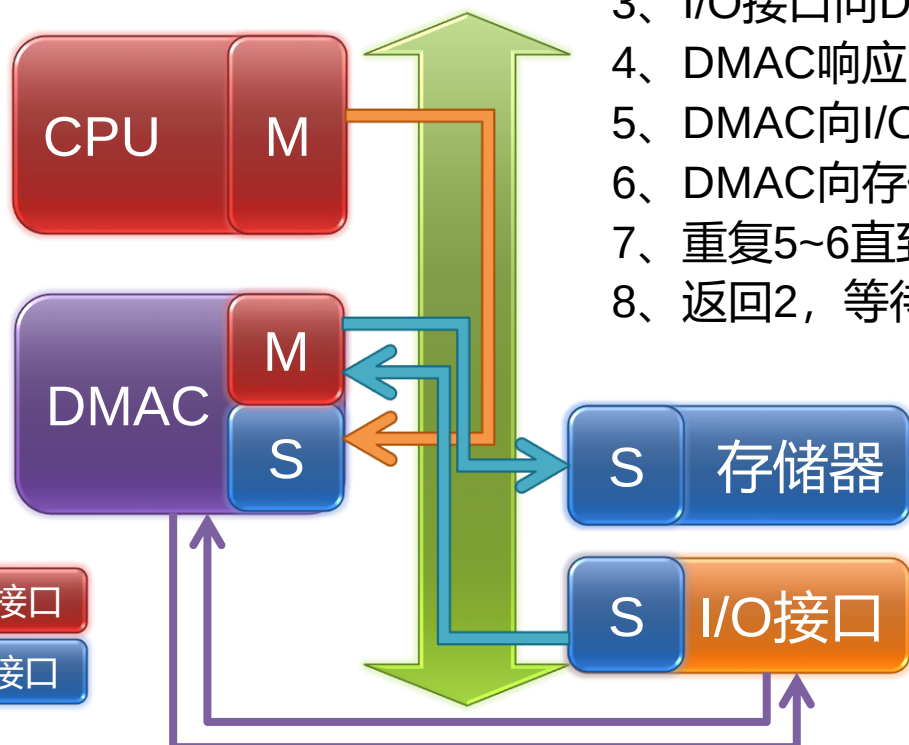
- DMA控制：





DMAC的基本工作步骤

以使用**独立DMAC**进行
外设到内存的传送为例



- 1、CPU设置DMAC内部配置寄存器
- 2、DMAC处于空闲等待状态
- 3、I/O接口向DMAC发出DMA传送申请
- 4、DMAC响应I/O接口的申请
- 5、DMAC向I/O接口发起总线读传输
- 6、DMAC向存储器发起总线写传输
- 7、重复5~6直到本次DMA传送完成
- 8、返回2，等待下一次DMA传送申请

注：通常，DMA传输完成后，DMAC会通过中断信号通知CPU

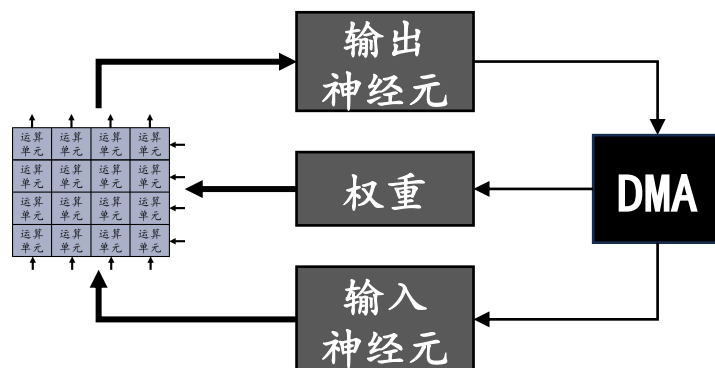
- CPU设置DMAC内部配置寄存器，一般包括：
 - ① 源地址的初始值及传送时的地址增减方式
 - ② 目的地址的初始值及传送时的地址增减方式
 - ③ 待传送数据的长度



分离式便笺存储器

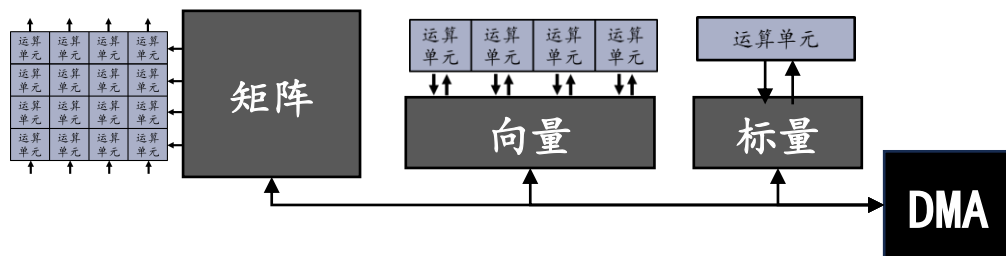
- 按数据划分

- 神经元/权值
- 输入神经元/输出神经元/权值



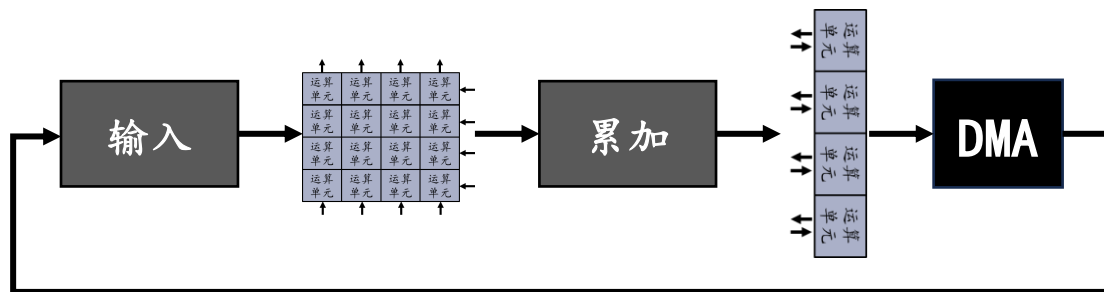
- 按功能单元划分

- 向量/标量
- 矩阵/向量/标量



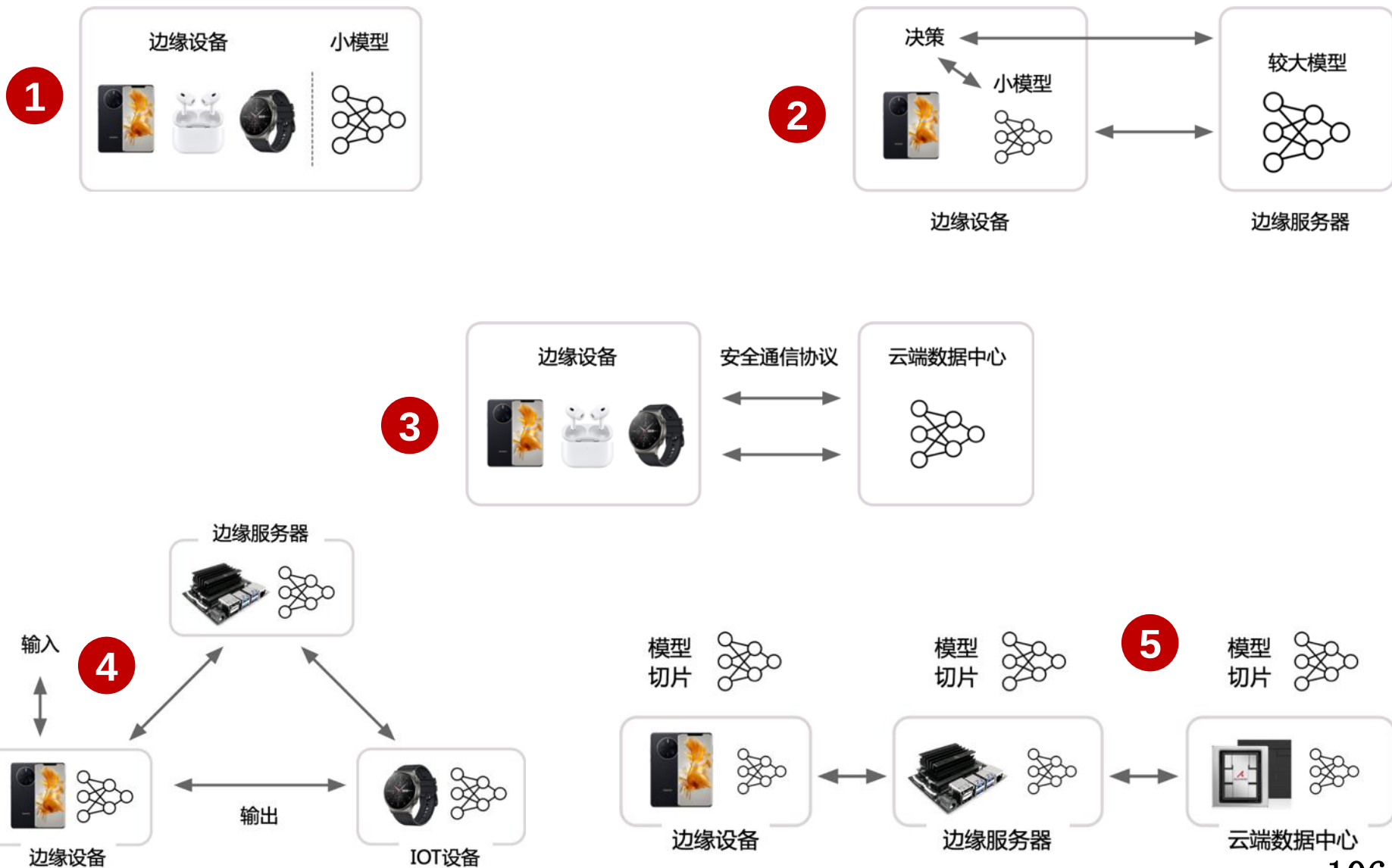
- 按处理阶段划分

- 输入数据/累加器





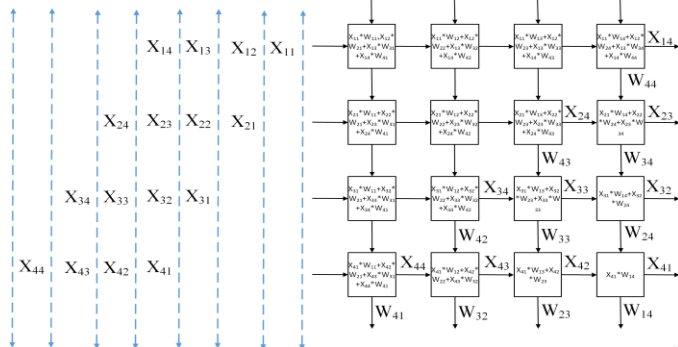
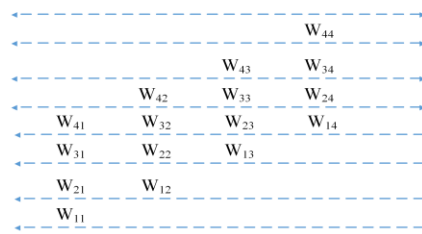
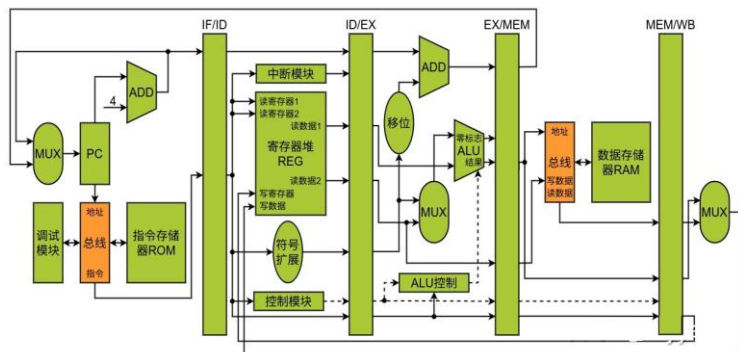
AI 芯片部署方式：云、边、端





实验：大模型辅助的图形化异构处理器设计

- 图形编码与开发板实践双轮驱动
- 图形化设计
 - 解决硬件描述语言学习难度大的问题
 - 解决大模型直接生成代码缺乏思考的问题
- 开发板实践
 - 嵌入式开发板创新智能应用：养老、教育、医疗等领域
 - 基于华为昇腾310，熟悉国产硬件和生态





提纲

01 课程建设背景

02 国内外课程情况

03 AI技术栈和课程建设思路

04 硬件体系结构与 AI 芯片

05 AI 编译和算子优化

06 并行计算和分布式训练

07 课程思政



为什么需要编程框架？

计算误差对权重 w_{ij} 的偏导数是两次使用链式法则得到的：

$$\frac{\partial E}{\partial w_{ij}} = \frac{\partial E}{\partial o_j} \frac{\partial o_j}{\partial \text{net}_j} \frac{\partial \text{net}_j}{\partial w_{ij}}$$

在右边的最后一项中，只有加权和 net_j 取决于 w_{ij} ，因此

$$\frac{\partial \text{net}_j}{\partial w_{ij}} = \frac{\partial}{\partial w_{ij}} \left(\sum_{k=1}^n w_{kj} o_k \right) = o_i.$$

神经元 j 的输出对其输入的导数就是激活函数的偏导数（这里假定使用逻辑函数）：

$$\frac{\partial o_j}{\partial \text{net}_j} = \frac{\partial}{\partial \text{net}_j} \varphi(\text{net}_j) = \varphi(\text{net}_j)(1 - \varphi(\text{net}_j))$$

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from mpl_toolkits.mplot3d import axes3d
4 from matplotlib import style
5
6 def SGD(samples, y, step_size=2, max_iter_count=1000):
7     m, var = samples.shape
8     theta = np.zeros(2)
9     y = y.flatten()
10    #进入循环内
11    loss = 1
12    iter_count = 0
13    iter_list=[]
14    loss_list=[]
15    theta1=[]
16    theta2=[]
```

有必要将算法中的常用操作封装成组件提供给程序员，以提高深度学习算法开发效率

但如果 j 是网络中任一内层，求 E 关于 o_j 的导数就不太简单了。

考虑 E 为接受来自神经元 j 的输入的所有神经元 $L = u, v, \dots, w$ 的输入的函数，

$$\frac{\partial E(o_j)}{\partial o_j} = \frac{\partial E(\text{net}_u, \text{net}_v, \dots, \text{net}_w)}{\partial o_j}$$

关于 o_j 取全微分，可以得到该导数的一个递归表达式：

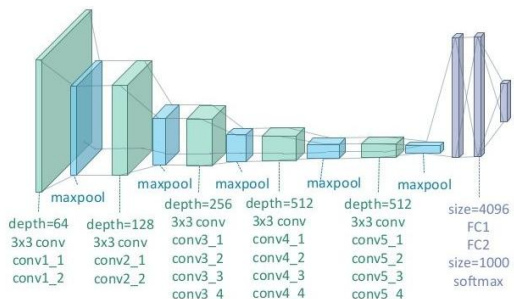
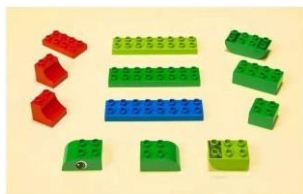
$$\frac{\partial E}{\partial o_j} = \sum_{l \in L} \left(\frac{\partial E}{\partial \text{net}_l} \frac{\partial \text{net}_l}{\partial o_j} \right) = \sum_{l \in L} \left(\frac{\partial E}{\partial o_l} \frac{\partial o_l}{\partial \text{net}_l} \frac{\partial \text{net}_l}{\partial o_j} \right) = \sum_{l \in L} \left(\frac{\partial E}{\partial o_l} \frac{\partial o_l}{\partial \text{net}_l} w_{jl} \right)$$

```
23 rand1 = np.random.randint(0,m,1)
24 h = np.dot(theta,samples[rand1].T)
25 #关键点，只需要一个样本点来更新权值
26 for i in range(len(theta)):
27     theta[i] =theta[i] - step_size*(1/m)*(h - y[rand1])*samples[rand1,i]
28 #计算总体的损失精度，等于各个样本损失精度之和
29 for i in range(m):
30     h = np.dot(theta.T, samples[i])
31     #每组样本点损失的精度
32     every_loss = (1/(var*m))*np.power((h - y[i]), 2)
33     loss = loss + every_loss
34
35 print("iter_count: ", iter_count, "the loss:", loss)
36
```

算法理论复杂

代码实现工作量大

-



110



编程框架

- 将深度学习算法中的基本操作封装成一系列组件，帮助研究人员更简单的实现已有算法，或设计新的算法。这一系列深度学习组件，即构成一套深度学习框架

▶ 以TensorFlow为例

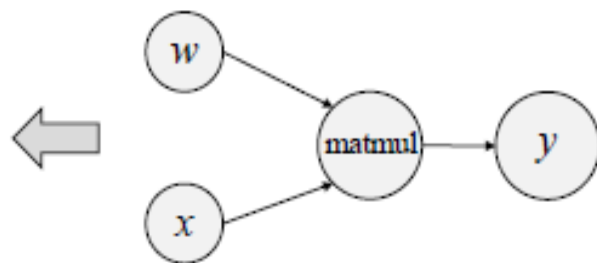
- ▶ 1、构建数据流图，描述计算过程
- ▶ 2、执行数据流图，获得计算结果

```
import tensorflow as tf

a = tf.constant([[3., 3.]])
b = tf.constant([[2.], [2.]])

y = tf.matmul(a, b)

with tf.Session() as sess:
    result = sess.run(y)
    print(result)
```



PYTORCH

caffe  Caffe2



dmlc
mxnet



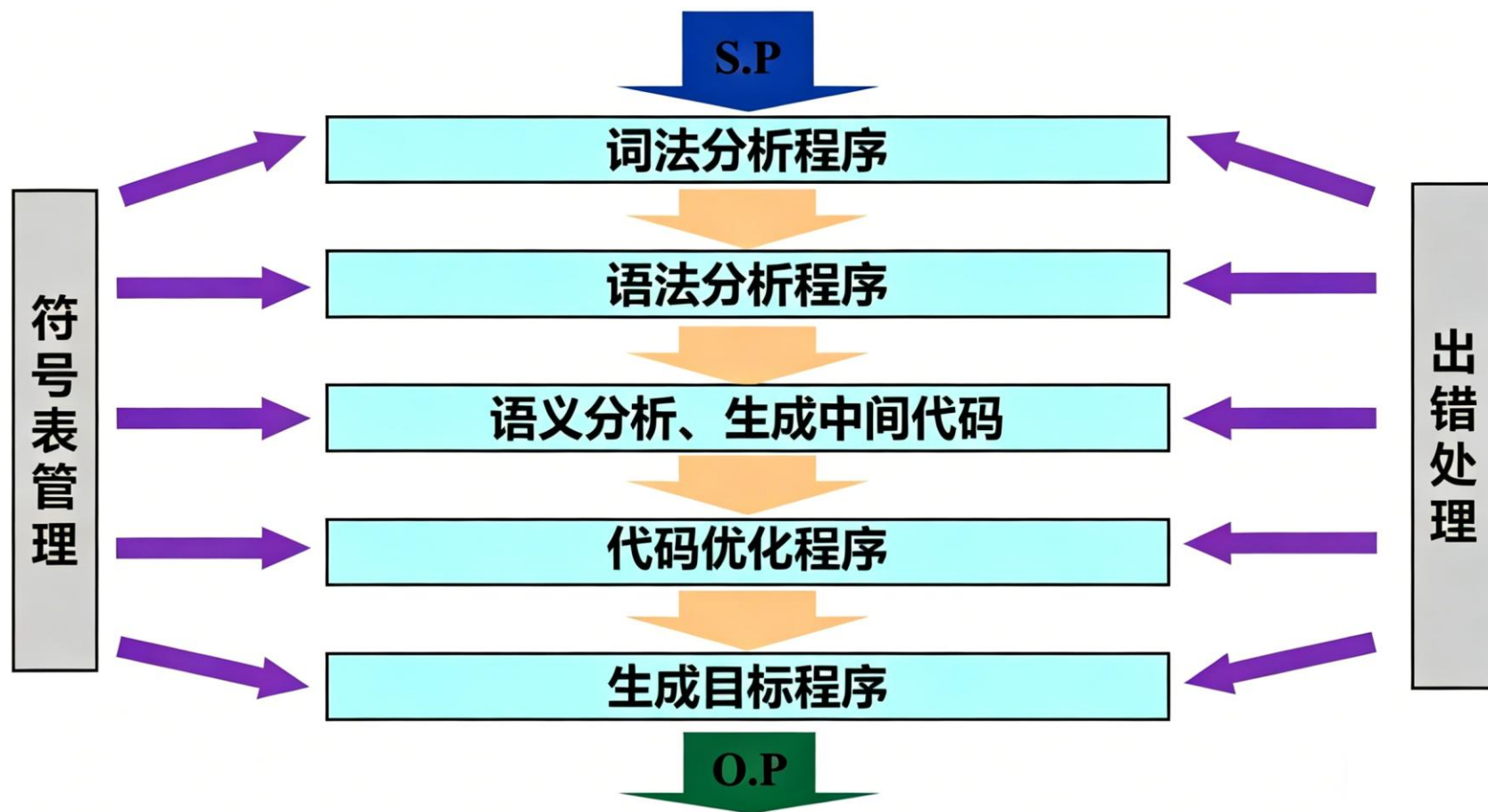
AI系统技术栈

应用算法 (Algorithm)	领域算法	视觉、自然语言处理、语音语义：CNN、RNN、LSTM、GAN、Transformer、Stable Diffusion、Reinforce Learning			
开发编程 (Development)	集成开发环境	编程环境：VS Code, Jupyter Notebook, PyCharm			
	编程语言	与主流语言集成：PyTorch and Tensorflow etc. inside Python			
框架 (Frameworks)	AI框架、推理引擎	中间表达（IR）、前端 API	编译优化（Compilation）	图优化与图执行调度	推理优化
		基本数据结构：张量（Tensor）	词法、语法、语义分析	计算图动静统一	常量折叠&冗余节点消除等
		基本计算单元：有向无环图（DAG）	自动微分	计算图控制流	模型压缩：蒸馏剪枝与量化
异构与编译 (Compiler)	编程编译	前端优化	数据格式布局转换	内存分配	公共表达式消除
		后端优化	代码优化、代码生成	算子循环优化	令和内存优化
		编程模型：CUDA/Ascend C/Band C		AI编译器：TVM、XLA、TC	
		LLVM/GCC			
	底层通信	HCCL/NCCL、通信原语、集合通信算法			
体系结构 (Architecture)	底层硬件	I芯片：CPU/GPU/AISC/FPGA/TPU/NPU			



编译流程

典型的编译程序具有7个逻辑部分





编译器与解释器

Interpreter 解释器

Compiler 编译器

1、创建代码

没有文件链接或机器代码生成

2、源语句在执行过程中逐行执行

1、创建代码

2、将解析或分析所有语言语句的正确性

3、将把源代码转换为机器码

4、链接到可运行程序

5、运行程序

Input

每次读取一行

整个程序

Output

不产生任何的中间代码

生成中间目标代码

工作机制

编译和执行同时进行

编译在执行之前完成

存储

不保存任何机器代码

存储编译后的机器代码在机器上

执行

程序执行是解释过程的一部分，因此是逐行执行的

程序执行与编译是分开的，它只在整个输出程序编译后执行

生成程序

不生成输出程序，所以他们在每次执行过程中都要评估源程序

生成可以独立于原始程序运行的输出程序(以exe的形式)

修改

直接修改就可运行

需要修改源代码，重新编译

运行速度

慢

快

内存

较少内存，不创建中间对象代码

内存需求更多的是由于目标代码的创建

错误

解释器读取一条语句并显示错误。你必须纠正错误才能解释下一行

编译器在编译时显示所有错误和警告。因此，不修正错误就不能运行程序

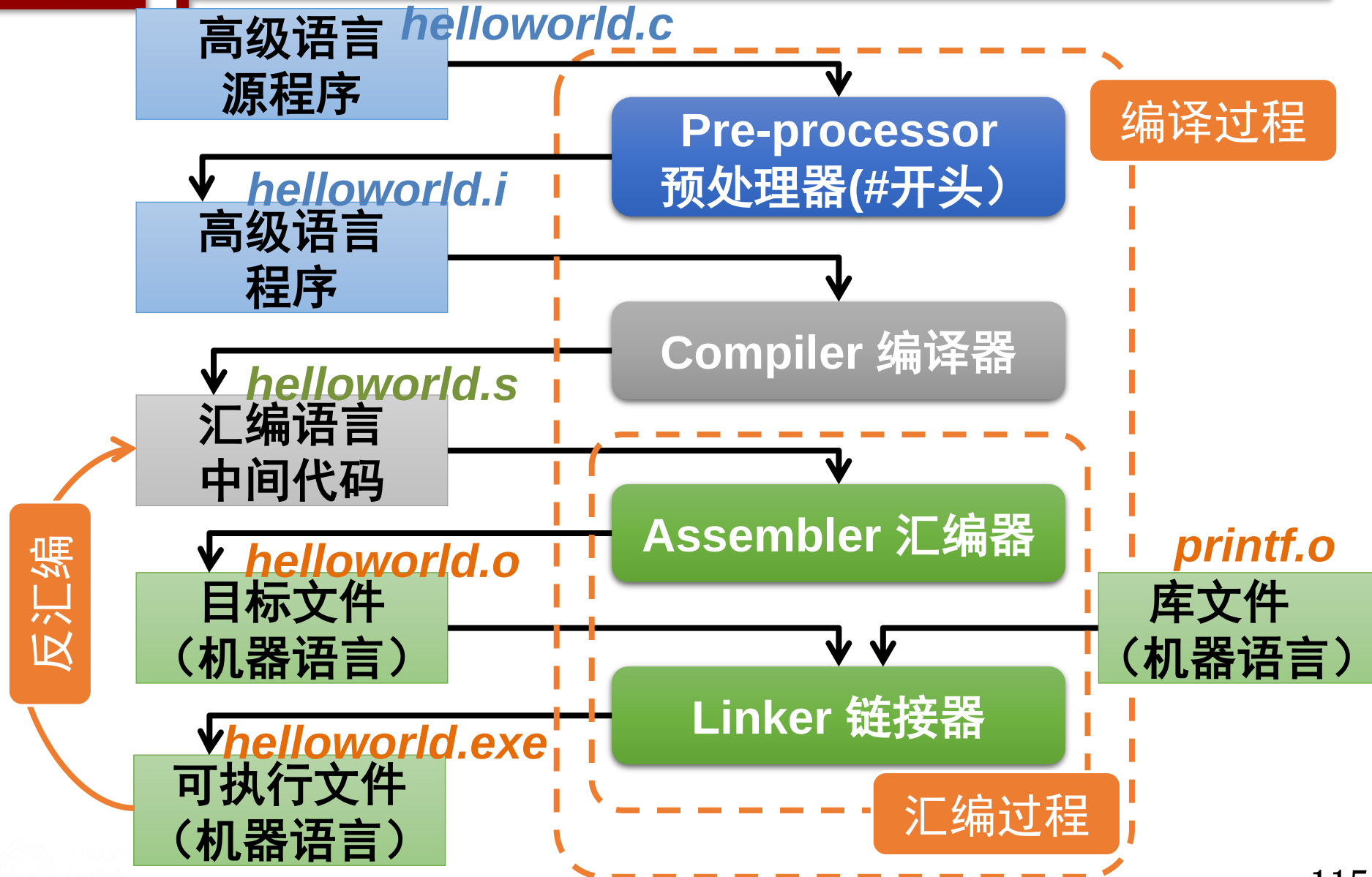
编程语言

PHP, Perl, Python, Ruby

C, C++, C#, Scala, Java

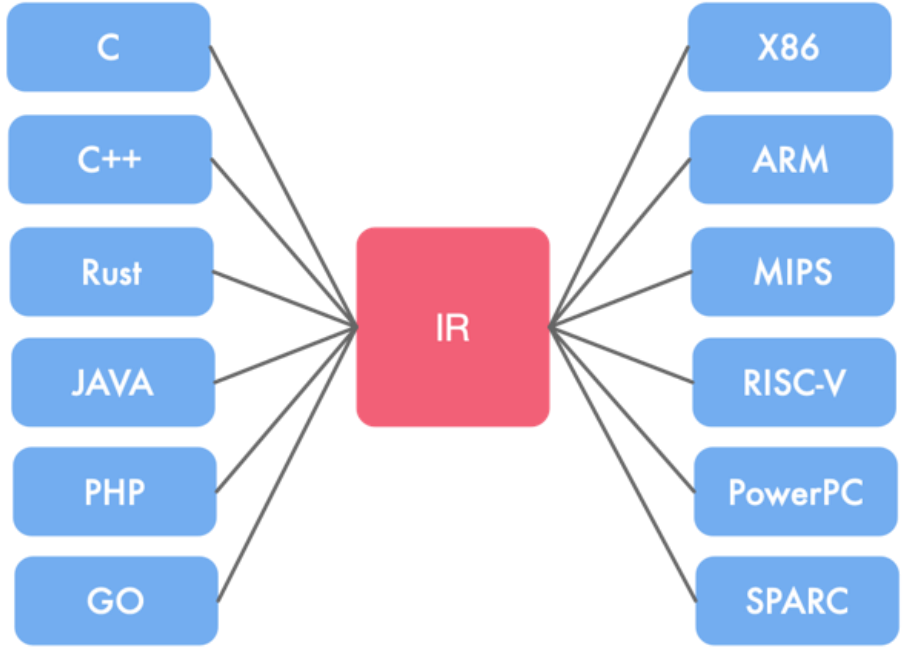
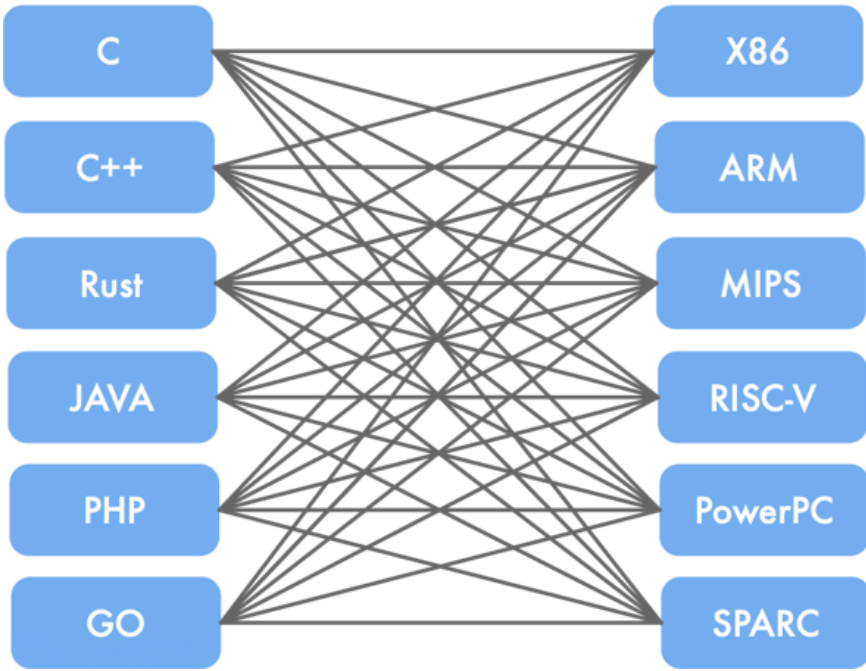


从高级语言源代码到可执行程序文件



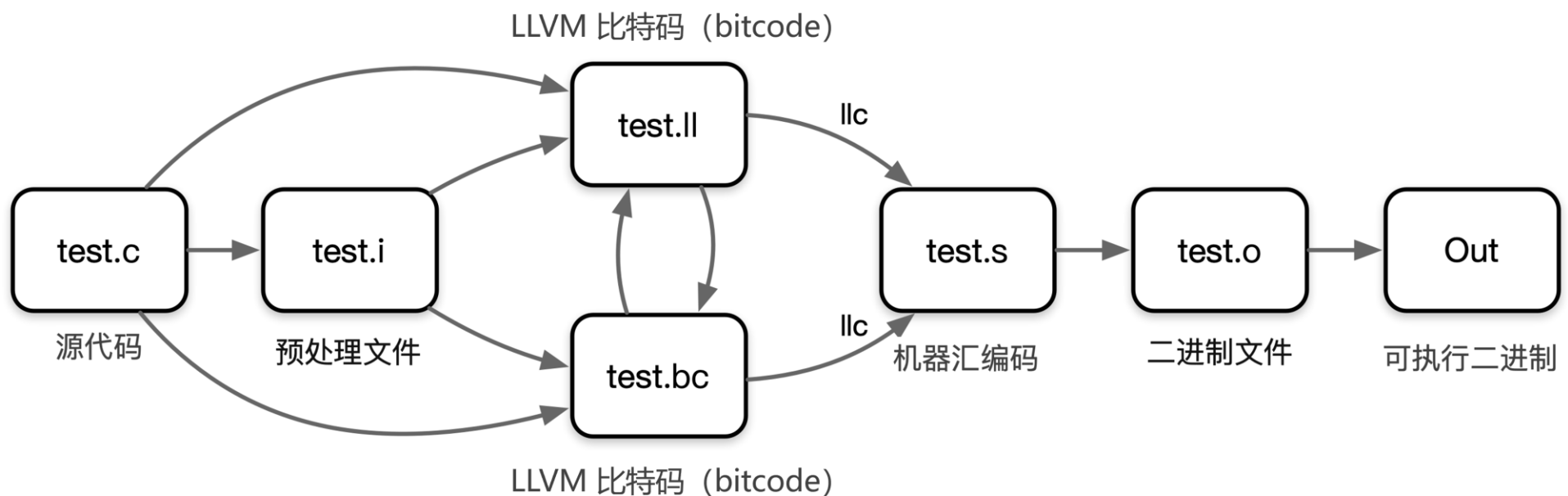
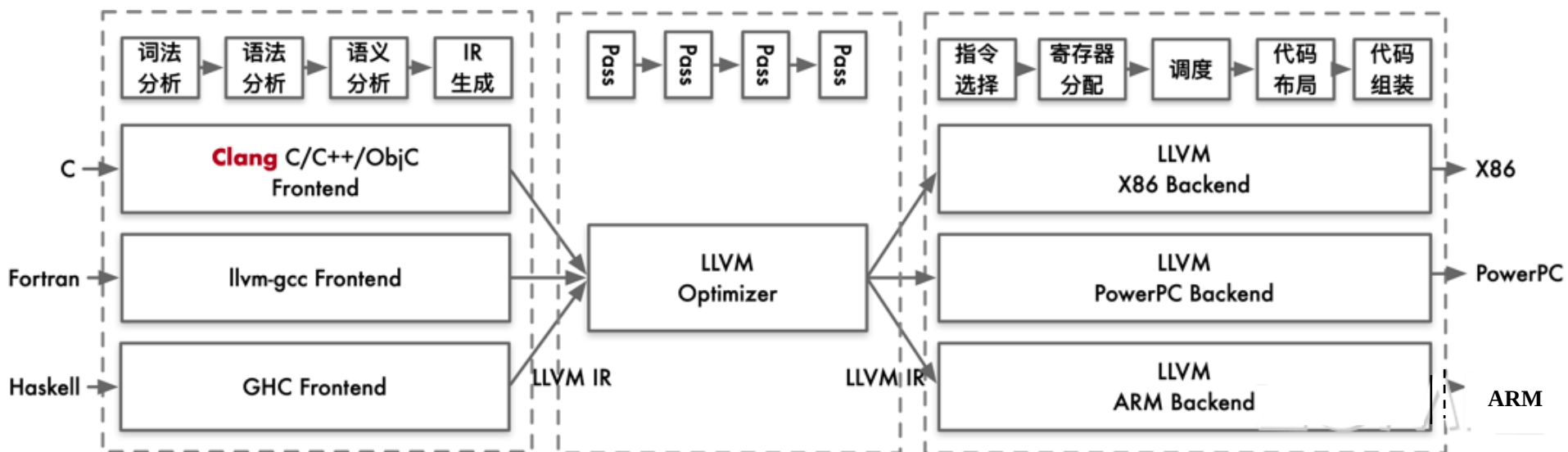


GCC vs LLVM





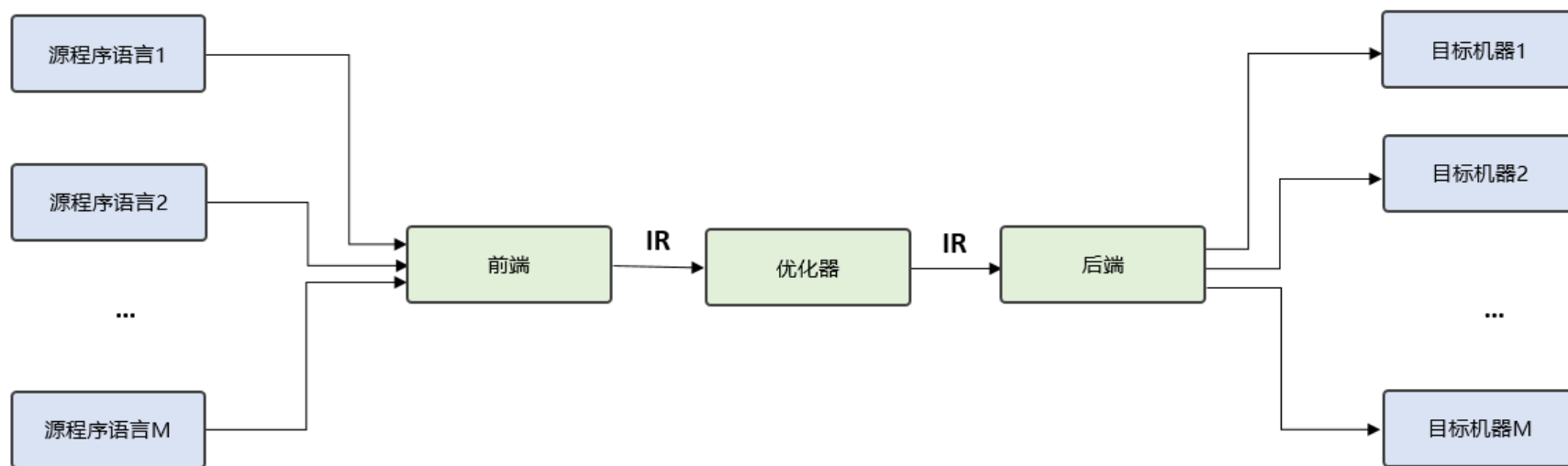
LLVM 整体架构





中间表示IR

- 中间表示(IR)，是编译器用于表示源代码的数据结构或代码，是程序编译过程中介于源语言和目标语言之间的程序表示
- 中间表示既能面向多个前端，表达多种源程序语言，又能对接多个后端，连接不同目标机器
- 一种平台无关的低级编程语言，用于连接前端和后端

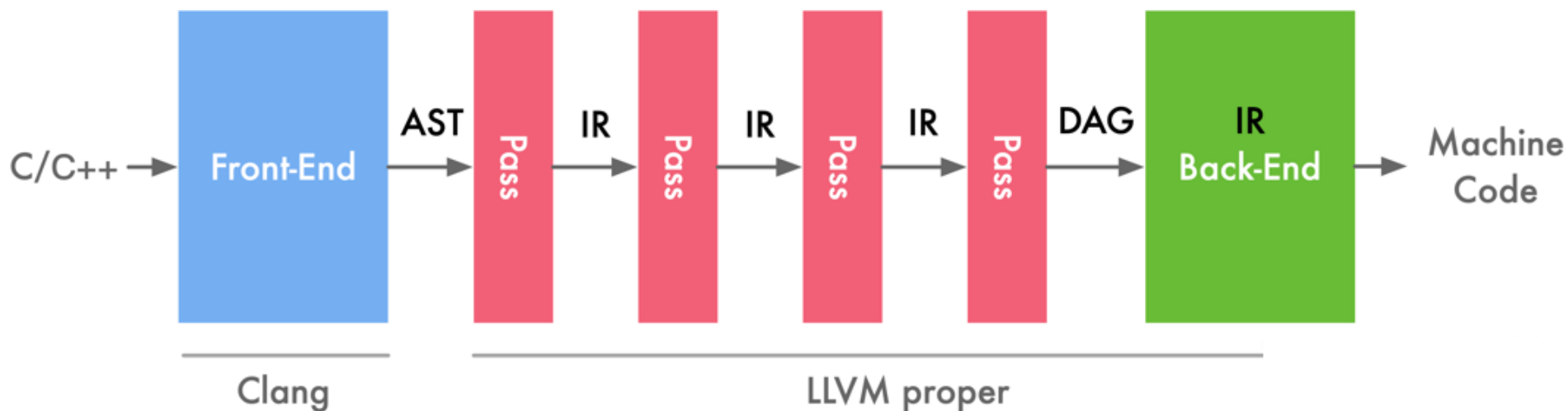




LLVM中间优化层

- Pass(遍)

- LLVM支持全生命周期包括编译时（compile-time）、链接时（link-time）、运行时（run-time）及离线（offline）优化
- 优化器（opt）是一个模块化的LLVM IR分析器和优化器，包含一系列编译“遍”，对LLVM IR程序进行分析和变换
- 遍管理器（PassManager）负责组织遍的执行顺序，用户也可以通过编译选项控制遍的执行





优化Pass

- 目标无关优化，每个 pass 执行的修改，理解 IR 如何在 pass 流水线中被修改，各个 pass 是以什么顺序被执行。
- 优化通常由分析 Pass 和转换 Pass 组成。
 - 分析 Pass: 负责发掘性质和优化机会;
 - 转换 Pass: 生成必需的数据结构，后续为后者所用
- Pass 举例
 - -adce: Aggressive Dead Code Elimination 死代码消除。
 - -constmerge: Merge Duplicate Global Constants 将重复的全局常量合并到一个共享的常量中。

```
-----
... Pass execution timing report ...
-----
Total Execution Time: 0.0003 seconds (0.0003 wall clock)

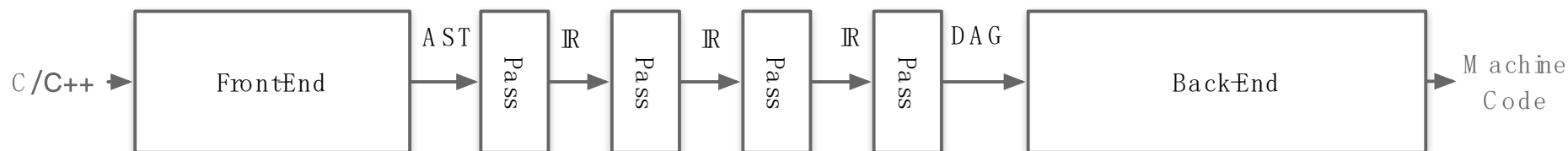
---User Time--- --System Time-- --User+System-- --Wall Time--- ---Instr--- --- Name ---
27 0.0001 ( 72.7%) 0.0000 ( 68.2%) 0.0001 ( 71.6%) 0.0002 ( 70.1%) 481094 BitcodeWriterPass
   0.0000 ( 11.7%) 0.0000 ( 13.6%) 0.0000 ( 12.2%) 0.0000 ( 12.7%) 127722 VerifierPass
   0.0000 ( 5.4%) 0.0000 ( 12.1%) 0.0000 ( 7.0%) 0.0000 ( 7.3%) 78056 RequireAnalysisPass-llvm::DominatorTreeAnalysis, llvm::Function>
   0.0000 ( 7.8%) 0.0000 ( 3.0%) 0.0000 ( 6.6%) 0.0000 ( 6.8%) 56681 VerifierAnalysis
   0.0000 ( 2.4%) 0.0000 ( 3.0%) 0.0000 ( 2.6%) 0.0000 ( 3.2%) 33040 DominatorTreeAnalysis
   0.0002 (100.0%) 0.0001 (100.0%) 0.0003 (100.0%) 0.0003 (100.0%) 776593 Total
28 -----

LLVM IR Parsing
-----
Total Execution Time: 0.0003 seconds (0.0003 wall clock)
$ opt -hello.bc -instcount -time-passes -domtree -g hello-temp.bc -stats
29 ---User Time--- --System Time-- --User+System-- --Wall Time--- ---Instr--- --- Name ---
   0.0003 (100.0%) 0.0001 (100.0%) 0.0003 (100.0%) 0.0003 (100.0%) 869304 Parse IR
   0.0003 (100.0%) 0.0001 (100.0%) 0.0003 (100.0%) 0.0003 (100.0%) 869304 Total
```



LLVM后端

- 后端由一套分析和转换 **Pass** 组成，它们的任务是代码生成，即将**LLVM IR**变换为目标代码（或者汇编）



```
#include <stdio.h>

int loop_add(int* data, int num) {
    int res = 0;
    for (int i = 0; i < num; i++) {
        res += data[i];
    }
    return res;
}
```

```
define dso_local i32 @loop_add(i32*, i32) #0 {
    %3 = alloca i32*, align 8
    %4 = alloca i32, align 4
    %5 = alloca i32, align 4
    %6 = alloca i32, align 4
    store i32* %0, i32** %3, align 8
    store i32 %1, i32* %4, align 4
    store i32 0, i32* %5, align 4
    store i32 0, i32* %6, align 4
    br label %7

; preds = %19, %2
; <label>: 7:
%8 = load i32, i32* %6, align 4
%9 = load i32, i32* %4, align 4
%10 = icmp slt i32 %8, %9
br i1 %10, label %11, label %22

; preds = %7
; <label>: 11:
%12 = load i32*, i32** %3, align 8
%13 = load i32, i32* %6, align 4
%14 = sext i32 %13 to i64
%15 = getelementptr inbounds i32, i32* %12, i64 %14
%16 = load i32, i32* %15, align 4
%17 = load i32, i32* %5, align 4
%18 = add nsw i32 %17, %16
store i32 %18, i32* %5, align 4
br label %19

; preds = %11
; <label>: 19:
%20 = load i32, i32* %6, align 4
%21 = add nsw i32 %20, 1
store i32 %21, i32* %6, align 4
br label %7

; preds = %7
; <label>: 22:
%23 = load i32, i32* %5, align 4
ret i32 %23
}
```





为什么需要AI编译

- 编程框架中早期优化方式存在的问题

- ▶ 框架维护成本高：对于新硬件和新算子，都需要程序员手动进行算子开发，开发数量呈平方级增长
- ▶ 性能受限：性能受限于程序员人工优化算子的能力，且没有充分探索计算图的优化空间

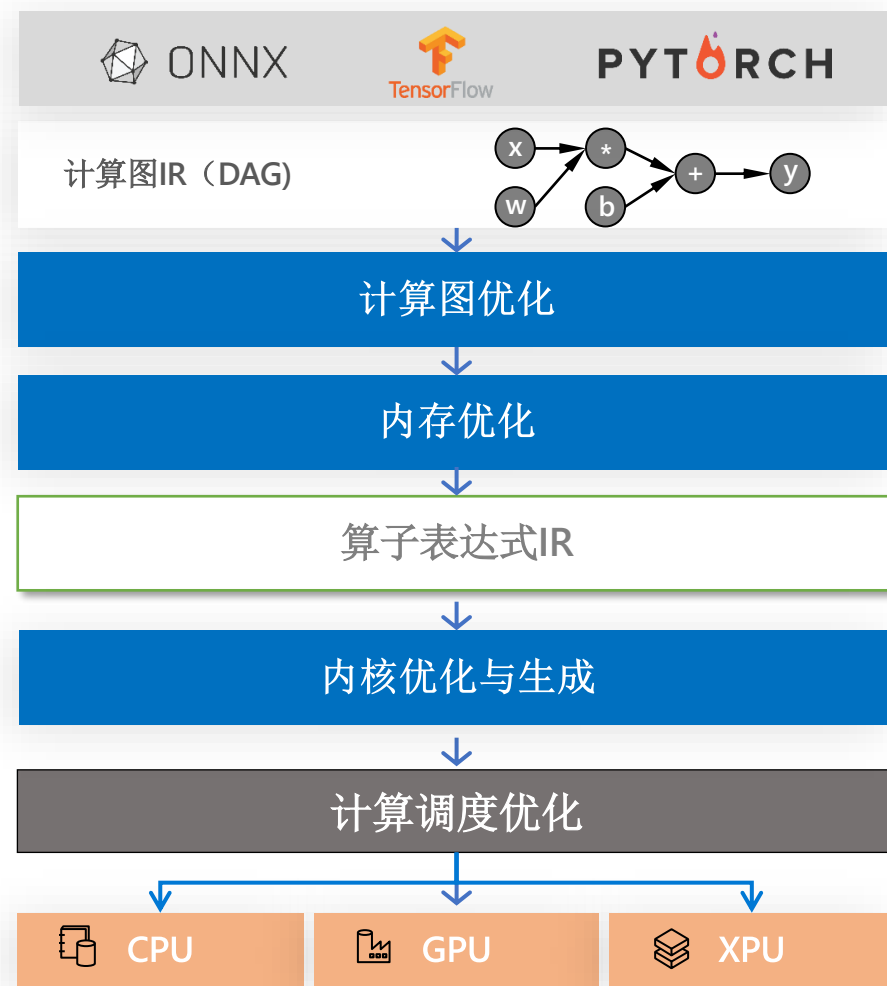
- ▶ 在编程框架中引入AI编译机制

- ▶ 减少人工开发工作量：可针对不同硬件平台进行代码生成
- ▶ 便于性能优化：（图层级）对完整的计算图进行静态分析和全局优化；（算子层级）利用自动调优技术优化算子，最大限度提升硬件利用率



AI编译器

- 前端：基于Python的DSL语言
 - 如PyTorch, TensorFlow等
- 后端：AI加速器
 - 如GPU, CPU, FPGA, TPU, IPU等
- 中间表达：计算图、算子表达式
 - 如DAG, TVM IR
- 优化过程：
 - 计算图优化、内存优化、内核优化、调度优化





内核优化与内核生成



CNTK



问题: 每个后端平台针对每个算子都需要单独实现至少一个内核
考虑: 编程模型、数据排布、线程模型、缓存大小等等





自动化算子优化

- 算子迭代更新快
- 如何应对AI领域算子迭代更新快？如BN-> Layer BN->Group BN
- 如何解决同一算子在多平台移植后一致性问题？
- 如何面对算子组合爆炸问题？如参数多样，融合大算子等

1) AI 编译器中正在尝试融合机器学习方法

Auto Tuning

2) 算子计算加速正在用编译技术来解决

Polyhedral



调度优化

- 循环展开 (Loop Unrolling)
- 循环分块 (loop tiling)
- 循环重排 (loop Reorder)
- 循环融合 (loop Fusion)
- 循环拆分 (loop Split)
- 向量化 (Vector)
- 张量化 (Tensor)
- 访存延迟 (Latency Hiding)
- 存储分配 (Memory Allocation)

循环优化 (Loop Optimization)

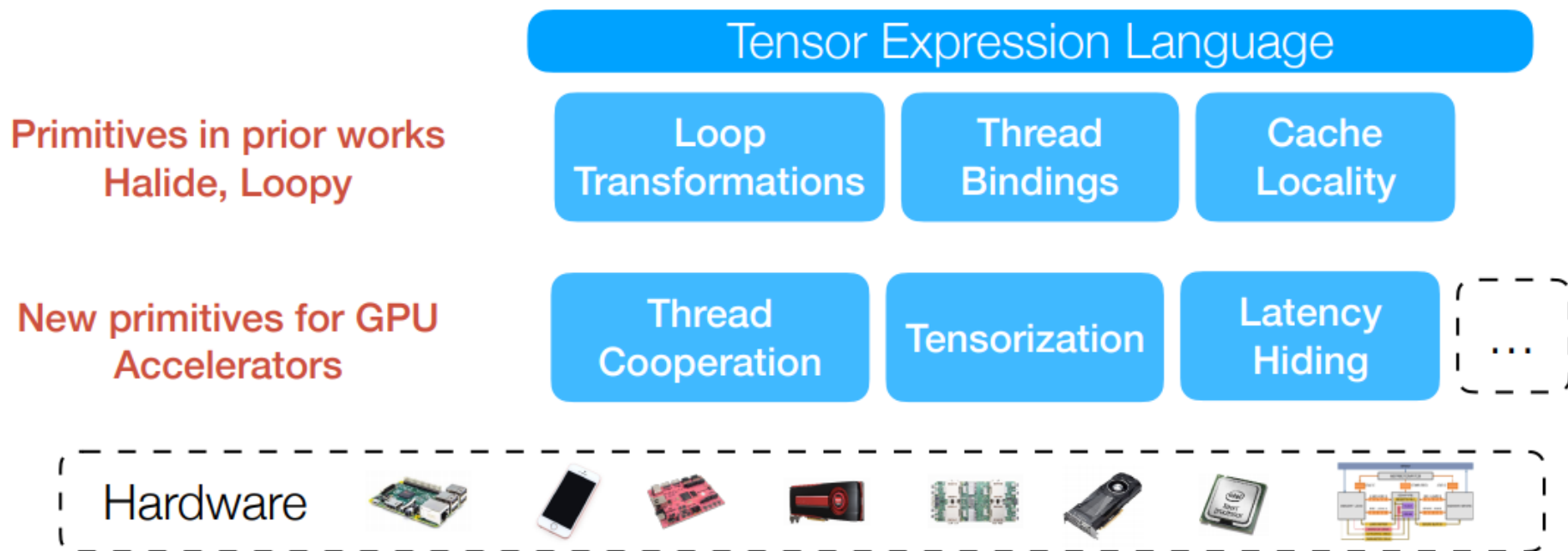
指令优化 (Instr Optimization)

存储优化 (Memory Optimization)



自动调优

- 每一种优化都可能产生出多个内核代码的实现
- 利用自动机器学习（Auto Tuner）在给定时间内搜索出最高效的实现





大模型辅助的国产硬件算子优化

✓大模型辅助的算子优化分析

- 算子调度优化
- 算子内核优化

✓全面拥抱AI，课程没有不及格学生

✓深入认识AI，前10名都有AI没有发现的优化点

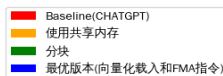
先导杯

#	用户名	Gemm算子	以2为底的Softmax	Attention 部分综合优化	总分
1	2022200648_yox	81 524.4058	95 400.42.43	285 485.29.13	461 524.40.58
1	2022201586_xhshidajun	80 365.39.59	94 529.23.55	287 528.19.42	461 529.23.55
3	2022201482_changpinkai	80 378.22.44	95 420.36.03	285 475.05.05	460 475.05.05
3	2022201383_changfancheng	80 355.07.29	95 424.30.53	285 485.21.52	460 485.21.52
3	2022201566_changshao	80 332.58.23	95 403.00.33	285 485.29.28	460 485.29.28
3	2022201432_yufuwei	80 503.17.24	95 400.53.32	285 486.32.37	460 486.32.37
3	2022201336_changlinkai	80 471.46.01	95 378.36.06	285 487.13.35	460 487.13.35
3	2022201454_humengqi	80 475.22.89	95 509.42.53	285 489.44.43	460 509.42.53
3	2022201466_qihuan	80 521.20.44	94 522.27.32	286 522.58.49	460 522.58.49
3	2022201280_juchihao	80 529.31.31	95 447.56.52	285 509.36.07	460 529.31.31
11	2022201283_YuhangWang	80 476.49.13	95 525.53.26	284 522.93.01	459 525.53.26
12	2022201367_xiaohumuan	80 441.43.15	94 403.48.30	284 515.06.13	458 515.06.13
		79	94	284	457

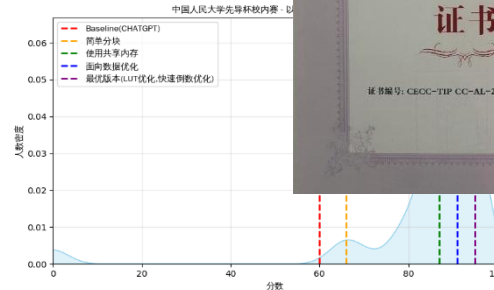
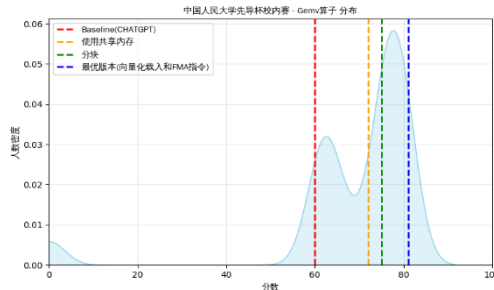
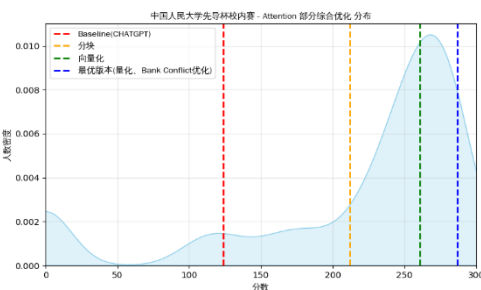
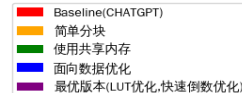
性能优化分析 - Attention 部分综合优化 参赛者优化占比



性能优化分析 - Gemv算子 参赛者优化占比



性能优化分析 - 以2为底的Softmax 参赛者优化占比





赛教融合-校内赛

竞赛启动

- 信息学院、高瓴人工智能学院联合明理书院主办
- 计算机协会、信息技术与管理国家级实验教学示范中心承办
- 教务处副处长、国家人工智能创新平台主任，书院院长
- 院长，教学院长，系主任

竞赛组织

- 搭建校内竞赛评测和排名打榜平台，支持国产加速器的评测
- 搭建校内先导杯网站，包括重要信息的公告，问题讨论论坛和练习题库
- 组织赛题讲解，基本工具培训，竞赛线上线下讨论

竞赛结果

- 104人报名参赛，作为课程结课期末考核，全部完赛
- 一等奖2名（艰难胜出，最后8小时连夜调试）二等奖4人，三等奖6人，优秀奖15人
- 书院院长和曙光领导颁奖



先导杯				
#	校内名	GitHub名字	以2为基数的Attention得分排名	总分
1	2022200648_yan	524.4035	465.4043	485.2913
2	2022201586_zhihaizhan	545.3939	529.2355	539.2355
3	2022201482_zhangyibai	576.2244	495.3635	475.0505
4	2022201383_zhangqiancheng	585.0729	454.3635	485.2352
5	2022201566_zhangyibai	532.5823	465.0033	485.2926
6	2022201413_zhangyibai	605.3724	465.0033	485.2352
7	2022201536_zhangyibai	471.4403	378.3636	487.2355
8	2022201454_humeng	479.2200	509.4233	485.2352
9	2022201466_zhangyibai	521.2044	522.2732	522.0849
10	2022201380_zhangyibai	529.3131	487.3632	509.3637
11	2022201383_zhangyibai	474.4413	525.5926	522.0849
12	2022201367_zhangyibai	441.4313	465.4033	510.0813





开源平台 XPUOJ

刷题

首页 / 题库

搜索筛选

阅读题面

提交代码

查看评测或讨论

比赛

比赛列表

比赛详情

选择题目

提交代码

查看排行或讨论

Agent

打开题目

连接本地 Agent

读取题面信息

生成或修改代码

确认提交或自动提交

```
提交 我的提交 统计
1 // add_half.cu
2 // Build: nvcc -O3 -shared -Xcompiler -fPIC add_half.cu -o libadd_half.so -arch=sm_70
3
4 #include <stdint.h>
5 #include <cuda_runtime.h>
6 #include <cuda_fp16.h>
7
8 // ----- device kernel -----
9 extern "C" __global__ void add_half(float* A, const __half* B, uint64_t numel) {
10     uint64_t idx = (uint64_t)blockIdx.x * (uint64_t)blockDim.x + (uint64_t)threadIdx.x;
11     uint64_t stride = (uint64_t)blockDim.x * (uint64_t)gridDim.x;
12     for (uint64_t i = idx; i < numel; i += stride) {
13         A[i] = __hadd(A[i], B[i]);
14     }
15 }
16
17 // ----- exported host function -----
18 // REQUIRED signature/name:
19 // run_kernel(__half* A, __half* B, uint64_t numel)
20 // Note:
21 // - A/B are expected to be DEVICE pointers (allocated on GPU).
22 // - This launches add_half on the current CUDA device/stream (default stream)
23 // - A/B are expected to be DEVICE pointers (allocated on GPU).
24 // - This launches add_half on the current CUDA device/stream (default stream)
25 extern "C" void run_kernel(__half* A, __half* B, uint64_t numel) {
26     if (A == nullptr || B == nullptr || numel == 0) {
27         return;
28     }
29
30     // Choose launch config
31     const int block = 256;
32     uint64_t grid_u64 = (numel + (uint64_t)block - 1) / (uint64_t)block;
33
34     // Cap grid size to something reasonable (optional)
35     // You can tune this, too large grid isn't always better.
36     const uint64_t grid_cap = 65536ULL; // 10 grid cap for legacy launch dims
37     int grid = (int)(grid_u64 > grid_cap ? grid_cap : grid_u64);
38
39     if (grid < 1)
40         grid = 1;
41
42     // Launch on default stream
43     cudaLaunchKernel((void*)add_half, grid, block, A, B, numel);
44 }
```

人工智能系统--2026春				
我的排名: #78 我的总分: 159.98				
#	选手	1 Sint SGA/MM	2 CrossEntropy	3 Single Profile with KV Cache
1	shuideyimel	126 25.92.37 (3038)	97.25 25.93.20 (349)	98.80 25.94.32 (376)
2	paradise-wuje	123 58.36.17 (13573)	97.38 58.37.25 (5294)	98.80 58.37.25 (6034)
3	Honkai	122.88 276.48.36 (2009)	97.13 288.44.57 (324)	98.80 289.52.39 (3331)
4	bryce	122.88 101.90.42 (646)	97.25 102.40.18 (37)	98.40 103.07.26 (70)
5	manhao	122.63 06.52.26 (4734)	97.13 63.03.46 (390)	98.60 65.03.15 (309)
6	flashly	122.63 10.12.50 (956)	97.13 11.19.43 (89)	98.40 11.42.17 (83)
7	hhh	122.50 37.34.17 (1375)	97.13 37.49.40 (340)	98 38.03.50 (337)
8	Wilber886	121.63 30.07.02 (188)	97 34.55.37 (10)	98.40 34.56.30 (24)
9	PegasusGP	121.25 216.07.49 (83)	97.13 220.52.44 (33)	98.40 222.39.55 (60)
10	1000046	120.63 07.52.29 (167)	97.13 08.06.35 (81)	98 08.21.32 (48)

Agent (MCP)

在本地运行一个中继脚本，并让你本地的 agent（如 Claude Code / Codex）连接到它。本页面作为 MCP 服务器，对外提供当前页面与你的提交信息。

本地中继地址

http://127.0.0.1:7423

配对令牌

.....

提交模式

完全允许

状态

未连接

连接

- (<https://xpuoj.com.cn/>)



FlagOS实验和竞赛

智源专家进校园赋能科创成长



斩获FlagOS全球挑战赛天津站桂冠

FlagOS 24小时算子赏金挑战赛影响力辐射全球，赛事共集结50名顶尖选手，覆盖中国、印尼、埃塞俄比亚、印度、意大利、瑞典6个国家，汇聚巴黎文理研究大学、北京大学、浙江大学、中国科学院大学等14所国内外知名高校。





竞赛和教改获奖

- 全国大学生系统能力大赛
 - 2025教师获得“先导智算”专项项目资助
 - 2025年学生获**全国第一**
 - 2026年3支队进决赛，进入决赛队伍**最多高校之一**
- 2026年FlagOS算子赏金赛**天津站第一，全球第二**
- 第七届中国计算机教育大会计算机类教学和实验案例**一等奖1项，二等奖1项**



2026全国大学生计算机系统能力大赛

智能计算创新设计赛(先导杯)

主办单位：全国高等学校计算机教育研究会、系统能力培养研究专委会、系统能力培养研究联盟、系统能力培养研究联盟

承办单位：清华大学、清华大学产业创新研究院

协办单位：机械工业出版社

支持单位：CQIP、高算平台、CQIP、高算平台、CQIP、高算平台、CQIP、高算平台

2026年全国大学生计算机系统能力大赛-智能计算创新设计赛（先导杯）

参赛队伍 345

基于国产加速卡的OpenAI大模型推理性能提升-初赛

136支参赛队伍

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

初赛得分统计

100%通过初赛

<



提纲

- 01 课程建设背景
- 02 国内外课程情况
- 03 AI技术栈和课程建设思路
- 04 硬件体系结构与 AI 芯片
- 05 AI 编译和算子优化
- 06 并行计算和分布式训练
- 07 课程思政

计算机体系结构

为什么学习计算机体系结构？

1

Core Principles

深入了解CPU、内存和存储机制



2

Performance Optimization

学会编写高效、高性能的软件



3

Modern Microarchitecture

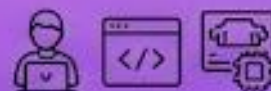
理解流水线、缓存和多核心设计



4

Low-Level Insight

弥合软件与硬件的差距



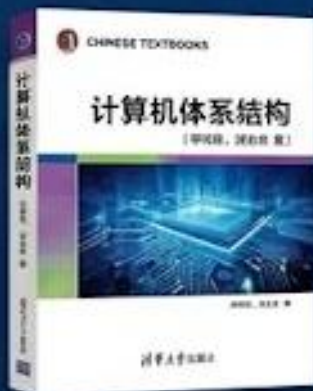
5

Future Tech Foundations

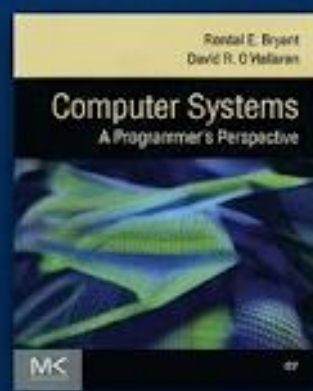
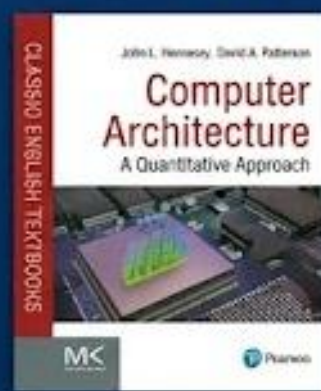
为AI、云计算、嵌入式系统打下基础



经典中文教材



CLASSIC ENGLISH TEXTBOOKS





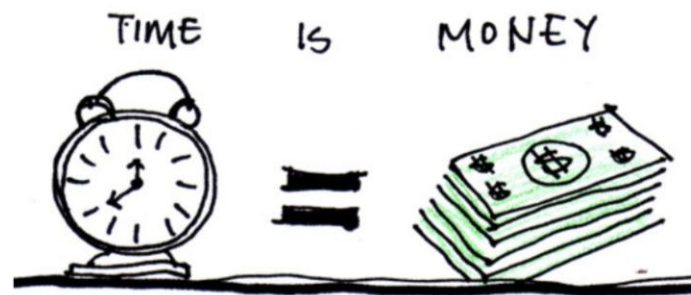
为什么需要并行计算

- 节省时间
- 使用更多的资源来减少运行时间，进而节约成本
- 更短的运行时间可以执行更多次，调程序更多次

4 hours of work



Finish in 1 hour!!!



	DUAL XEON CPU server	DGX-1 GPU server (8 GPUs)
FLOPS	3TF	170TF
Node Mem BW	76GB/s	768GB/s
Alexnet Train Time	150 Hr	2Hr
Train in 2Hr	>250Nodes	1Node



大模型需要更长的训练时间



老板：你上个月做了什么？



你：把模型训练了一个 **epoch**。



老板：嗯，好吧，那你下个月的计划是什么？

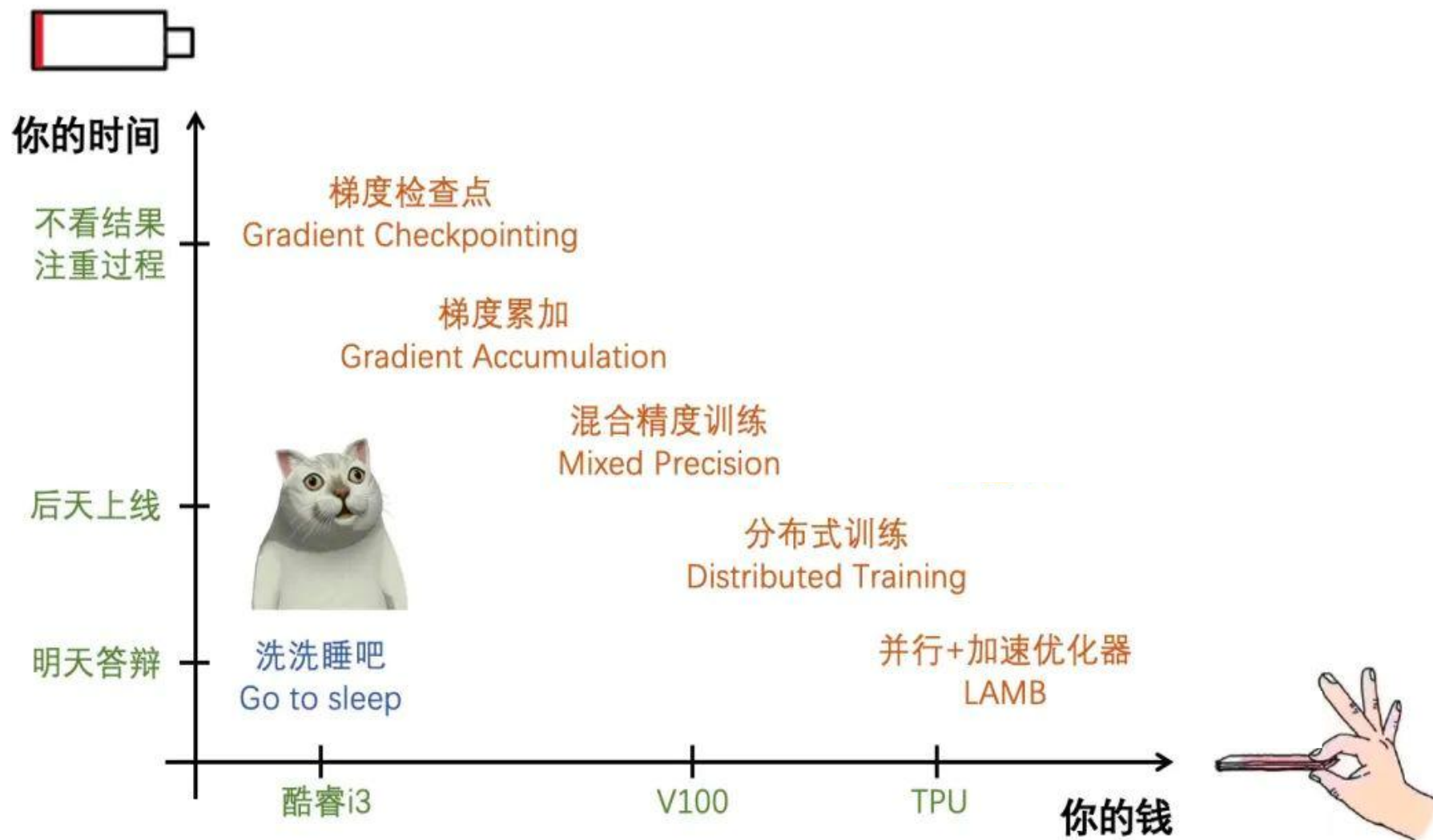


你：再.....再把模型训练一个 **epoch**？





分布式并行





并行技术的发展

单核时代

Enabled by:

摩尔定律
电压缩放

Constraint by:

功耗
复杂性

汇编 → C/C++ → Java ...



多核时代

Enabled by:

摩尔定律
SMP

Constraint by:

功耗
并行软件可
扩展性

Pthread → OpenMP ...



分布式系统时代

Enabled by:

网络技术

Constraint by:

同步
通信开销

MPI → MapReduce ...



异构系统时代

Enabled by:

丰富的数据并行性
高效 GPU

Constraint by:

编程模型
通信开销

着色器 (Shader) → CUDA → OpenCL ...





提纲

➤ 指令级并行

- 记分牌
- Tomasulo 算法
- ROB: 重排序缓冲区

➤ 线程级并行

➤ 数据级并行

- 向量处理器
- GPGPU



相关性与冒险

- 相关性：
 - 反映原始程序顺序（会影响执行结果）
 - 表示可能发生冒险
 - 决定并行度
- 相关性是程序的属性
 - 在设计良好的流水线中不一定导致冒险
- 冒险是硬件组织结构的属性
 - 由指令重排、重叠执行等引起
- 三类相关性：
 - 数据、名称和控制



记分牌示例

指令状态

		<i>j</i>	<i>k</i>
1. LD	F6	34	R2
2. LD	F2	45	R3
3. MUL.D	F0	F2	F4
4. SUB.D	F8	F6	F2
5. DIV.D	F10	F0	F6
6. ADD.D	F6	F8	F2

IS	RO	EX	WR

寄存器结果状态

F0	
F2	
F4	
F6	
F8	
F10	

Function 功能单元 Status

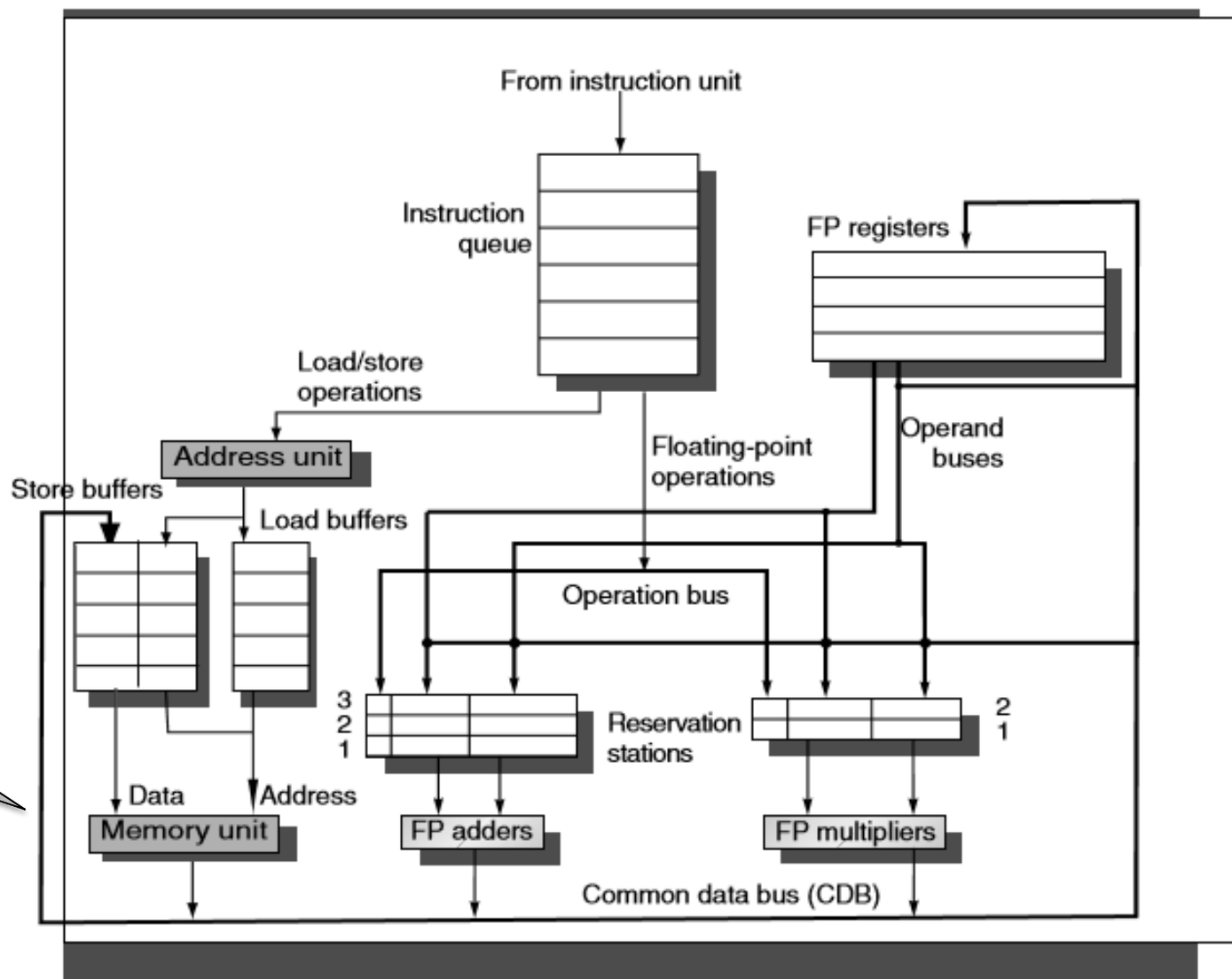
Dest. S1 S2 j 对应 FU k 对应 FU Fj 就绪 Fk 就绪
? ?

Time	Name	Busy	Op	Fi	Fj	Fk	Qj	Qk	Rj	Rk
	Integer	No								
	Mult1	No								
	Mult2	No								
	Add	No								
	Divide	No								

假设：Load（2 周期）、Add（2 周期）、Mult（10 周期）、Div（40 周期）



Tomasulo 算法的基本结构



CDB:

数据 + 源标签
(“来自何处”总线)



Tomasulo 示例

指令状态

		<i>j</i>	<i>k</i>
1. LD	F6	34	R2
2. LD	F2	45	R3
3. MUL.D	F0	F2	F4
4. SUB.D	F8	F6	F2
5. DIV.D	F10	F0	F6
6. ADD.D	F6	F8	F2

IS	EX	WR



寄存器结果

F0	
F2	
F4	
F6	
F8	
F10	

保留站

<i>Time</i>	<i>Name</i>	<i>Busy</i>	<i>Op</i>	<i>Vj</i>	<i>Vk</i>	<i>Qj</i>	<i>Qk</i>
	Add1	No					
	Add2	No					
	Add3	No					
	Mult1	No					
	Mult2	No					

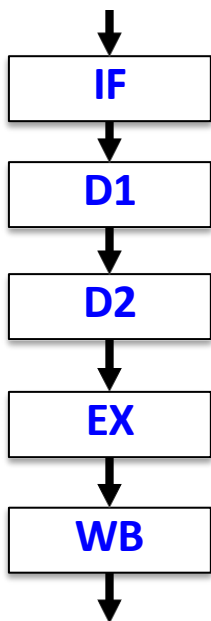
	<i>Busy</i>	<i>Addr.</i>
Load1	No	
Load2	No	
Load3	No	

加载缓冲区

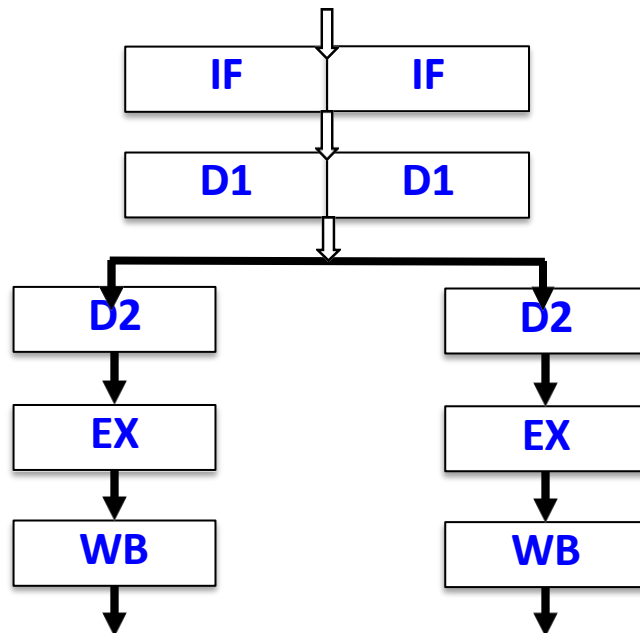
假设: Load (2 周期)、Add (2 周期)、Mult (10 周期)、Div (40 周期)



从标量到超标量



5 级 i486 标量流水线

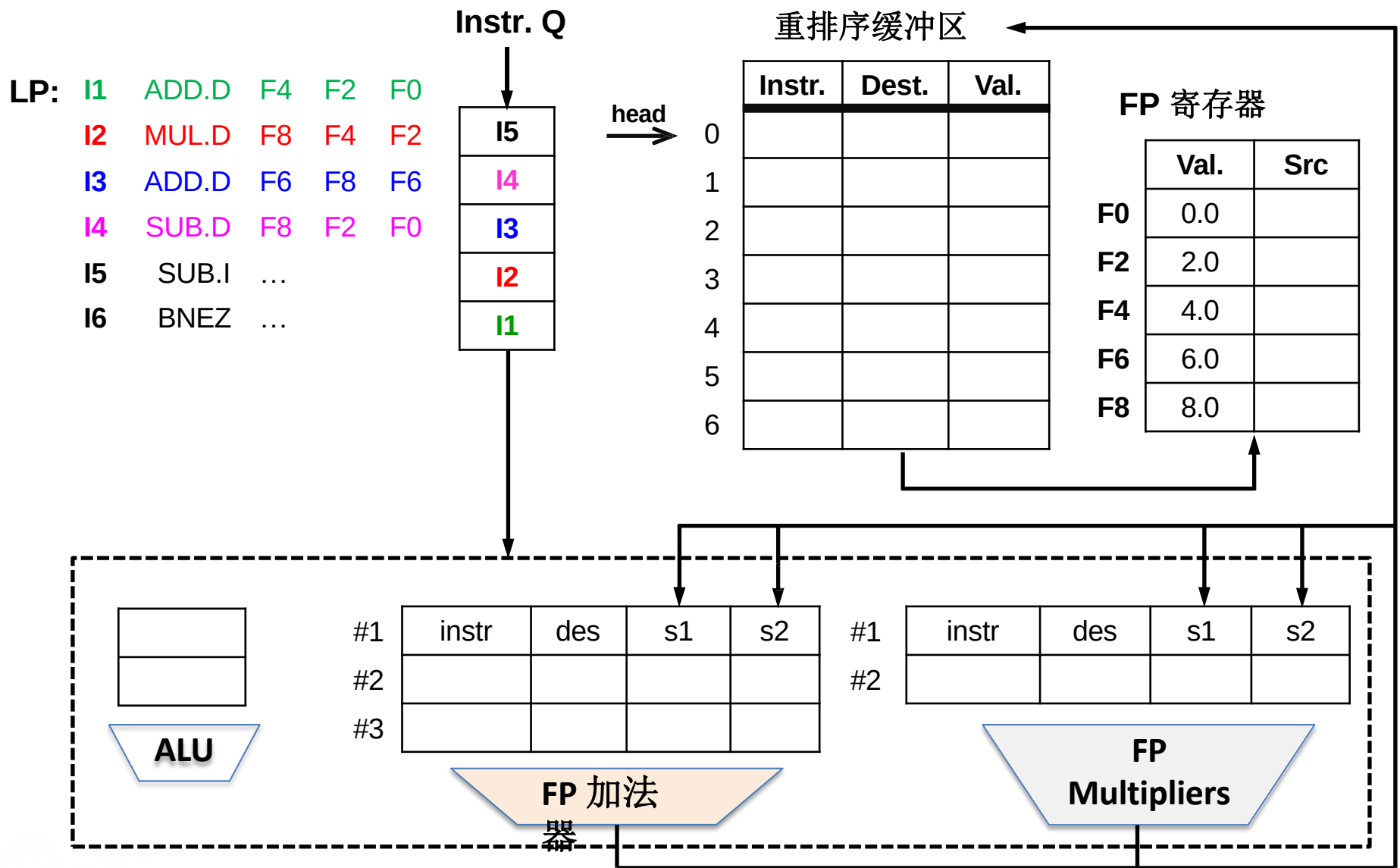


5 级 Pentium 超标量流水线：宽度 = 2

- 性能加速比：
 - 标量：相对于非流水化设计进行衡量
 - 主要由标量流水线深度决定
 - 超标量：相对于标量流水线进行衡量
 - 主要由并行流水线宽度决定



“Tomasulo+ROB” 示例





两类多处理器

集中式共享内存

- 多个处理器共享同一物理内存
- 统一内存访问（**UMA**）

分布式共享内存

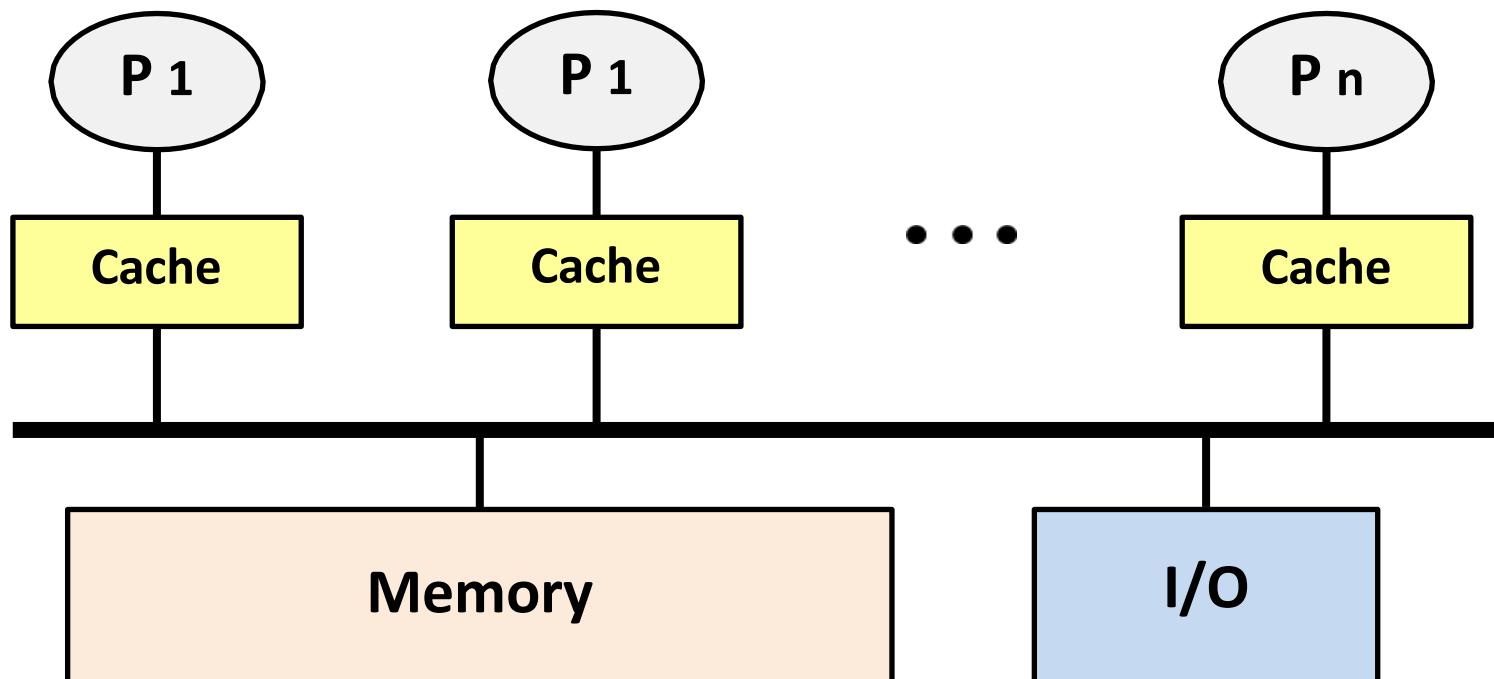
- 每个处理器拥有物理上分离的内存
- 非统一内存访问（**NUMA**）

- 按内存组织方式分类
- 共享内存意味着地址空间是共享的



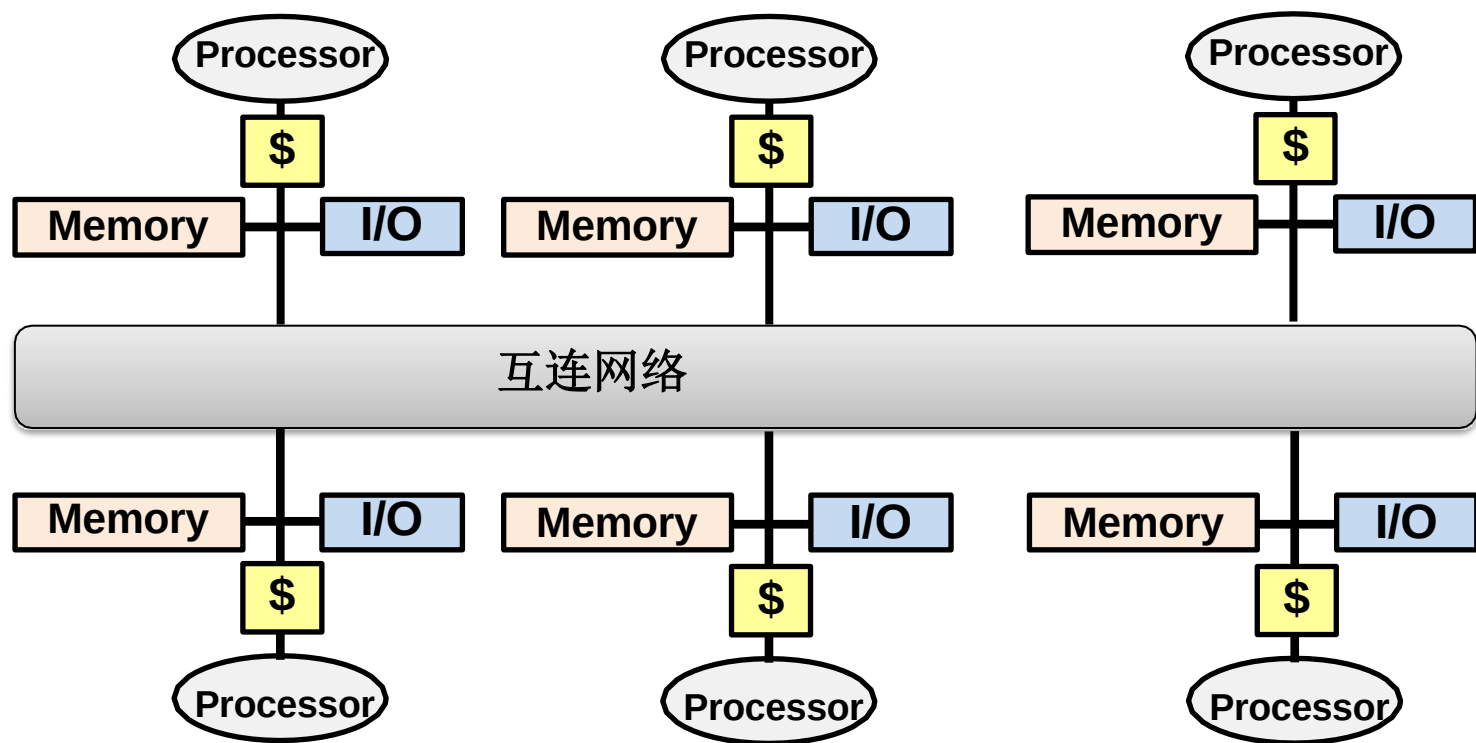
集中式共享内存

- 通常称为对称多处理器（**SMP**）
 - 处理器访问任意内存位置的时间相同





分布式共享内存（DSM）



- 两大优势

- 可扩展的内存带宽
- 低延迟本地访问

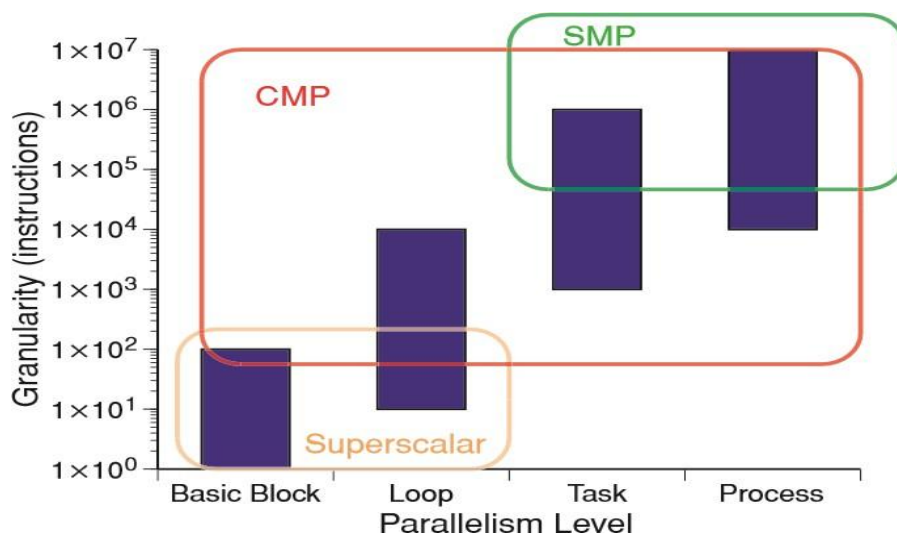
- 主要缺点

- 通信复杂
- 节点间延迟更高



CMP 与 SMT

- **CMP: 片上多处理**
 - 多个物理核心，各自拥有独立资源
 - **L1 缓存、TLB、PC、GPR 独立；L2 缓存可共享**
- **SMT: 同时多线程**
 - 多个线程共享全部处理器资源
 - 缓存和 **TLB 共享，PC 和 GPR 独立**



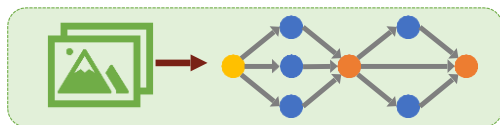
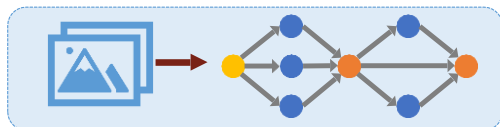


提纲

- 指令级并行
 - 记分牌算法
 - Tomasulo 算法
 - ROB: 重排序缓冲区
- 线程级并行
- 数据级并行
 - 向量处理器
 - GPGPU

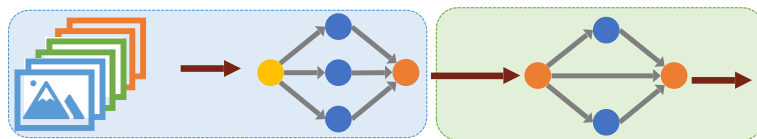


分布式训练



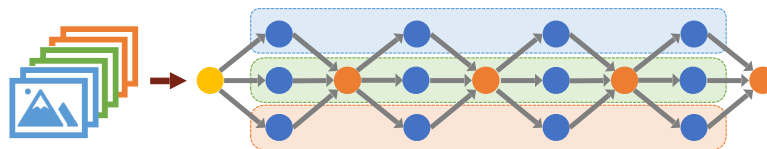
1. 数据并行:

- 划分数据
- 模型N个副本
- 利用率高，内存开销高，通讯开销低
- 典型方法: ZeRO 1/2/3, FSDP
- Infiniband
 - EDR: 100Gb/s
 - HDR: 200Gb/s
 - NDR: 400Gb/s



2. 流水线并行:

- 模型按层划分
- 一份模型
- 利用率低，内存开销低，通讯开销适中



3. 张量并行:

- 按照张量粒度划分模型
- 一份模型
- 利用率高，内存开销低，通信开销高



FlagScale

AI大模型算法

(NLP, CV, speech, multi-modal, etc.)

各种深度学习框架

(PyTorch, Tensorflow, PaddlePaddle, Mindspore, etc.)

**FlagGems/FlagAttention: 大模型
算子库**

**FlagScale:
大模型训练推理并行框架**

基于Triton的统一、增强编译器

FlagCX: 统一通信库

后端编译器
A

底层通信库
A

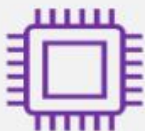
后端编译器
B

底层通信库
B

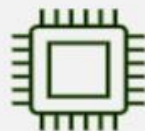
后端编译器
C

底层通信库
C

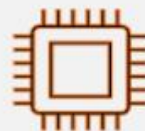
Chip A



Chip B



Chip C



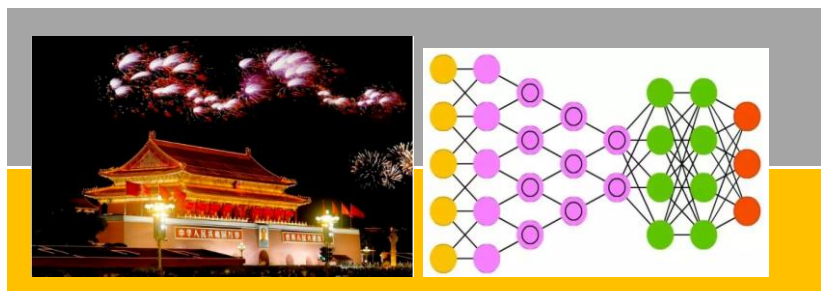
几种任务划分模式

▶ 数据并行

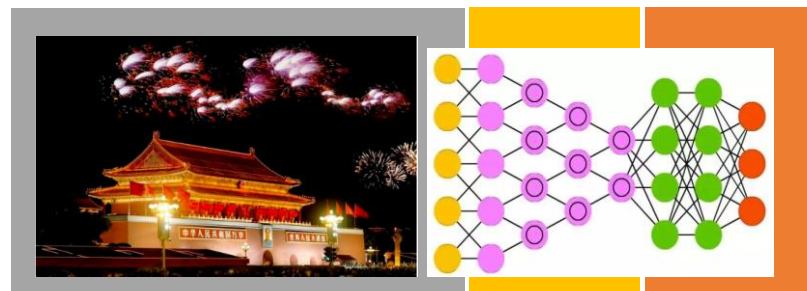


▶ 模型并行

- 算子并行



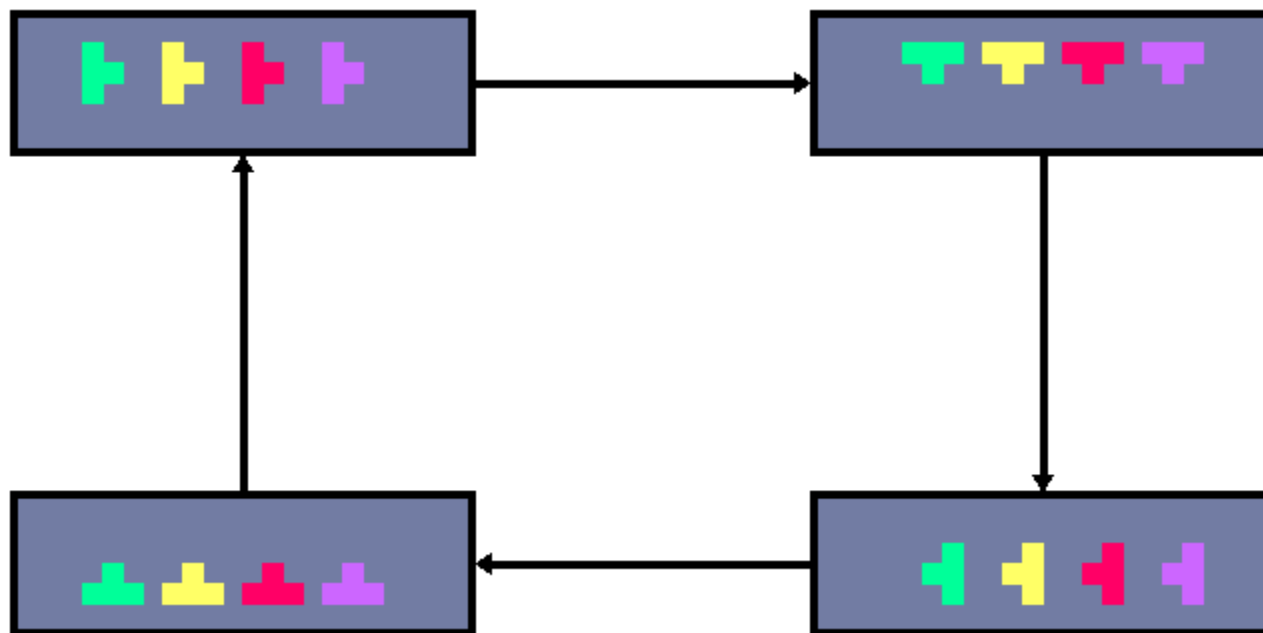
▶ 流水线并行



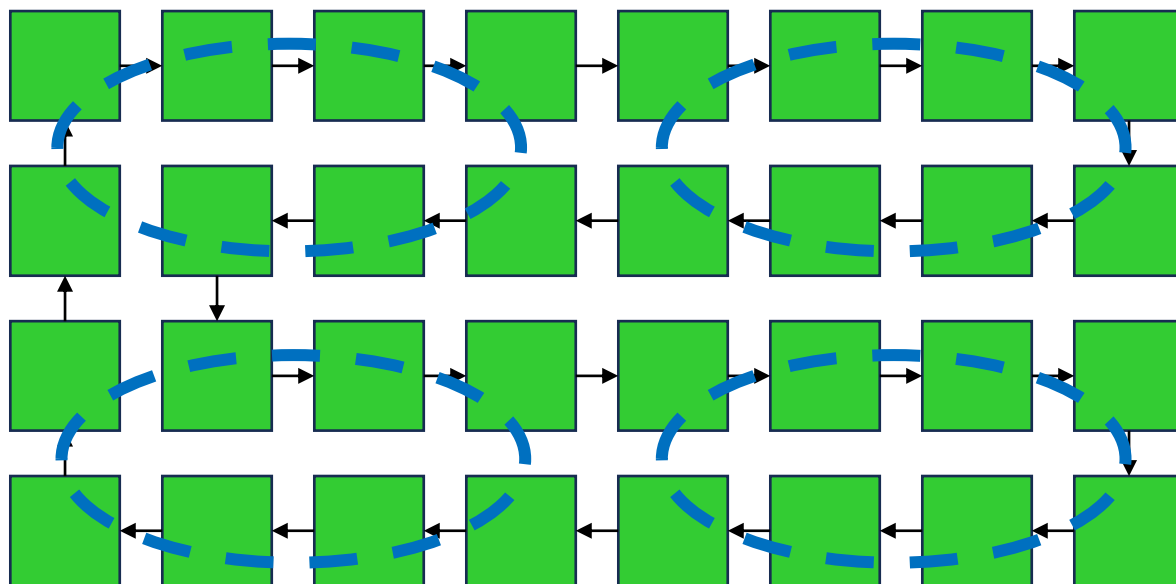
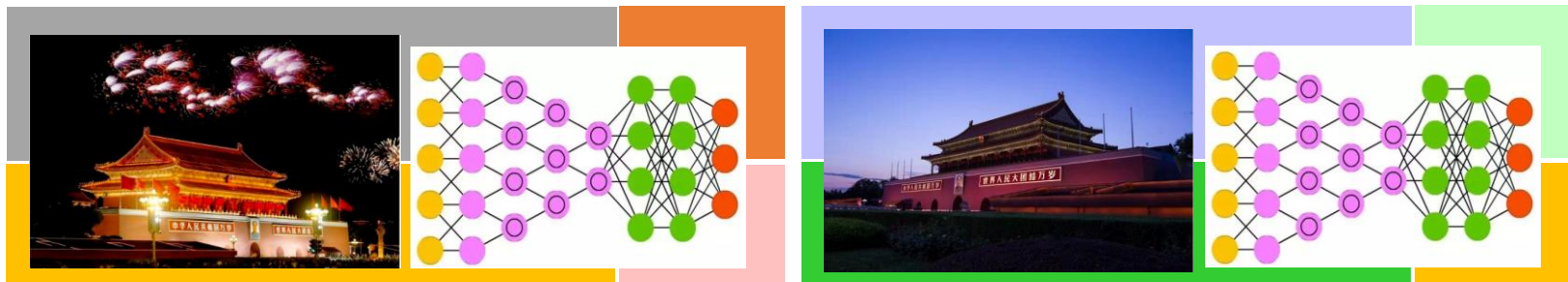


全局归约

通过一个环（ring），就可以高效实现全局归约



混合使用几种任务划分模式

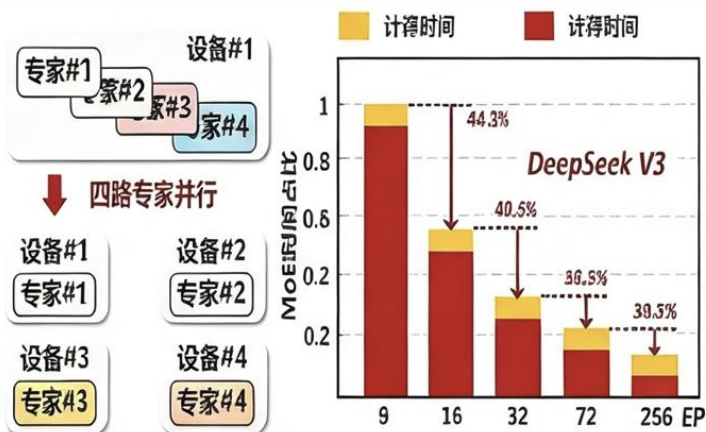




通信墙问题

- AI算力（存储和带宽）跟大模型Scaling law相背离

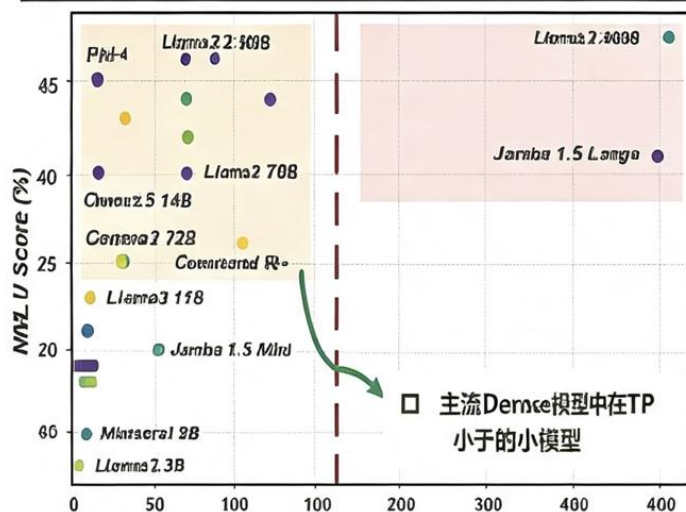
访存/通讯成本限制专家并行数量



集群的跨节点通讯限制专家并行

算力平台的带宽制约了对更大型模型的探索

模型	隐藏层1	隐藏层2	层数	最优P
模型	隐藏层1	隐藏层2	层数	最优TP
Qwen2.5-14B	5120	13824	48	4
Phi4-14.7B	5120	17920	40	4
Llama3.3-70B	8192	28672	80	8
Qwen2-72B	8192	25568	80	8



MoE依赖大规模专家并行减少访存开销占比



提纲

01 课程建设背景

02 国内外课程情况

03 AI技术栈和课程建设思路

04 硬件体系结构与 AI 芯片

05 AI 编译和算子优化

06 并行计算和分布式训练

07 课程思政



课程思政

国产 AI 大比拼 你最爱用哪个

Logo	名称	所属公司	上线时间
豆包	豆包	字节跳动	2023年6月
DeepSeek	DeepSeek	深度求索	2025年1月
元宝	元宝	腾讯集团	2024年5月
千问	千问	阿里巴巴	2023年9月
文心	文心	百度集团	2023年3月



国内AI产品排行榜 2025年5月

排名	Logo	名称	功能分类
1	DeepSeek	Deepseek	复杂对话
2	豆包	豆包	对话、绘图智能体
3	文心一言	文心一言	对话、办公
4	ima	ima	知识库
5	通义	通义	会议纪要、办公
6	扣子Coze	扣子Coze	智能体
7	听脑AI	听脑AI	录音助手、语音转文字
8	秘塔AI	秘塔AI	学术搜索

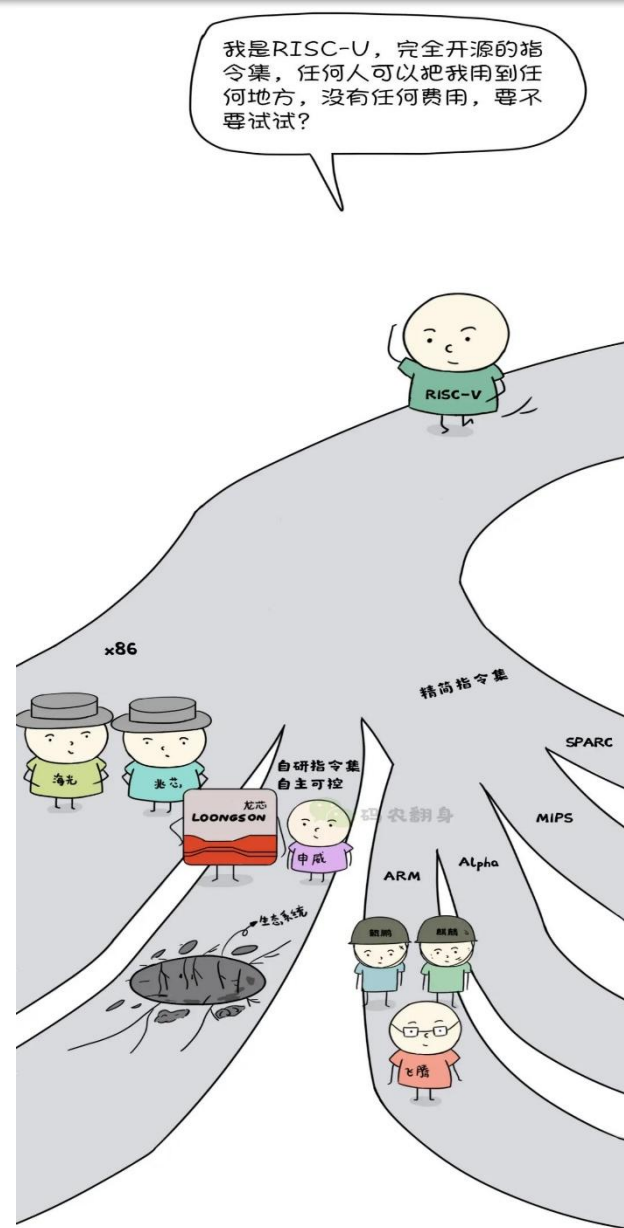
AI应用生态图

AI对话	AI办公	AI设计	AI视频	AI音频	AI编程
AI对话: ChatGPT, DeepSeek, 豆包, Kimi, 通义千问	AI办公: PicDoc, 博思AIPPT, boardmix, 飞书, 扣子空间	AI设计: Midjourney, 创客贴, 通义万相, Pixso, Nano banana	AI视频: 可灵AI, Vidu, Sora, 万兴天幕, 海螺AI	AI音频: Udio, DubbingX, Minimax, 海绵音乐, Stable Audio	AI编程: Cursor, Trae, Gemini, Junie, Codeium





中国芯





AI芯片产业全景图

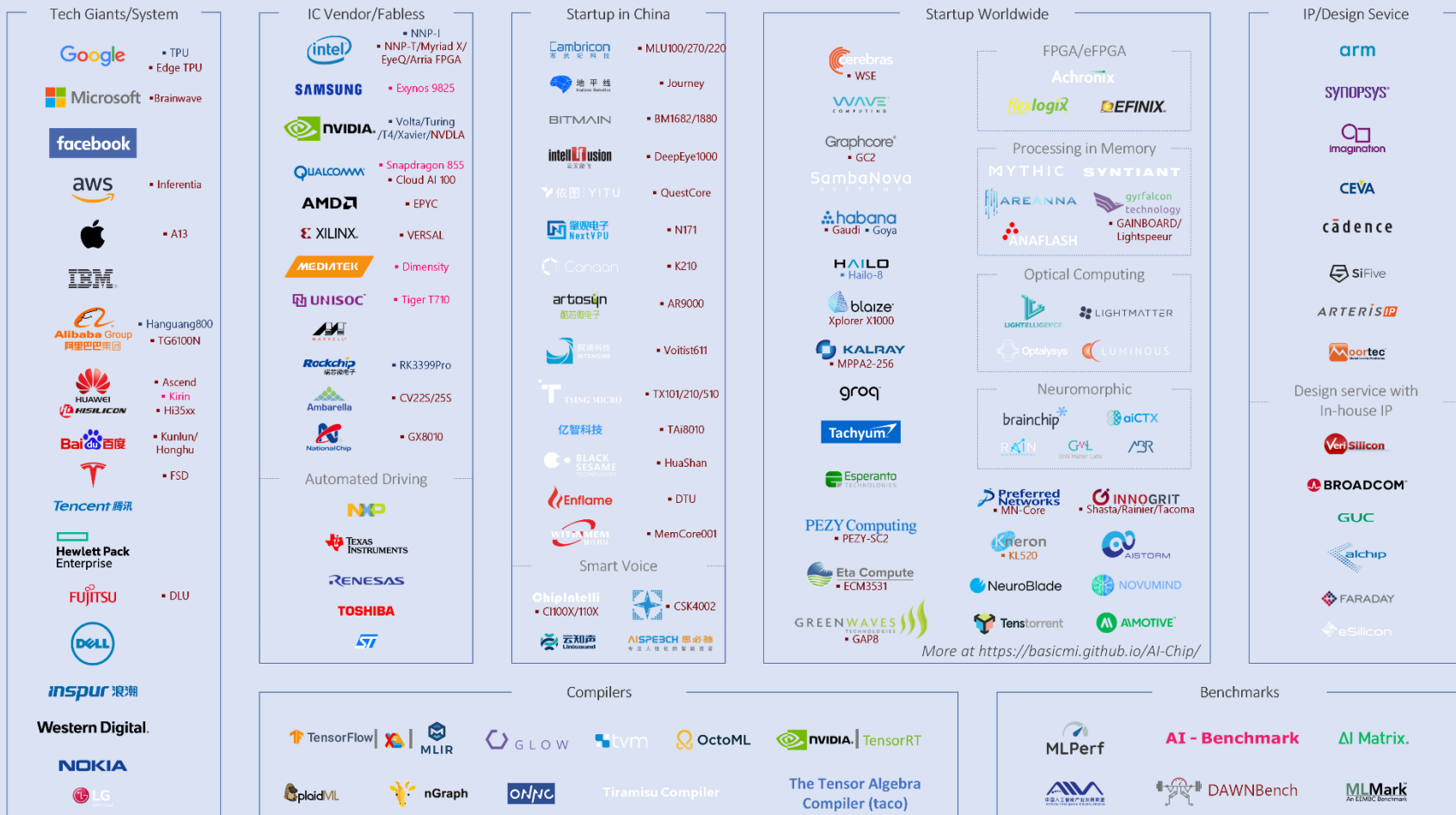
basicmi.github.io/AI-chip

■ MLPerf results available ■ AI-Benchmark results available

AI Chip Landscape

V0.7 Dec., 2019

S.T.

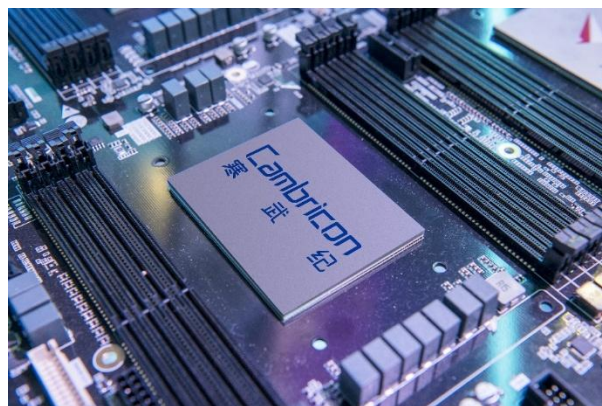
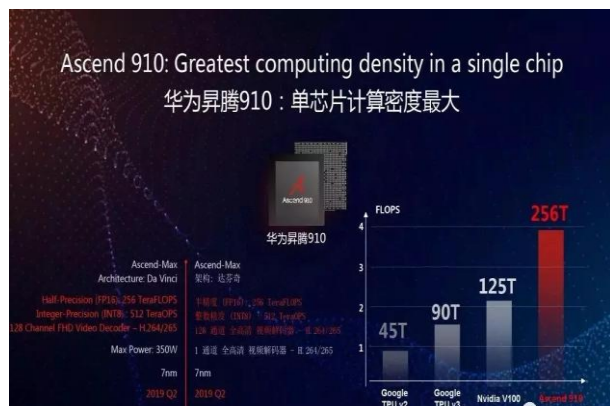


All information contained within this infographic is gathered from the internet and periodically updated, no guarantee is given that the information provided is correct, complete, and up-to-date.

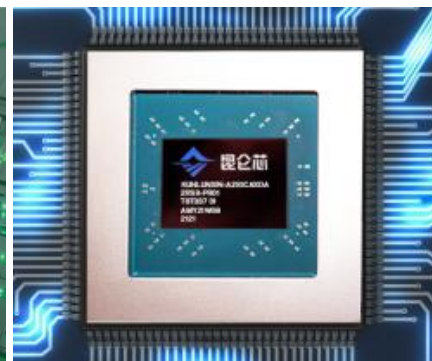
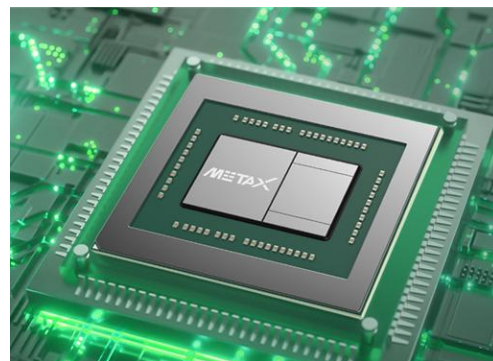
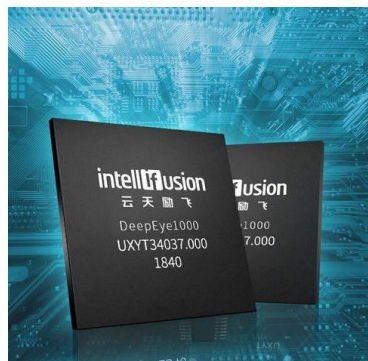
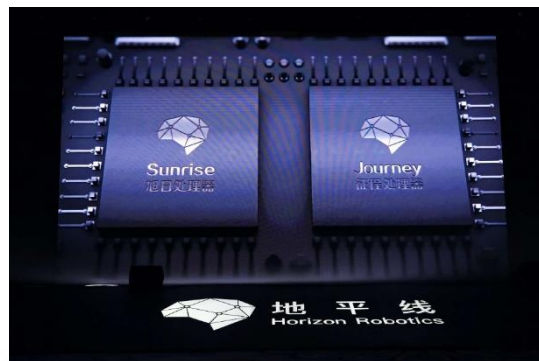


AI芯片

• 华为、寒武纪、海光



• 沐兮、昆仑芯、摩尔线程，天数等





国产AI生态



大模型新基建进入国家战略，建设从“模型-系统-芯片”闭环

算力

大模型庞大数据处理的先决条件
决定大模型迭代效率的主要影响因素

算法

大模型底层的运作原理方法
从理论到应用有一定的工程部署难度
需要持续迭代优化



数据

决定了大模型的生成结构
提升模型能力与准确率的前提
数据来源决定了模型的适用范围

生态

先发构筑产品生态、用户心智以及提升产品力



FlagOS, 统一AI系统软件栈

AI大模型

语言模型

DeepSeek, Qwen, Seed-oss, GPT-oss, Step, Grok, Llama, GLM, Hunyuan等

多模态模型

EMU, MiniCPM-v, Qwen2.5/3-VL, ERNIE4.5, Llava系列

具身智能模型

RoboBrain2, RoboBrainX0.5, Pai-0, Pai-0.5

FlagOS 生态使能项目

Megatron-FL

vLLM-plugin-FL

TransformerEngine-FL

深度学习框架

(PyTorch, PaddlePaddle, etc)

FlagOS 开源核心库

FlagGems
通用大模型算子库

FlagScale
训练推理并行框架

FlagTree
统一编译器

FlagCX
统一通信库

FlagOS 开源工具

KernelGen v1.0 算子自动生成工具

FlagRelease 自动迁移和发版工具

FlagPerf 多芯片评测工具

支持多种芯片架构

Nvidia, NPU, GPGPU, DSA, RISC-V
AI, ARM等

支持多种系统硬件形态

端侧计算模组

AI PC

AI 服务器

超节点系统

支持多种应用场景

数据中心
(train & Inference)

→ 机器人
(cloud-edge
cooperation) ←

边缘
(inference)



参考资料

- 中国科学院大学，《智能计算系统》，陈云霁等
- 哈尔滨工业大学，《AI系统》，苏统华等
- 北京大学，《计算机组成原理》，陆俊林
- 上海交通大学，《计算机体系结构》，李超
- 计算机教指委，《智能时代计算机专业系统能力培养纲要》，2021



中國人民大學
RENMIN UNIVERSITY OF CHINA

站在当下

AI需求正在驱动教育的一场变革

加入这场变革

我们还在路上

我们携手并肩